Project Report

On GoReserve: Bus Reservation System

Submitted for partial fulfillment of the requirement for the award of the degree of

Master of Computer Application

Session (2024-2026)

Department of Computer Science and Applications

Hindu Institute of Management, Sonipat-131001

Affiliated by:

Deenbandhu Chhotu Ram University of Science & Technology, Murthal (Sonipat), Haryana

Submitted by:

Vishal Kaushik
MCA II YEAR
Roll no: 24012541019

Under Supervision of:

Mr. Ajay Bajaj

Bell Blaze Technologies Pvt. Ltd.

Unit No. G 03
H-159, Ground Floor,
VDS Work Eutopia Business Park,
Sector-63, Noida - 201301

Date: 20-05-2026

TO WHOM IT MAY CONCERN

This is to inform you that **Mr. Vishal Kaushik**, a student of **Master of Computer Application** from **Deenbandhu Chhotu Ram University of Science and Technology**, is currently undergoing an internship at Bell Blaze Technologies Pvt Ltd, Noida.

The internship commenced on 1st Dec 2025 and is scheduled to conclude on 31st May 2026. During this period, the student is working as a Software Developer Intern in our Engineering Business Unit.

He has been actively involved in tasks related to the project titled **"GoReserve - Bus Reservation System"** and is gaining practical exposure to industry practices and technologies.

We confirm that the internship is ongoing as of the date of this letter.

Should you require any further information, please feel free to contact us.

Thank you.

Yours Sincerely,

Tanu Ahuja
HR & Admin Director
Bell Blaze Technologies Pvt Ltd

HINDU INSTITUTE OF MANAGEMENT,SONEPAT

(Approved by All India Council for Technical Education

&

Affiliated to Deenbandhu Chhotu Ram University of Science & Technology,
Murthal (Sonepat)

Date:-

CERTIFICATE

This is to certify that Mr. /Ms_____ S/O/D/O _____ Of MCA IV semester is a bonafide student of the Hindu Institute of Management in the session 2024-2026.

(Director)

Telephone: (0130) 2244474, 2257352, 2242596, 8901440704, 8901440705
E-mail : directorhimsonepat@gmail.com, Visit us at : www.himsonepat.org

Certificate from Department

This is to certify that the project report entitled "GoReserve - Bus Reservation System", submitted by Vishal Kaushik (Roll No: 24012541019) in partial fulfilment of the requirements for the award of the degree of Master of Computer Application from Hindu Institute of Management, Deenbandhu Chhotu Ram University of Science and Technology is an authentic record of the student's own work carried out under my guidance and supervision. The matter embodied in this report has not been submitted in part or full to any other University or Institute for the award of any degree or diploma.

Date: **Mr. Ajay Bajaj**

(Project Guide)

Declaration

I hereby declare that the project report entitled "GoReserve - Bus Reservation System" submitted for partial fulfillment of the requirements for the award of the degree of Master of Computer Application from Hindu Institute of Management, Deenbandhu Chhotu Ram University of Science and Technology, is my original work and the project report has not formed the basis for the award of any degree, diploma, fellowship or similar title.

Date:

Signature:

Name: Vishal Kaushik

Acknowledgement

I would like to express my profound gratitude to everyone who supported me throughout the course of this project, "GoReserve - Bus Reservation System".

First and foremost, I would like to thank my project guide, Mr. Ajay Bajaj, for their invaluable guidance, continuous encouragement, and constructive feedback. Their technical insights were instrumental in shaping the backend architecture, database schema, and the secure implementation of the Java Spring Boot and Thymeleaf application.

I also extend my sincere thanks to the faculty members of my department and Hindu Institute of Management for providing a solid academic foundation and the necessary resources to bring this full-stack project to life.

PLAGIARISM VERIFICATION Cum CERTIFICATE

Date:

9.1 Name of Academic Program (Ph.D/M.Tech/M.Sc./M.Arch./MBA/MHA/MCA/ Other PG programs) : .....

Title of Dissertation/Thesis/Project: .....

Total Pages: .....

Roll No/Regn No of Student: .....

Name of Student : .....

Department/Subject: .....

Name of Research Supervisor/Co-Supervisor(s) : .....

This is to certify that the above thesis was scanned for similarity detection process and outcome is given below:

Software used: TURNITIN

Similarity Index: Total Word Count:

9.2 Signature of Student Signature of Supervisor

Verified as per clause 4 of plagiarism ordinance & modalities approved by competent authority.

Software used: TURNITIN

Similarity Index:

Attach report generated by Turnitin

Nodal Officer of Deptt. Chairperson of the Deptt.

ProjectFileGoReserve.pdf

by Turnitin Official

Submission date: 15-May-2026 04:56PM (UTC+0900)
Submission ID: 2913795695
File name: ProjectFileGoReserve.pdf (3.83M)
Word count: 16987
Character count: 95473

ProjectFileGoReserve.pdf

ORIGINALITY REPORT

9%	3%	1%	6%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to Islington College,Nepal Student Paper	2%
2	moldstud.com Internet Source	<1%
3	Submitted to University of Wales Institute, Cardiff Student Paper	<1%
4	ijircce.com Internet Source	<1%
5	Submitted to Glyndwr University Student Paper	<1%
6	Submitted to Middlesex University Student Paper	<1%
7	Submitted to Arab Open University Student Paper	<1%
8	Submitted to Melbourne Institute of Technology Student Paper	<1%
9	eprints.utar.edu.my Internet Source	<1%
10	Submitted to University of Leicester LTI Student Paper	<1%
11	Gaikwad, Purva Arun. "Development of Web Platform for Cyanobacteria Research by Integrating Industry and Academic Practices	<1%

	for Software Development", University of Cincinnati, 2024 Publication	
12	Submitted to Strategy First Institute Student Paper	< 1 %
13	ijsrem.com Internet Source	< 1 %
14	code-b.dev Internet Source	< 1 %
15	www.coursehero.com Internet Source	< 1 %
16	Submitted to Asia Pacific University College of Technology and Innovation (UCTI) Student Paper	< 1 %
17	Submitted to De Montfort University Student Paper	< 1 %
18	eluminoustechnologies.com Internet Source	< 1 %
19	Submitted to Nottingham Trent University Student Paper	< 1 %
20	Submitted to Al Ain University Student Paper	< 1 %
21	Submitted to Frederick University Student Paper	< 1 %
22	Submitted to Kakatiya Institute of Technology and Science Student Paper	< 1 %
23	retailcard-activation.com Internet Source	< 1 %
24	Submitted to Swinburne University of Technology	< 1 %

	Student Paper	
25	Submitted to Kent Institute of Business and Technology Student Paper	<1%
26	Submitted to Moodle2025 Student Paper	<1%
27	Submitted to University College Birmingham Student Paper	<1%
28	Submitted to University of Bradford Student Paper	<1%
29	irigs.iiu.edu.pk:64447 Internet Source	<1%
30	Submitted to ESoft Metro Campus, Sri Lanka Student Paper	<1%
31	Howard Anderson, Sharon Yull. "BTEC Nationals - IT Practitioners", Routledge, 2019 Publication	<1%
32	Submitted to Poornima University Student Paper	<1%
33	www.atlantis-press.com Internet Source	<1%
34	Submitted to CTI Education Group Student Paper	<1%
35	Submitted to Coventry University Student Paper	<1%
36	Submitted to Taylor's Education Group Student Paper	<1%
37	Submitted to Universiti Teknologi Petronas Student Paper	<1%
38	Submitted to University of Greenwich Student Paper	<1%

39	Submitted to University of Queensland Student Paper	<1%
40	duffion.com Internet Source	<1%
41	eprints.mdx.ac.uk Internet Source	<1%
42	advaiya.com Internet Source	<1%
43	data.mitsgwalior.in Internet Source	<1%
44	nino-leet.top Internet Source	<1%
45	users.utcluj.ro Internet Source	<1%
46	Submitted to Bannari Amman Institute of Technology Student Paper	<1%
47	Submitted to Management Development Institute Of Singapore (MDIS) LTI 1.3 Student Paper	<1%
48	Submitted to Thakur College of Engineering and Technology (TCET) Student Paper	<1%
49	Submitted to Uttaranchal University, Dehradun Student Paper	<1%
50	William Wolfgang Arrasmith. "Handbook of Systems Engineering and Analysis of Electro-Optical and Infrared Systems - Applications, Tools, and Techniques", CRC Press, 2025 Publication	<1%

51	spectrum.library.concordia.ca Internet Source	<1%
52	techbehindit.com Internet Source	<1%
53	Submitted to Universiti Malaysia Pahang Al-Sultan Abdullah (UMPSA) Student Paper	<1%
54	fairsarchive.org.uk Internet Source	<1%
55	indah.ump.edu.my Internet Source	<1%
56	www.filehorse.com Internet Source	<1%
57	www.researchgate.net Internet Source	<1%
58	Phillip A. Laplante. "Requirements Engineering for Software and Systems", Auerbach Publications, 2019 Publication	<1%

Exclude quotes	Off	Exclude matches	Off
Exclude bibliography	On		

ABSTRACT

GoReserve – Online Bus Reservation System

GoReserve is a web-based Online Bus Reservation System developed to simplify and modernize the process of booking bus tickets. The project is designed using Java Spring Boot, Thymeleaf, Spring Security, Hibernate, and MySQL to provide a secure, responsive, and user-friendly platform for both passengers and administrators.

The main objective of the system is to reduce the difficulties of traditional bus ticket booking methods by providing an online solution where users can easily search buses, view routes, book tickets, make payments, and manage booking history from anywhere. The system allows passengers to find buses based on source and destination, select seats, enter passenger details, and confirm bookings efficiently.

GoReserve includes separate modules for users and administrators. The user module provides functionalities such as registration, login, bus searching, ticket booking, payment processing, booking history management, and ticket cancellation. The admin module allows administrators to manage buses, monitor users, and control overall system operations. Spring Security is implemented to ensure secure authentication and role-based authorization for different users.

The project also focuses on improving user experience by integrating responsive web design, dark mode support, and ticket-style booking history pages. Payment records and booking details are stored securely in the database, ensuring proper management and data consistency.

The system is developed using JDK 21 and follows the MVC (Model-View-Controller) architecture, making the application organized, maintainable, and scalable. GoReserve demonstrates the practical implementation of full-stack web development concepts and provides an efficient digital solution for modern bus reservation management.

Table of Content

List of Tables

List of Figures

CHAPTER – 1

INTRODUCTION TO PROJECT

1.1 About Project:

GoReserve is a web-based bus reservation system that offers a simple, efficient, and user-friendly way to book bus tickets online. The main goal of the project is to replace the traditional offline ticket booking process with a digital solution that saves time and cuts down on manual work.

Users can register, log in securely, and search for available buses by entering their start and end points. After picking a suitable bus, users can book tickets by entering passenger details. Once the booking is confirmed, they can view their booking history in a clear ticket format. The system also includes a cancellation feature, allowing users to cancel tickets and get a notification about the refund process.

On the admin side, the system has a panel that lets administrators manage buses and users easily. Admins can add new buses, view existing bus details, and maintain control of the overall system.

The project uses modern technologies like Spring Boot for backend development, Thymeleaf for frontend rendering, and MySQL for database management. It implements security with Spring Security to ensure safe authentication and role-based access control.

GoReserve follows the MVC (Model-View-Controller) architecture, which helps organize the code and improve maintainability. The system is designed to be responsive and user-friendly, with additional features like dark mode to enhance the user experience.

Overall, GoReserve shows how modern web technologies can create a reliable and efficient online reservation system.

1.2 Problem with Existing System:

Many bus ticket booking systems still rely on manual processes or outdated platforms. This creates challenges for both passengers and service providers. Traditional ticket booking methods force users to visit ticket counters in person. This leads to long lines, wasted time, and frustration. There is also a higher chance of human errors in keeping records, which can cause incorrect bookings or data loss.

Current systems often don’t have user-friendly interfaces or real-time information. This makes it hard for users to find buses based on their preferred routes. Managing bookings, cancellations, and payments

manually becomes complicated and inefficient. Users also struggle to access their booking history or get timely updates about their tickets.

From the administrative side, managing bus details, user data, and bookings without a centralized system is time-consuming and error-prone. There is also a lack of secure authentication systems, leading to unauthorized access and data security problems

There is a need for a reliable, secure, and easy-to-use online bus reservation system. This system should automate the booking process, lower manual effort, and improve service for users. It should allow for easy searching, booking, cancellation, and management of tickets while ensuring data accuracy and security.

GoReserve has been developed to tackle these challenges and provide a digital solution that streamlines bus reservation management.

1.3 Key Benefits:

The GoReserve system offers several important benefits for users and administrators by improving the bus reservation process.

One main advantage is time efficiency. Users can book tickets online from anywhere, eliminating the need to visit a ticket counter. This saves time and effort. The system also provides easy access to information, letting users quickly search for buses based on their starting point and destination.

Another key benefit is accuracy and reliability. The automated system reduces human errors in booking and record management. All booking details are stored securely in the database, making it simple to retrieve information when needed.

The project ensures secure authentication using Spring Security, which protects user data and prevents unauthorized access. The system also includes a booking history feature, allowing users to view their past and current bookings in a clear ticket format.

For administrators, the system simplifies management tasks, such as adding buses and monitoring user activity. This leads to better control and improved efficiency in handling operations.

Features like ticket cancellation with refund notifications and a user-friendly interface with dark mode further improve the user experience.

Overall, GoReserve makes the bus reservation process faster, more secure, and more convenient.

1.4 Various Functionalities of Project:

The GoReserve project stands out from basic bus reservation systems due to several unique design choices and user-focused features. These enhancements improve both system performance and user experience.

1.4.1 Simplified Database Design

One of the most important unique aspects of this project is the absence of a separate route table. Instead of creating an additional entity for routes, the system directly stores source and destination information within the Bus and Booking data. This approach simplifies the database structure, reduces complexity, and makes data handling more efficient while still supporting effective bus search functionality.

1.4.2 Ticket-Style Booking Interface

The project provides a ticket-style layout for displaying booking history. Instead of showing plain text records, bookings are presented in a structured format similar to real bus tickets. This improves readability and gives users a more interactive and realistic experience.

1.4.3 Role-Based Authentication and Redirection

GoReserve implements a secure login system using role-based authentication. After logging in, users are redirected to different pages based on their roles. Admins are directed to the admin dashboard, while normal users are redirected to the user interface. This ensures proper access control and enhances system security.

1.4.4 Cancellation

The system includes a user-friendly ticket cancellation feature. When a user cancels a booking, a clear message is displayed stating that the refund will be processed within a specific time period. This feature improves transparency and builds user trust in the system.

1.4.5 Dark Mode Feature

A modern dark mode option is included to enhance user experience. It allows users to switch between light and dark themes based on their preference, making the interface more comfortable to use, especially in low-light conditions.

1.4.6 Responsive and User-Friendly Design

The system is designed to be responsive and easy to use across different devices. The clean layout, intuitive navigation, and well-structured pages ensure that users can interact with the system smoothly without confusion.

1.5 Objectives of Project:

The primary objective of the GoReserve project is to design and develop a reliable, efficient, and user-friendly web-based bus reservation system that simplifies the entire process of ticket booking and management. The system aims to transform the traditional manual booking method into a modern digital platform that is accessible anytime and from anywhere.

One of the key objectives of this project is to provide users with a **simple and efficient bus search functionality**, where they can easily find available buses by entering their source and destination. This helps users save time and reduces the effort required to gather travel information.

Another important objective is to enable users to **book tickets online in a secure and convenient manner**. The system allows users to enter passenger details, confirm bookings, and access their booking information at any time. It also provides a **booking history feature**, which helps users keep track of their previous and current reservations in an organized ticket format.

The project also focuses on implementing a **secure authentication and authorization system** using Spring Security. This ensures that user data is protected and access to system features is controlled based on roles such as admin and user. Maintaining data privacy and system security is one of the core goals of this project.

From an administrative perspective, the system aims to **simplify management operations**. Admins can easily add new buses, view existing records, and manage users efficiently through a centralized platform. This reduces manual work and minimizes the chances of errors.

Additionally, the project aims to improve overall user experience by providing features like **easy ticket cancellation with refund notification**, a **responsive interface**, and **dark mode support**.

Overall, the objective of GoReserve is to create a complete, secure, and scalable solution that enhances the efficiency, accuracy, and convenience of the bus reservation process for both users and administrators.

1.6 Scope of Project:

The scope of the GoReserve project includes developing an online system for bus reservation with basic facilities available for users and administrators. The project deals with the complete cycle of booking, managing and simple administration of bus tickets in an efficient manner.

The features available for the users will be login, registration, search bus by source and destination, list available buses, book bus tickets by entering passenger details, show history, cancel bus tickets, notify with refund policy.

Features available for the administrators will be managing buses, managing users (add buses, show bus details, see user activities) in a centralized system.

Advanced features like real-time bus tracking, seat booking and using external payment gateways have not been considered for the scope of this project but will be incorporated in future phases.

Technically the project includes backend development with Spring Boot, front end with Thymeleaf and the database is maintained using MySQL. The system has Spring Security for providing authentication and authorization.

All features are developed with the goal of maintaining a functional, safe, scalable web application system for bus reservation.

1.7 Assumptions & Constraints:

The development and implementation of the GoReserve bus reservation system are based on certain assumptions and constraints that define the scope, functionality, and limitations of the project. These factors help in setting realistic expectations and provide a clear understanding of the system's operational boundaries.

1.7.1 Assumptions

The system assumes that users have basic knowledge of operating web applications, such as registering, logging in, and navigating through different pages. It is also assumed that users have access to a stable internet connection, as the application is entirely web-based and requires online access for all operations.

Another important assumption is that users will provide accurate and valid information while registering and booking tickets. The system relies on user-input data for storing booking details, and incorrect information may lead to issues in booking management.

It is also assumed that the administrator will manage the system responsibly. The admin is expected to regularly update bus details, monitor bookings, and ensure that the system data remains accurate and up to date. Additionally, the system assumes that the server environment, database, and application services will function smoothly without major technical failures or downtime.

Furthermore, it is assumed that the system will be used in a controlled environment with a moderate number of users, as it is primarily developed for academic purposes.

1.7.2 Constraints

The GoReserve system has several constraints that limit its functionality. One of the major constraints is the absence of integration with a real-time payment gateway. The current system simulates payment functionality, and actual online transactions are not implemented.

Another limitation is that advanced features such as seat selection, real-time bus tracking, and automated notifications (SMS or email) are not included in this version of the project. These features can be added in future enhancements.

The system is highly dependent on internet connectivity, and any network issues may affect performance and accessibility. Additionally, the application is designed for a limited scale and may not efficiently handle a very large number of concurrent users without further optimization.

Finally, the project is developed under academic constraints, including limited development time, resources, and infrastructure. These limitations have influenced the scope and feature set of the system.

1.8 Organization of Report:

This report for the project is presented in a structured format in separate chapters according to the need of the project. The report is organized as follows:
Chapter 1: Introduction describes the whole project overview which is divided into the introduction to the system, problem definition, objective of the project, scope of the project and overall description of the report.
Chapter 2: Literature Review / Existing System is about the existing system, literature survey for the already existing system or the related previous system, also the shortcomings of the existing system are stated.

Chapter 3: Proposed System / Methodology describes the proposed solution of the system, designing and working of the system GoReserve, its architecture and methodology used and its benefits.

Chapter 4: System Analysis & Design describes about system requirement analysis including functional requirements, non-functional requirements and various types of diagram which includes Use Case diagram, DFD, ER diagrams and Database Design.

Chapter 5: Implementation gives the information about the tool used, technology used and the various components of the project and its screen-shots.

Chapter 6: Testing describes the testing and various type of testing done to make sure that the system is working as expected. It also shows the test cases and results obtained.

Chapter 7: Result and Discussion shows the result of the project done and performance of the project and its features.

Chapter 8: Conclusion and Future Scope summarizes the whole project, gives its limitations and possible future developments for the system.

Last part includes the reference and the appendix.

CHAPTER-2

Literature Review / Existing System

2.1 Introduction to Literature Review:

A literature review is an important part of any project as it helps in understanding the work that has already been done in a particular field. It gives an idea about existing systems, technologies, and approaches used by others to solve similar problems. By studying these systems, we can identify their strengths and weaknesses, which helps in building a better and more improved solution.

In the case of the GoReserve project, the literature review focuses on studying different types of reservation systems, especially those related to transportation such as bus, train, and flight booking systems. Over time, many systems have been developed to make the booking process easier and faster. These systems allow users to check availability, book tickets, and manage their reservations online.

This chapter mainly discusses how reservation systems have evolved from traditional manual methods to modern online platforms. It also includes a review of existing systems, the technologies used in them, and the common features they provide. Along with this, the limitations of these systems are also analyzed to understand what problems still exist.

By studying all these aspects, we get a clear idea of what is missing in current systems and what improvements can be made. This helps in designing a better system like GoReserve, which aims to provide a simple, efficient, and user-friendly bus reservation experience.

Another important aspect of reviewing existing work is learning from the mistakes and limitations of earlier systems. Some systems may have good features but lack proper user interface design, while others may be secure but difficult to use. By analyzing both the positive and negative aspects, it becomes easier to design a system that balances functionality, usability, and performance.

2.2 Evolution of Reservation Systems:

Reservation systems have changed a lot over time with the advancement of technology. Earlier, booking tickets was a completely manual process, but today it has become fast, digital, and easily accessible. This evolution has played an important role in improving the overall experience of users.

In the beginning, ticket booking was done through **manual methods at ticket counters**. People had to visit bus stations or travel agencies to book their tickets. This process was time-consuming and often involved long queues. There was also a higher chance of errors, as all records were maintained manually. Managing bookings and cancellations was difficult, and users had very limited options.

After that, the concept of **telephone booking** was introduced. In this system, users could call a booking office and reserve their tickets. This reduced the need to visit ticket counters physically, but it still had limitations. The process depended heavily on the availability of operators, and there was no proper system to track bookings efficiently.

With the advancement of technology, **early computer-based reservation systems** were developed. These systems allowed staff to manage bookings digitally using computers. Data storage became more organized, and searching for records became faster. However, these systems were still not directly accessible to users and required trained personnel to operate them.

The next major step was the introduction of **online reservation websites**. These platforms allowed users to book tickets directly through the internet. Users could search for buses, check availability, and make bookings from their homes. This greatly improved convenience and reduced dependency on manual systems. Online systems also introduced features like booking history and cancellation options.

In recent years, reservation systems have further evolved into **mobile applications**. With the increasing use of smartphones, users can now book tickets anytime and anywhere. Mobile apps provide a faster, more user-friendly experience with additional features like notifications, real-time updates, and easy payment options.

2.3 Review of Related Work:

In recent years, online booking systems have become an important part of the transportation and travel industry. Many applications and websites have been developed to simplify the process of ticket booking for buses, trains, and flights. These systems are designed to provide users with convenience, speed, and easy access to booking services without the need to visit physical ticket counters.

Most online booking systems follow a similar working pattern. Users are required to register and log in to the system, after which they can search for available services by entering details such as source, destination, and travel date. Once the results are displayed, users can select a suitable option and proceed

with the booking process. These systems have made ticket booking faster and more efficient compared to traditional methods.

One of the key features of modern booking systems is the **search functionality**. It allows users to quickly find available buses or other transport options based on their requirements. Advanced systems also provide filters such as price range, timing, and availability, which help users make better decisions.

Another important feature is the **online booking and reservation system**, where users can enter passenger details and confirm their tickets instantly. This eliminates the need for manual intervention and reduces the chances of errors. Along with this, most systems provide a **booking history feature**, which allows users to view their past and current reservations.

The **payment system** is also a major part of online booking platforms. Many systems integrate secure payment gateways that support multiple payment methods such as credit cards, debit cards, and digital wallets. This ensures a smooth and secure transaction process for users.

From a technical perspective, many modern reservation systems are developed using frameworks like **Spring Boot**, which helps in building scalable and efficient backend applications. For frontend development, technologies like HTML, CSS, JavaScript, and templating engines such as **Thymeleaf** are commonly used. These tools help in creating dynamic and user-friendly interfaces.

For database management, systems often use relational databases like **MySQL**, which store user data, booking details, and transaction records in an organized manner. Security is also an important aspect, and many systems implement authentication mechanisms using frameworks like **Spring Security** to protect user data and prevent unauthorized access.

Although these systems provide many useful features, they can sometimes become complex and difficult to manage. Some platforms focus heavily on advanced features but ignore simplicity and user experience.

2.4 Study of Existing Systems:

Before developing any new system, it is important to study the systems that already exist. This helps in understanding how current booking systems work, what features they provide, and what problems users face while using them. In the case of reservation systems, both offline and basic online systems have been widely used over time.

2.4.1 Offline Reservation System

The offline reservation system is the traditional method of booking tickets. In this system, users have to visit bus stations or travel agencies physically to book their tickets. The entire process is handled manually by staff members. They check availability, record passenger details, and issue tickets.

Although this system has been used for many years, it has several drawbacks. It is time-consuming because users have to stand in long queues, especially during peak travel seasons. There is also a higher chance of human errors, such as incorrect data entry or loss of records. Managing cancellations and refunds is also difficult in this system.

Another limitation is the lack of transparency. Users cannot easily check availability or compare different options. They depend completely on the staff for information. Overall, the offline system is less efficient and not suitable for modern needs.

2.4.2 Basic Online Reservation Systems

With the growth of the internet, basic online reservation systems were introduced to overcome the limitations of offline systems. These systems allow users to book tickets through websites using computers or mobile devices.

In basic online systems, users can register, log in, and search for available buses by entering details such as source and destination. After selecting a bus, users can proceed with booking by entering passenger details. Some systems also provide features like booking history and cancellation options.

These systems are more convenient compared to offline methods, as users can book tickets from anywhere without visiting a physical location. They also reduce manual work and improve data accuracy.

However, many basic online systems still have limitations. Some systems have complex interfaces that are difficult to use. Others may lack proper security features or do not provide real-time updates. In some cases, the performance of the system depends heavily on internet speed.

2.4.3 Working of Existing Systems

In earlier years, reservation systems were not as advanced as modern web applications. The working of these systems was mostly based on manual processes and simple computer-based applications. These systems were designed mainly for internal use by staff and were not directly accessible to customers.

In the **offline reservation system**, the process started when a customer visited a ticket counter. The staff would check the availability of seats using registers or basic computer records. If seats were available, the staff manually entered the passenger details into a register or a simple software system. After that, a ticket was issued to the customer. All records were maintained either on paper or in basic computer files, which made the process slow and difficult to manage.

With the introduction of computers, some organizations started using **desktop-based reservation systems**. These systems were developed using older programming languages such as **C, C++, and early versions of Java**. The user interface was very simple and not user-friendly. These applications were installed on specific computers and could not be accessed remotely.

In these systems, the staff would enter booking details into the software, and the data was stored in local databases or files. There was no real-time synchronization, meaning that data was not updated instantly across different locations. This sometimes led to issues like double booking or incorrect availability information.

Networking was also limited during that time. Some systems used **basic client-server architecture**, but they required a local network and were not connected to the internet. As a result, users had to depend completely on the staff for booking and information.

Overall, the working of existing systems in earlier times was slow, less efficient, and highly dependent on manual effort. These limitations created the need for more advanced and user-friendly online reservation systems.

2.5 Technologies Used in Existing Systems:

Earlier reservation systems were developed using basic technologies that were available at that time. These systems were mainly designed for internal use and did not provide direct access to users. The technologies used were simple, less interactive, and had limited capabilities compared to modern systems.

2.5.1 Programming Languages

Most early reservation systems were developed using programming languages such as **C and C++**. These languages were used to build desktop-based applications that could run on specific computers. In later stages, early versions of Java were also introduced, which helped in developing slightly more advanced applications.

However, these programs were not very user-friendly and required trained staff to operate them. The focus was more on functionality rather than user experience.

2.5.2 User Interface (UI)

The user interface of these systems was very basic. Most applications used **command-line interfaces (CLI)** or simple text-based screens. In some cases, basic graphical user interfaces (GUI) were used, but they were not visually appealing or easy to navigate.

Users could not interact with these systems directly. Only staff members could operate them, which limited accessibility.

2.5.3 Database Systems

Data in early reservation systems was stored using **file-based systems** or simple databases. Instead of advanced database management systems, many applications used flat files or basic storage methods to save booking records.

Later, some systems started using early relational databases, but they were limited in functionality and not optimized for handling large amounts of data.

2.5.4 Networking Technology

Networking capabilities in old systems were very limited. Most systems were **standalone applications**, meaning they worked on a single computer without any connection to other systems.

In some cases, **local area networks (LAN)** were used to connect multiple computers within the same office. However, these systems were not connected to the internet, so remote access was not possible.

2.5.5 Security Mechanisms

Security in older systems was minimal. Basic login systems were used, but there was no strong encryption or advanced authentication methods. This made the system less secure compared to modern applications.

2.6 Comparative Analysis of Reservation Systems:

2.6.1 Traditional System VS Online Reservation System

Feature	Traditional System (Old)	Online Reservation System
Booking Method	Manual booking at counters	Online booking through website/app
Time Required	High (long queues)	Low (quick process)
Accessibility	Limited to specific location	Accessible anytime, anywhere
User Interaction	Staff-dependent	Direct user interaction
Accuracy	Low (human errors possible)	High (automated system)
Data Storage	Paper records or local files	Digital database systems
Availability Check	Manual checking	Real-time availability
Payment System	Cash-based	Online payment options
Booking History	Not easily available	Easily accessible
Cancellation Process	Difficult and slow	Simple and quick
User Interface	Basic or no interface	User-friendly interface
Security	Low security	High security with authentication
Scalability	Limited	Highly scalable

Table 2.1

2.6.2 Explanation of Comparison

In traditional reservation systems, the entire booking process was manual and required users to visit ticket counters. This made the process slow and inconvenient, especially during peak times when long queues were common. In contrast, online reservation systems allow users to book tickets from anywhere, reducing time and effort significantly.

Another major difference is accessibility. Traditional systems were limited to specific locations and working hours, whereas online systems are available 24/7. This makes modern systems much more flexible and user-friendly.

Accuracy is also greatly improved in online systems. Manual systems often led to errors in data entry, while automated systems ensure that booking information is stored and processed correctly. Similarly, data storage has evolved from paper-based records to organized digital databases, making it easier to manage and retrieve information.

Payment methods have also improved. Earlier systems mainly relied on cash transactions, but modern systems provide multiple online payment options, making transactions faster and more secure.

In addition, features like booking history, cancellation options, and real-time availability were either missing or difficult to manage in traditional systems. Online systems have simplified these processes and made them more transparent.

2.7 Limitations of Existing Systems:

Although reservation systems have improved over time, earlier systems (especially traditional and basic online systems) had several limitations. These issues made the booking process less efficient and created inconvenience for users as well as administrators.

2.7.1 Time Consuming Process

One of the major limitations of existing systems was that they were very time-consuming. In traditional systems, users had to visit ticket counters physically and wait in long queues to book their tickets. This process was slow and frustrating, especially during busy seasons or holidays.

Even in early computer-based systems, the process was not very fast because it depended on staff to handle each request manually. There was no instant access to booking services, which increased the overall time required to complete a reservation.

2.7.2 High Chances of Errors

Another important issue was the high possibility of human errors. In manual systems, staff had to write or enter data themselves, which could lead to mistakes such as incorrect names, wrong travel details, or duplicate bookings.

Since data was often stored in registers or simple files, it was difficult to correct errors once they were made. These mistakes could create serious problems for both users and system administrators.

2.7.3 Lack of Real-Time Information

Older systems did not provide real-time updates. Seat availability was not updated instantly, which sometimes led to confusion or double booking. Users had to depend on staff for information, and the data provided was not always accurate.

This lack of real-time data made it difficult to plan travel efficiently and reduced the reliability of the system.

2.7.4 Poor User Interface

The user interface of existing systems was very basic and not user-friendly. In many cases, systems used text-based or simple screens that were difficult to understand. Only trained staff could operate these systems, which limited accessibility for general users.

There was no focus on user experience, which made the system less convenient and harder to use.

2.7.5 Security Issues

Security was another major limitation of older systems. There were no strong authentication or encryption mechanisms to protect user data. Basic login systems were used, but they were not secure enough to prevent unauthorized access.

Sensitive information such as personal details and booking records could be easily misused or lost, making the system unreliable.

2.8 Need for Proposed System:

From the study of existing reservation systems, it is clear that although these systems have helped in improving the booking process, they still suffer from several limitations. Issues such as time-consuming procedures, lack of real-time updates, poor user interface, and weak security make these systems less efficient and inconvenient for users.

In traditional systems, users had to depend on ticket counters and staff for booking, which created delays and reduced accessibility. Even basic online systems, though better than manual methods, often lacked simplicity and user-friendly design. Many systems were complex to use, making it difficult for users to navigate and complete bookings easily.

Due to these limitations, there is a strong need for a system that is simple, fast, secure, and easily accessible. A modern reservation system should allow users to search for available buses, book tickets, and manage their bookings without any confusion or delay. It should also provide accurate and real-time information to avoid issues like incorrect availability or double booking.

The proposed system, GoReserve, is developed to overcome these problems and provide an improved solution. It focuses on creating a user-friendly interface that makes the booking process simple and efficient. The system also ensures better data management, improved accuracy, and secure access through proper authentication mechanisms.

In addition, GoReserve aims to provide features like booking history, easy cancellation, and a smooth user experience. By addressing the limitations of existing systems, the proposed system offers a more reliable and convenient platform for bus reservation.

Chapter 3

Proposed System / Methodology

3.1 Introduction to Proposed System:

The limitations of existing reservation systems highlight the need for a better and more efficient solution. Traditional methods were time-consuming and required manual effort, while some early online systems were complex and not very user-friendly. These issues created the need for a system that is simple, fast, and reliable.

The proposed system, GoReserve, is designed to overcome these problems by providing a web-based platform for bus ticket booking. It allows users to easily search for buses, book tickets, and manage their bookings without any difficulty. The system focuses on improving user experience by offering a clean interface and smooth navigation.

Unlike older systems, the proposed system is accessible anytime and from anywhere, making it more convenient for users. It also ensures better accuracy by automating the booking process and reducing human errors. Security is another important aspect, and the system provides proper authentication to protect user data.

In addition, the proposed system simplifies the work of administrators by providing tools to manage buses and users efficiently. All data is stored in an organized manner, making it easy to access and maintain.

In addition to this, the system is designed to provide a smooth and hassle-free experience for users by reducing unnecessary steps in the booking process. It focuses on making the interaction simple so that even first-time users can easily understand and use the system without any difficulty.

The proposed system also aims to improve overall reliability by ensuring that data is stored properly and can be accessed whenever required. This helps in maintaining consistency and avoids issues like data loss or incorrect information.

3.2 Description of the Proposed System:

The proposed system, GoReserve, is a web-based application designed to simplify and improve the process of bus ticket booking. It provides an easy-to-use platform where users can search for available

buses, book tickets, and manage their bookings without any confusion. The system is developed to overcome the limitations of traditional and existing reservation systems by offering a more efficient and user-friendly solution.

In this system, users first need to register and log in securely. Once logged in, they can search for buses by entering their source and destination. The system then displays a list of available buses based on the entered details. Users can select a suitable bus and proceed with the booking process by entering passenger information.

After confirming the booking, the system stores all the details in the database and generates a record that can be accessed later. Users can view their booking history at any time, which helps them keep track of their previous and current reservations. The system also provides a cancellation feature, allowing users to cancel their bookings easily. A message is displayed informing them about the refund process, which improves transparency and user trust.

From the administrative side, the system provides a separate interface for admins. Administrators can add new buses, update existing details, and manage users. This helps in maintaining proper control over the system and ensures that the data remains accurate and up to date.

One of the key aspects of the proposed system is its focus on simplicity. The interface is designed in a way that even users with basic computer knowledge can easily use it. Navigation is kept clear and straightforward, reducing confusion and improving overall user experience.

The system also ensures better performance by handling data efficiently. Since all operations are automated, the chances of errors are significantly reduced. In addition, proper authentication mechanisms are used to protect user information and prevent unauthorized access.

3.3 System Architecture:

The system architecture of GoReserve is designed in a structured way to ensure that the application works efficiently and is easy to manage. The project follows the **Model-View-Controller (MVC)** architecture, which helps in separating different parts of the application and makes development more organized.

In this architecture, the system is divided into three main components: Model, View, and Controller. Each component has a specific role, and together they handle the complete working of the application.

3.3.1 Model (Data Layer)

The Model represents the data and business logic of the system. It is responsible for storing and managing all the information related to users, buses, bookings, and payments.

In the GoReserve system, the model includes entities such as User, Bus, Booking, and Payment. These entities are connected to the database and help in storing and retrieving data whenever required. Whenever a user performs an action like booking a ticket, the data is processed and saved through the model.

The model ensures that all data is handled properly and remains consistent throughout the system.

3.3.2 View (User Interface Layer)

The View is the part of the system that users interact with. It is responsible for displaying information and collecting input from users.

In this project, the view is developed using web pages that provide a clean and user-friendly interface. Users can register, log in, search for buses, and book tickets through these pages. The design is kept simple so that users can easily understand how to use the system.

The view only displays data and does not handle any logic. It communicates with the controller to get the required information.

3.3.3 Controller (Logic Layer)

The Controller acts as a bridge between the Model and the View. It handles user requests, processes them, and returns the appropriate response.

When a user performs an action, such as searching for a bus or booking a ticket, the request is first sent to the controller. The controller then interacts with the model to fetch or update data and sends the result back to the view.

This separation of responsibilities makes the system more organized and easier to maintain.

3.3.4 Working of the Architecture

The working of the system follows a simple flow. First, the user interacts with the interface (View) by entering input. This input is sent to the Controller, which processes the request. The controller then communicates with the Model to retrieve or update data. Finally, the processed data is sent back to the View and displayed to the user.

This flow ensures smooth communication between different parts of the system and helps in maintaining a clear structure.

3.3.5 Benefits of MVC Architecture

Better organization of code

MVC separates the application into Model, View, and Controller, which makes the code clean and well-structured. This makes it easier to understand and manage.

Easy maintenance and updates

Since each part of the system is independent, changes can be made in one section without affecting others. This makes the system easier to maintain and update.

Supports teamwork and development

Different developers can work on different parts (UI, logic, data) at the same time. This speeds up development and improves productivity.

3.4 Methodology Used:

The development of the GoReserve system follows a structured and step-by-step approach to ensure that the final product is efficient, reliable, and easy to use. The methodology used in this project is based on an iterative and practical development process, where the system is built and improved in stages.

At the beginning of the project, the first step was **understanding the problem**. This included studying existing reservation systems and identifying their limitations, such as time-consuming processes, lack of real-time data, and poor user experience. This step helped in defining the requirements of the new system.

The next step was **requirement analysis**, where all the features of the system were clearly defined. This included both user and admin functionalities, such as registration, login, bus search, booking, and management. This step ensured that the system would meet all the necessary needs.

After that, the **system design phase** was carried out. In this phase, the overall structure of the system was planned. The MVC architecture was selected to organize the system properly. Database design was also considered to store user, bus, and booking information efficiently.

Once the design was completed, the project moved to the **implementation phase**. In this stage, the actual coding of the system was done. Different modules were developed step by step, such as the user module, booking module, and admin module. Each part was tested individually to ensure it worked correctly.

The next step was **testing and debugging**. The system was tested to check for errors and ensure that all functionalities were working as expected. Any issues found during testing were fixed to improve system performance and reliability.

Finally, the system was **reviewed and improved** based on testing results. Small changes and improvements were made to enhance usability and performance.

Overall, the methodology followed in this project ensures a smooth development process and helps in building a system that is efficient, reliable, and user-friendly.

3.5 Advantages of the Proposed System:

The proposed system, GoReserve, offers several advantages over traditional and existing reservation systems. It is designed to provide a better, faster, and more convenient experience for both users and administrators.

One of the main advantages of the system is **time efficiency**. Users can book tickets online without visiting any ticket counter. The entire process, from searching buses to confirming bookings, can be completed within a few minutes. This saves a lot of time and effort, especially during busy travel periods.

Another important advantage is **ease of use**. The system is designed with a simple and user-friendly interface, making it easy for users to navigate and perform actions like searching, booking, and cancellation. Even users with basic computer knowledge can use the system without difficulty.

The system also provides **accurate and reliable information**. Since all operations are handled automatically, the chances of human errors are reduced. Users get correct details about bus availability, booking status, and other information.

Accessibility is another key benefit. The system can be accessed anytime and from anywhere, as long as there is an internet connection. This makes it much more convenient compared to traditional systems that are limited by location and working hours.

GoReserve also improves **data management**. All information related to users, buses, and bookings is stored in an organized manner in the database. This makes it easy to retrieve and manage data whenever required.

From a security perspective, the system provides **secure authentication**. Only authorized users can access their accounts, which helps in protecting personal and booking information.

For administrators, the system simplifies management tasks. Admins can easily add buses, update details, and monitor the system. This reduces manual work and improves efficiency.

Chapter 4

System Analysis & Design

4.1 Requirement Analysis:

Requirement analysis is an important step in system development. It helps in identifying what the system should do and what features it must include. In the GoReserve project, requirement analysis is done to understand both user and admin needs so that the system can work efficiently.

Requirements are mainly divided into **functional requirements** and **non-functional requirements**.

4.1.1 Functional Requirements

Functional requirements describe what the system should do.

- The system should allow users to **register and log in** securely.
- Users should be able to **search buses** using source and destination.
- The system should display **available buses** based on user input.
- Users should be able to **book tickets** by entering passenger details.
- The system should store booking information and allow users to **view booking history**.
- Users should be able to **cancel bookings**.
- Admin should be able to **add, update, and manage buses**.
- Admin should be able to **manage users**.

4.1.2 Non-Functional Requirements

Non-functional requirements describe how the system should perform.

- The system should be **fast and responsive**.
- It should be **secure**, with proper authentication.
- The system should be **easy to use** with a simple interface.
- It should be **reliable**, with minimal errors.
- The system should be **scalable** for future improvements.

4.2 Use Case Diagram:

A use case diagram shows how users interact with the system.

4.2.1 Main Actors

- User
- Admin

4.2.2 User Actions

- Register
- Login
- Search Bus
- Book Ticket
- View Booking
- Cancel Booking

4.2.3 Admin Actions

- Login
- Add Bus
- Manage Bus
- Manage Users

DIAGRAM:

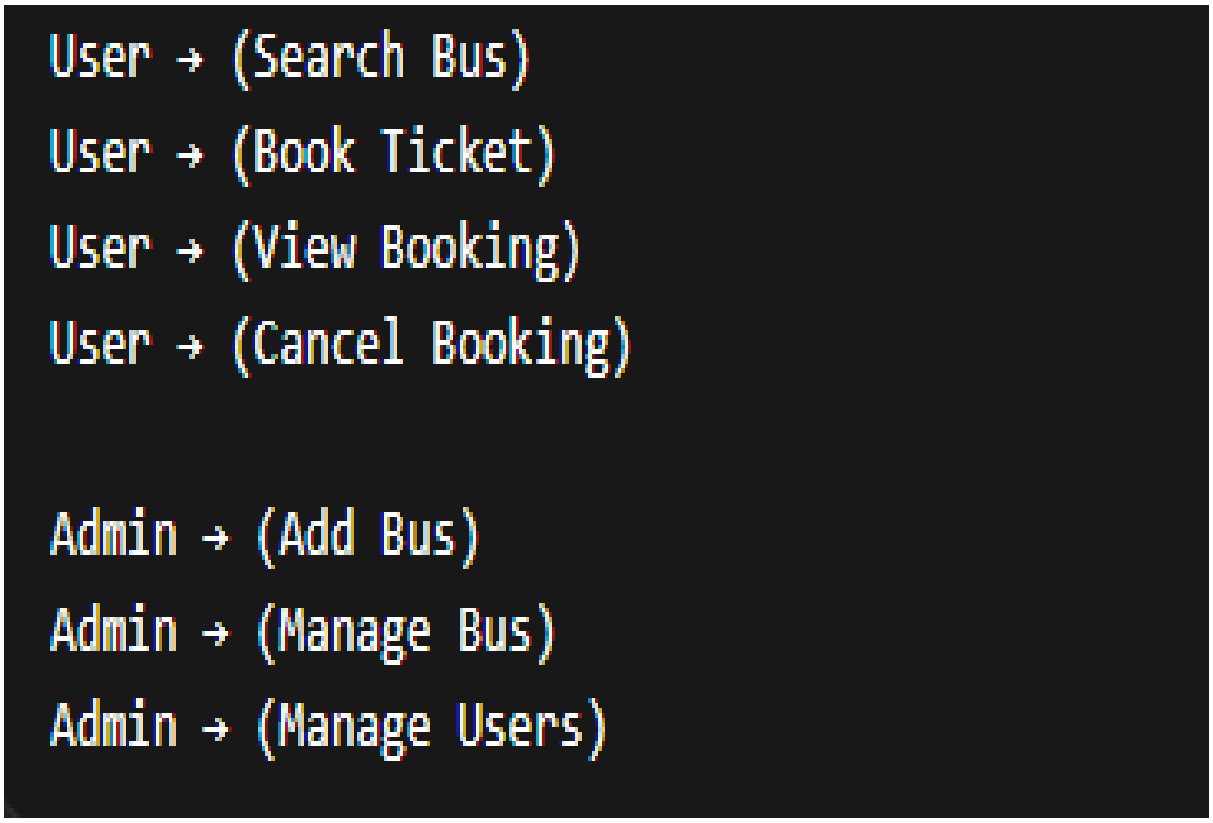


Figure 4.1

4.3 Project Lifecycle with Gantt Chart:

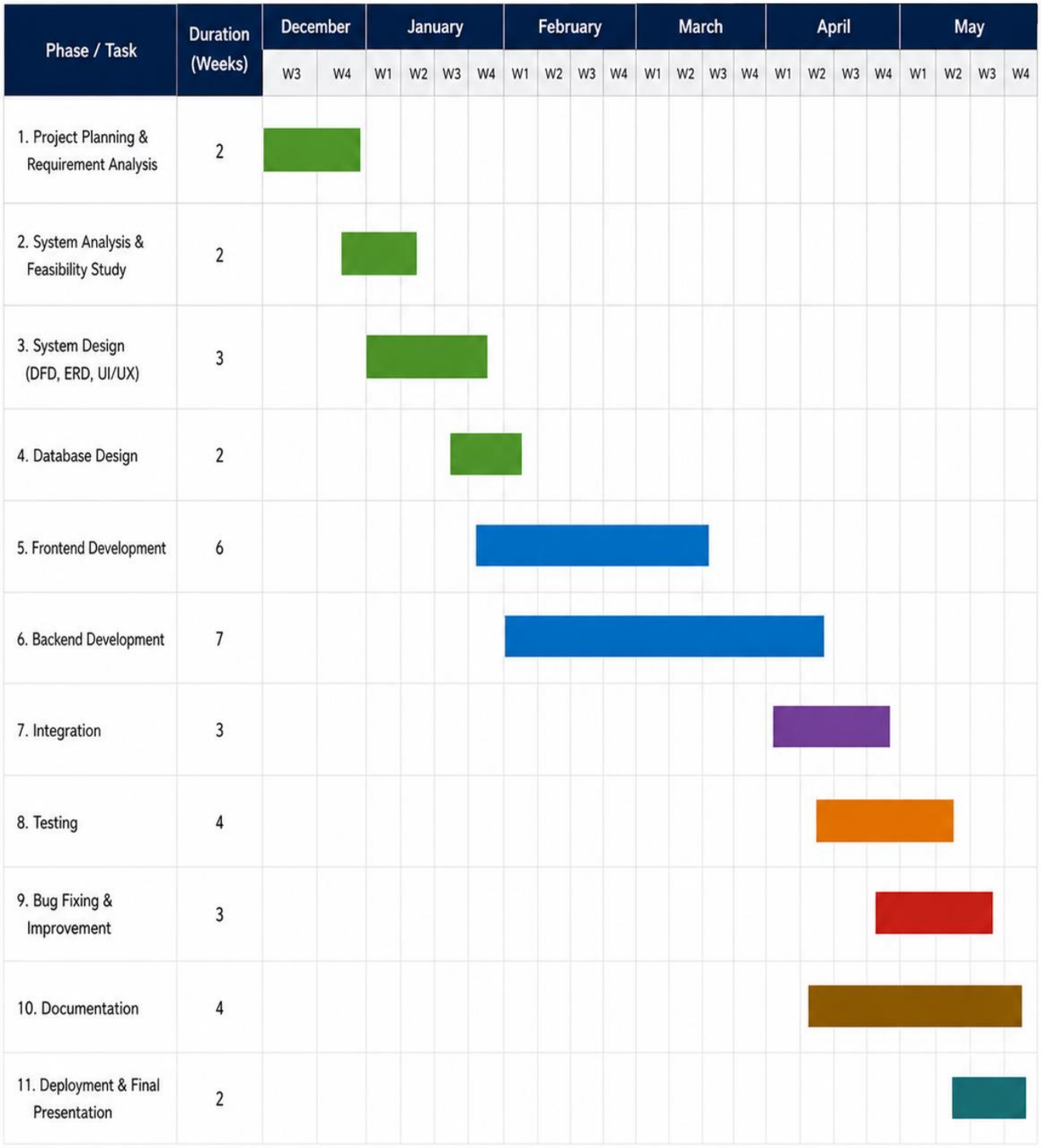


Figure 4.2

4.4 Pert Chart

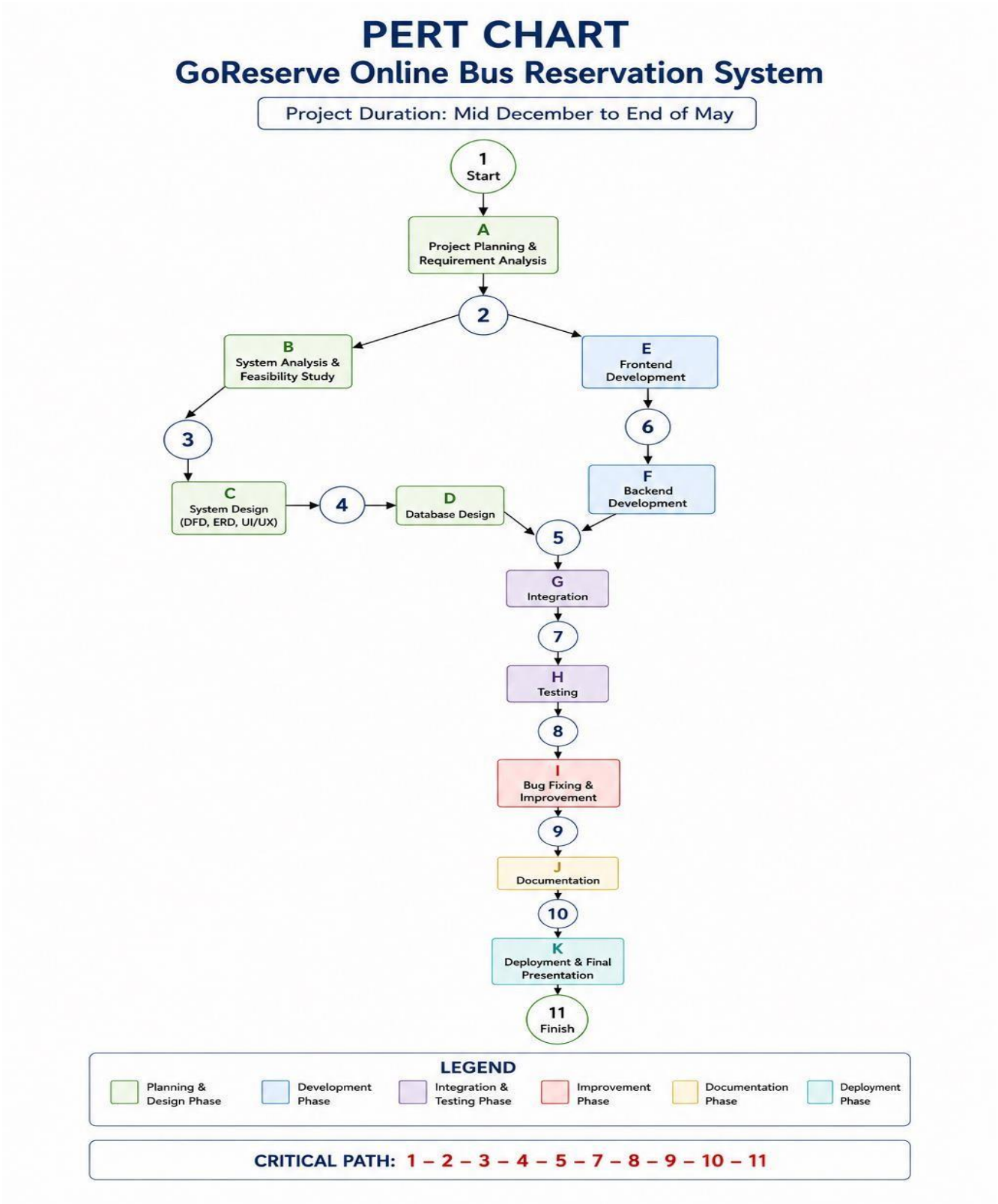


Figure 4.3

4.5 Data Flow Diagram (DFD):

A DFD shows how data moves inside the system

4.5.1 Level 0 DFD

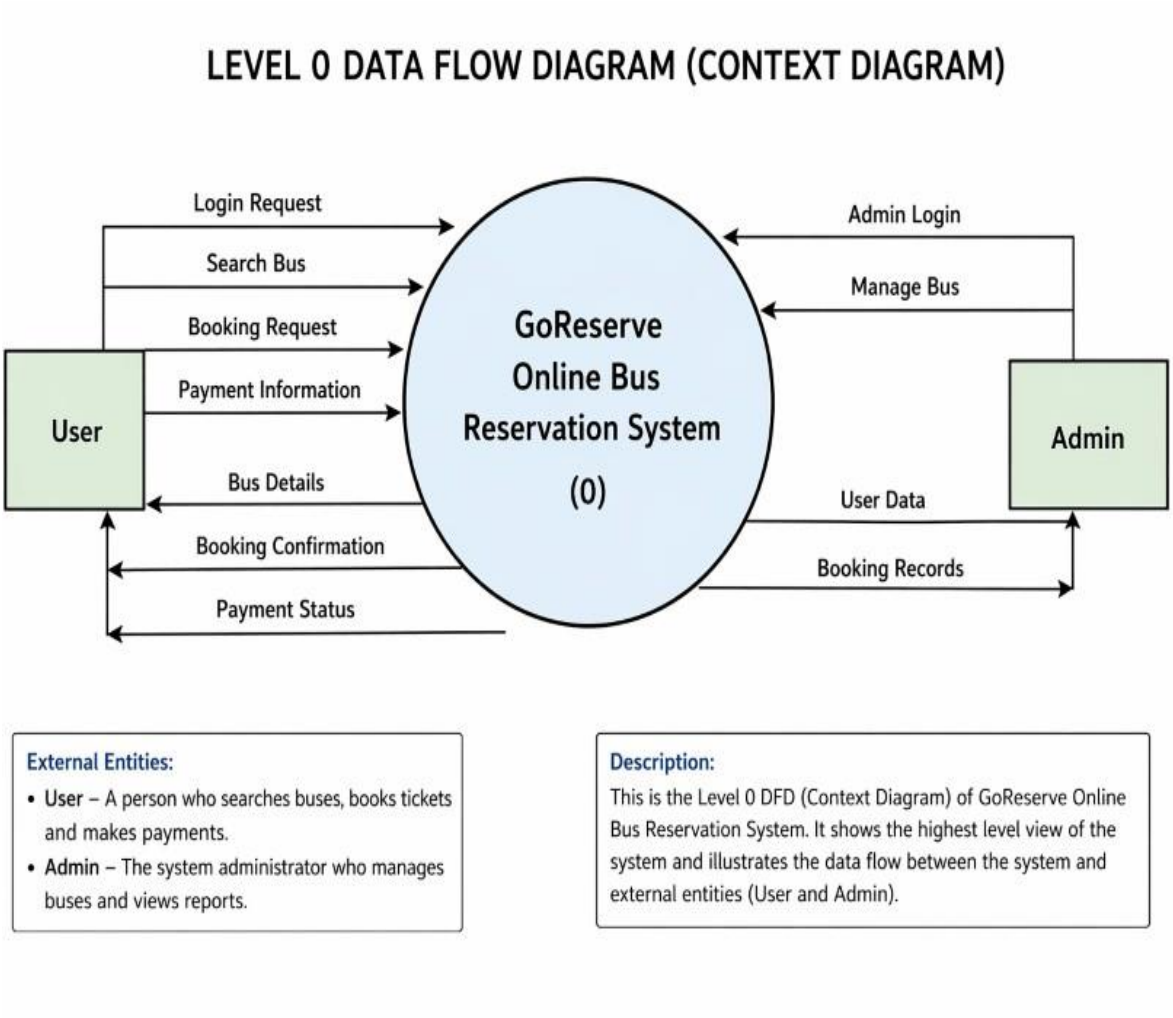


Figure 4.4

Explanation:

The Level 0 Data Flow Diagram, also known as the **Context Diagram**, provides a high-level overview of the GoReserve Online Bus Reservation System. It shows how the system interacts with external entities such as the **User** and the **Admin**, and how data flows between them.

In this diagram, the entire system is represented as a single process labeled **"GoReserve Online Bus Reservation System (0)"**. This process acts as the central unit that handles all operations related to bus booking and management.

User Interaction with the System

The **User** is one of the main external entities in the system. The user interacts with the system to perform various actions related to booking.

- The user sends a **login request** to access the system.
- After logging in, the user can **search for buses** by entering travel details such as source and destination.
- The user then sends a **booking request** by selecting a bus and entering passenger details.
- The user also provides **payment information** to complete the booking process.

In response, the system sends back:

- **Bus details** based on the search
- **Booking confirmation** after successful booking
- **Payment status** indicating whether the transaction was successful

This interaction ensures that the user can complete the entire booking process smoothly.

Admin Interaction with the System

The **Admin** is responsible for managing the system.

- The admin sends an **admin login request** to access administrative features.
- The admin can **manage buses**, which includes adding or updating bus details.

In return, the system provides:

- **User data**
- **Booking records**

4.5.2 LEVEL 1 DFD:

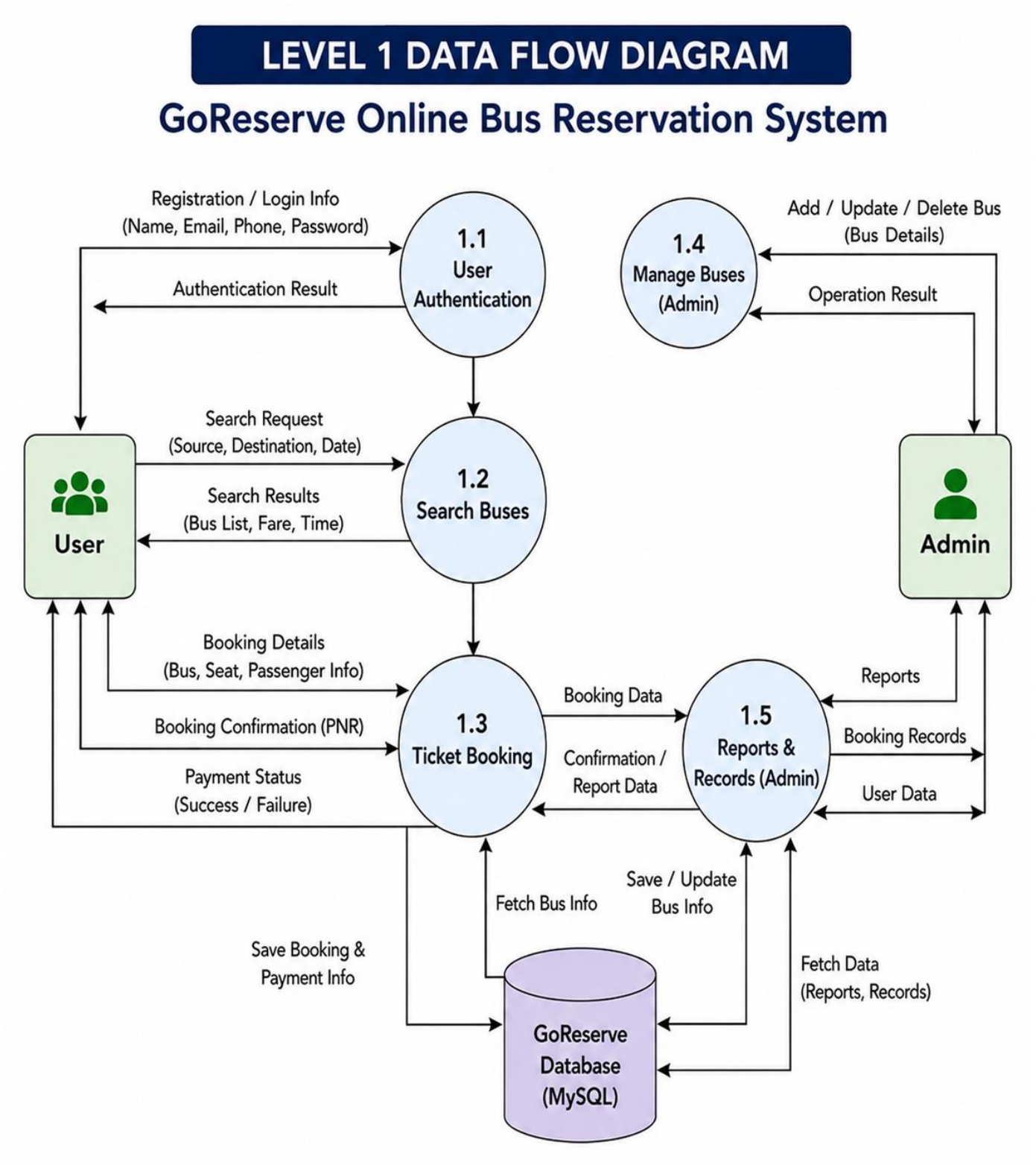


Figure 4.5

Explanation:

The Level 1 Data Flow Diagram (DFD) of the GoReserve Online Bus Reservation System provides a more detailed view of how data moves between users, administrators, system processes, and the database. It explains the internal working of the system by dividing the main system into multiple functional modules.

The diagram contains two external entities: **User** and **Admin**. The user interacts with the system to perform operations such as registration, login, searching buses, booking tickets, and making payments. The admin manages buses and monitors booking records and reports.

User Authentication

This process handles user registration and login operations. Users provide details such as name, email, phone number, and password for registration or login. The system verifies the credentials and returns the authentication result to the user. User-related information is stored in the GoReserve MySQL database.

Search Buses

In this process, users enter source, destination, and travel date details to search for available buses. The system retrieves matching bus information from the database and displays search results including bus list, fare, and timing information.

Ticket Booking

This module manages the booking process. Users provide booking details such as selected bus, seat information, and passenger details. The system stores booking and payment information in the database and generates booking confirmation details including PNR information. Payment status is also returned to the user.

Manage Buses (Admin)

This process is used by the administrator to add, update, or delete bus information. The admin sends bus management data to the system, and the operation results are returned after successful processing.

Reports & Records (Admin)

This module helps administrators monitor booking records, user data, and system reports. The system fetches report-related information from the database and displays it to the admin.

GoReserve Database (MySQL)

The diagram uses a single centralized MySQL database named “GoReserve” to store all system data, including user information, bus details, bookings, payments, and reports.

4.5.3 Level 2 DFD:

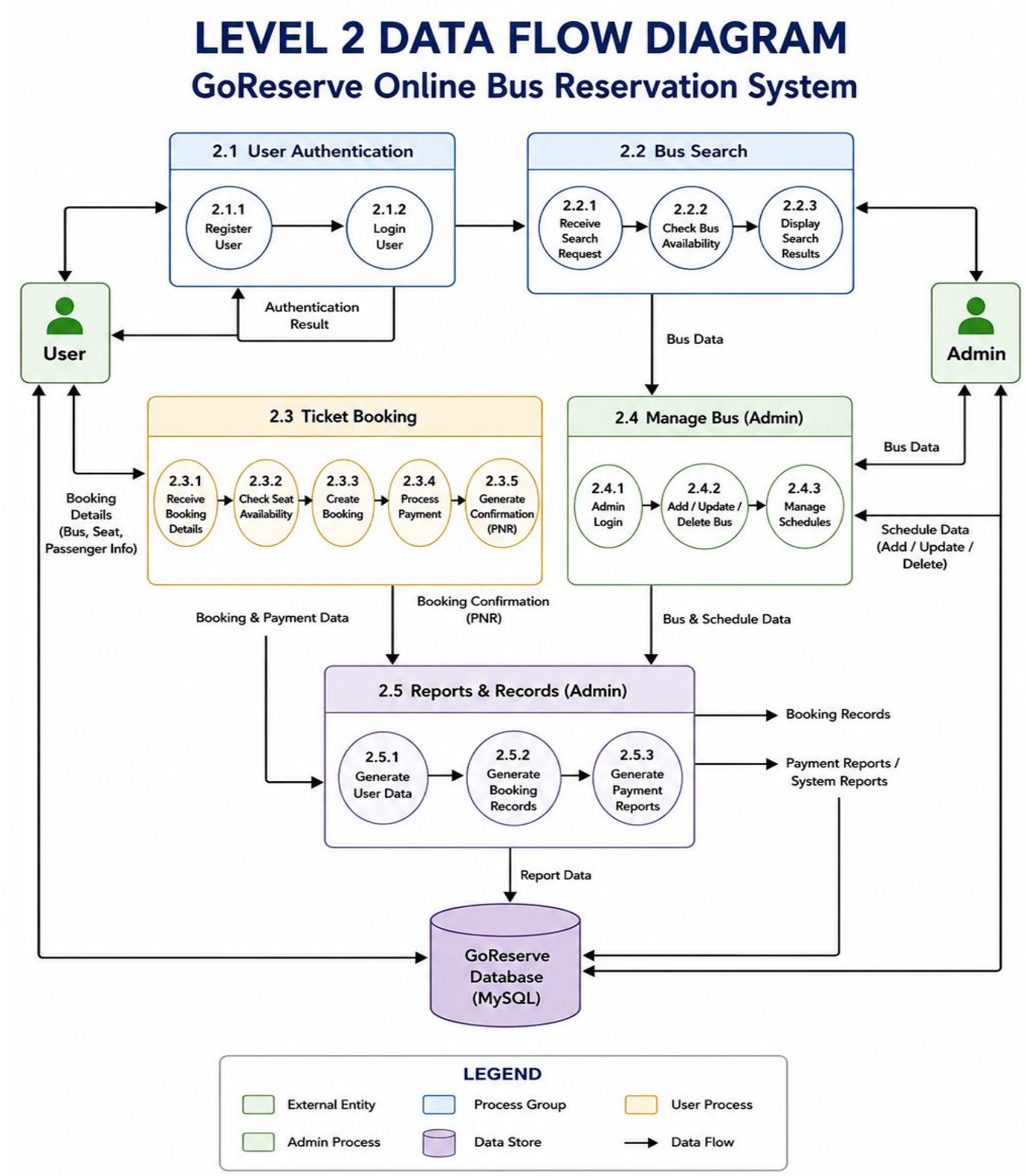


Figure 4.6

Explanation:

User Authentication

This module manages the user registration and login process.

- **Register User**
  The user enters registration details such as name, email, and password. The system validates and stores the information.
- **Login User**
  Registered users enter login credentials to access the system. Authentication is verified, and the login result is returned to the user.

This module ensures secure access to the application.

Bus Search

This process handles searching and displaying available buses.

- **Receive Search Request**
  Users enter source, destination, and travel date.
- **Check Bus Availability**
  The system checks matching buses from the database.
- **Display Search Results**
  Available buses along with fare and timing information are displayed to the user.

This module helps users easily find suitable buses.

Ticket Booking

This module manages the complete booking process.

- **Receive Booking Details**
  Passenger and seat information is collected.
- **Check Seat Availability**
  The system verifies available seats.
- **Create Booking**
  Booking details are stored in the database.

- **Process Payment**
  Payment information is processed and validated.
- **Generate Confirmation (PNR)**
  Booking confirmation and payment status are generated for the user.

This process ensures successful reservation and payment handling.

Manage Bus (Admin)

This module is used by administrators to manage transportation data.

- **Admin Login**
  Admin credentials are verified.
- **Add / Update / Delete Bus**
  Bus information can be managed by the admin.
- **Manage Schedules**
  Bus timings and schedule details are maintained.

This module allows efficient control over bus operations.

Reports & Records (Admin)

This process generates system-related records and reports.

- **Generate User Data**
  Displays registered user information.
- **Generate Booking Records**
  Shows booking-related details.
- **Generate Payment Reports**
  Provides payment and system reports for monitoring purposes.

Database Name - GoReserve

4.6 ER DIAGRAM:

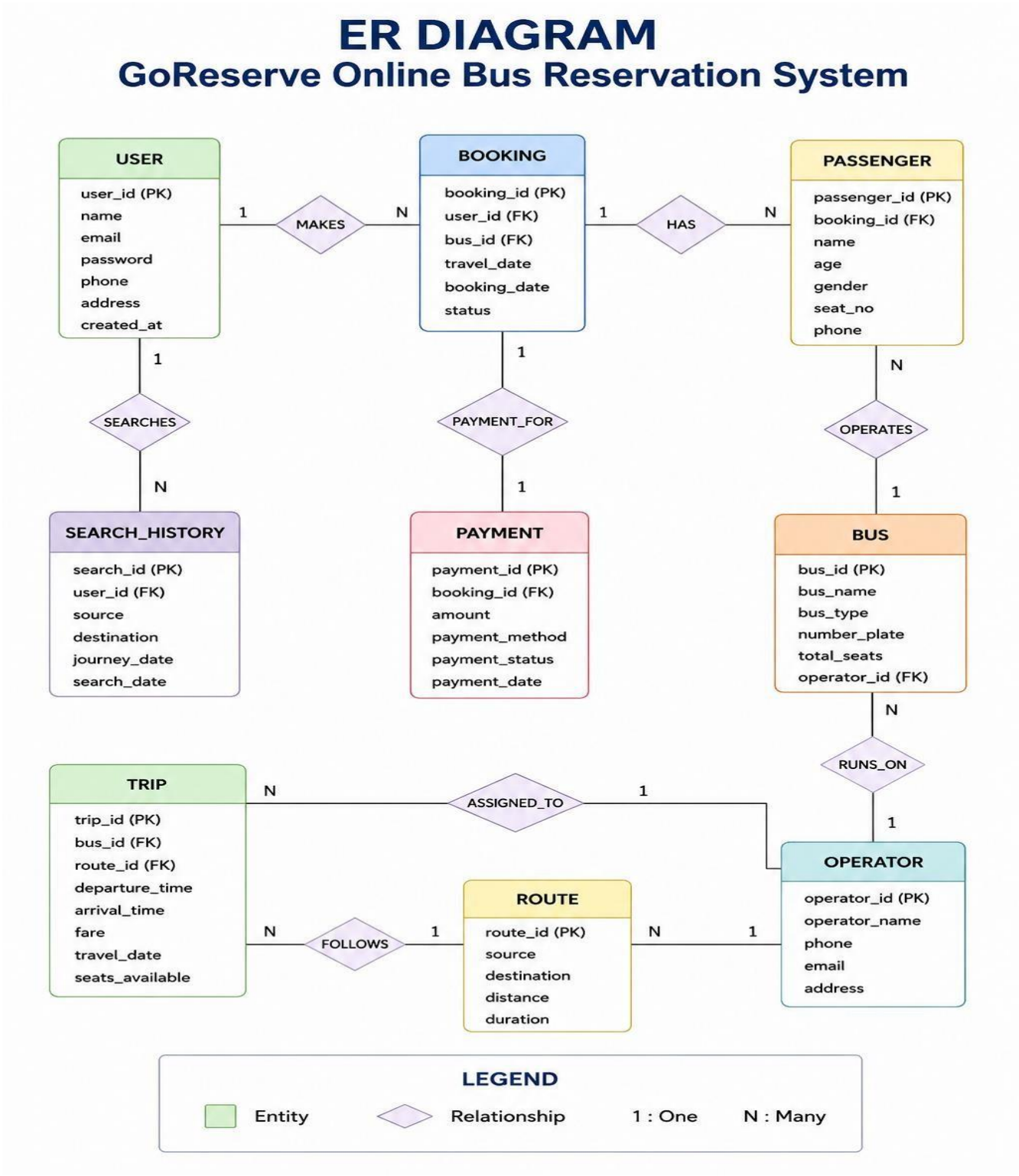


Figure 4.7

Explanation:

◻ User Entity

The **User** entity stores information about people who use the system.
Attributes include user_id, name, email, password, phone, address, and created_at.

A user can:

- Make bookings
- Search for buses

Relationship:

- One user can make multiple bookings (**1 : N**)
- One user can have multiple search records

◻ Booking Entity

The **Booking** entity contains details of ticket reservations.
Attributes include booking_id, user_id, bus_id, travel_date, booking_date, booking_status, and total_amount.

Relationship:

- Each booking is made by one user
- One booking can include multiple passengers (**1 : N**)
- Each booking is linked to one payment

◻ Passenger Entity

The **Passenger** entity stores details of passengers traveling in a booking.
Attributes include passenger_id, booking_id, name, age, gender, seat_no, and phone.

Relationship:

- One booking can have multiple passengers
- Each passenger belongs to one booking

Bus Entity

The **Bus** entity contains information about buses available in the system.
Attributes include bus_id, bus_name, bus_type, number_plate, and total_seats.

Relationship:

- One bus can have multiple bookings
- Bus details are managed by the admin

Payment Entity

The **Payment** entity stores payment-related details.
Attributes include payment_id, booking_id, amount, payment_method, payment_status, and payment_date.

Relationship:

- Each booking has one payment (**1 : 1**)
- Payment is linked directly to booking

Search History Entity

The **Search History** entity stores user search activities.
Attributes include search_id, user_id, source, destination, journey_date, and search_date.

Relationship:

- One user can perform multiple searches

Overall Relationships

- User → Booking (**1 : N**)
- Booking → Passenger (**1 : N**)
- Booking → Payment (**1 : 1**)
- User → Search History (**1 : N**)
- Bus → Booking (**1 : N**)

4.7 DATABASE DESIGN:

EER DIAGRAM:

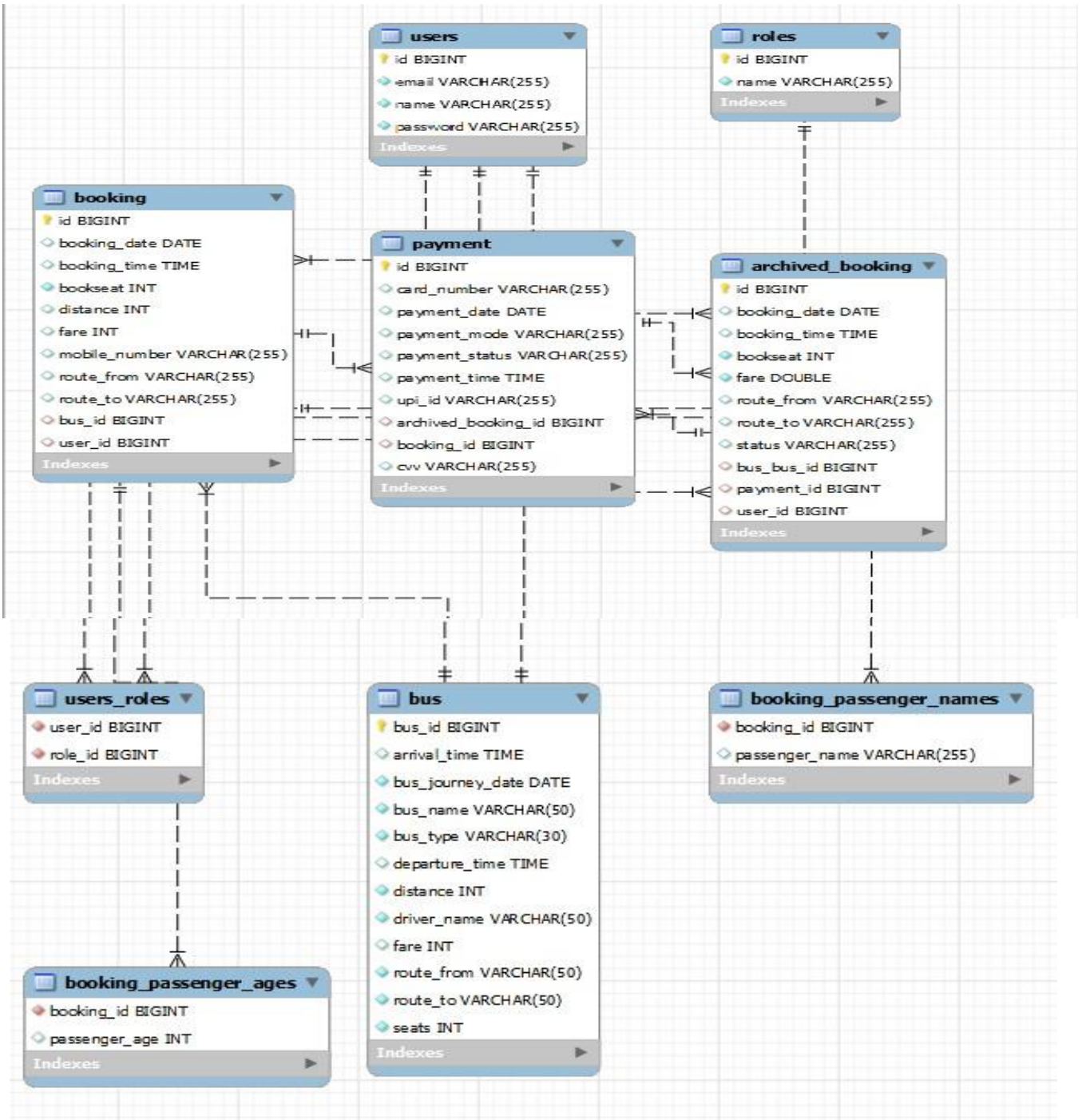


Figure 4.7

The **GoReserve** system uses a relational database (MySQL) to store and manage application data. The database is designed to efficiently handle users, buses, bookings, payments, and archived records. Route information is not stored in a separate table; instead, source and destination details are stored within the booking and bus records.

User Table:

Field	Type	Null	Key	Default	Extra
id	bigint	NO	PRI	NULL	auto_increment
email	varchar(255)	NO	UNI	NULL	
name	varchar(255)	NO		NULL	
password	varchar(255)	NO		NULL	

Table 4.1

Bus Table:

Field	Type	Null	Key	Default	Extra
bus_id	bigint	NO	PRI	NULL	auto_increment
arrival_time	time(6)	YES		NULL	
bus_journey_date	date	NO		NULL	
bus_name	varchar(50)	NO		NULL	
bus_type	varchar(30)	NO		NULL	
departure_time	time(6)	YES		NULL	
distance	int	NO		NULL	
driver_name	varchar(50)	NO		NULL	
fare	int	YES		NULL	
route_from	varchar(50)	NO		NULL	
route_to	varchar(50)	NO		NULL	
seats	int	NO		NULL	

Table 4.2

Booking Table:

Field	Type	Null	Key	Default	Extra
id	bigint	NO	PRI	NULL	auto_increment
booking_date	date	YES		NULL	
booking_time	time(6)	YES		NULL	
bookseat	int	NO		NULL	
distance	int	YES		NULL	
fare	int	YES		NULL	
mobile_number	varchar(255)	YES		NULL	
route_from	varchar(255)	YES		NULL	
route_to	varchar(255)	YES		NULL	
bus_id	bigint	YES	MUL	NULL	
user_id	bigint	YES	MUL	NULL	

Table 4.3

Payment Table:

Field	Type	Null	Key	Default	Extra
id	bigint	NO	PRI	NULL	auto_increment
card_number	varchar(255)	YES		NULL	
payment_date	date	YES		NULL	
payment_mode	varchar(255)	YES		NULL	
payment_status	varchar(255)	YES		NULL	
payment_time	time(6)	YES		NULL	
upi_id	varchar(255)	YES		NULL	
archived_booking_id	bigint	YES	UNI	NULL	
booking_id	bigint	YES	UNI	NULL	
cvv	varchar(255)	YES		NULL	

Table 4.4

Archived Booking Table:

Field	Type	Null	Key	Default	Extra
id	bigint	NO	PRI	NULL	
booking_date	date	YES		NULL	
booking_time	time(6)	YES		NULL	
bookseat	int	NO		NULL	
fare	double	NO		NULL	
route_from	varchar(255)	YES		NULL	
route_to	varchar(255)	YES		NULL	
status	varchar(255)	YES		NULL	
bus_bus_id	bigint	YES	MUL	NULL	
payment_id	bigint	YES	UNI	NULL	
user_id	bigint	YES	MUL	NULL	

Table 4.5

CHAPTER – 5

IMPLEMENTATON

5.1 Tools and Technologies Used

The GoReserve system is developed using a combination of modern tools and technologies that help in building a reliable, secure, and efficient web-based application. These technologies are selected to ensure smooth performance, easy maintenance, and better user experience.

5.1.1 Operating System

- Windows 10 / Windows 11
  (Any modern operating system supporting Java can be used)

5.1.2 Programming Languages

- Java (JDK 21)
- HTML5
- CSS3
- JavaScript

5.1.3 Frameworks & Technologies

- Spring Boot – Used for backend development and application configuration
- Spring MVC – Helps in implementing MVC architecture
- Spring Security – Provides authentication and authorization
- Thymeleaf – Used for creating dynamic web pages
- Hibernate / JPA – Used for database interaction and ORM

5.1.4 Database

- MySQL – Used as a relational database to store all system data

🔹 5.1.5 Development Tools & IDE

- Eclipse IDE (Spring Tool Suite – STS)
- Maven – Used for dependency management and project build

🔹 5.1.6 Web Technologies

- Bootstrap – Used for responsive UI design
- HTTP/HTTPS – Used for communication between client and server

🔹 5.1.7 Server

- Embedded Apache Tomcat server is used to run the application without requiring external server setup.

5.2 Hardware Requirements

The hardware requirements for the GoReserve project are minimal, making the system cost-effective and accessible.

- Processor: Intel i5 or above
- RAM: Minimum 8 GB
- Storage: 256 GB SSD
- Internet Connection

The system was developed and tested on an average-specification PC and does not require high-end hardware.

5.3 Module Description

The GoReserve system is divided into multiple logical modules, each responsible for a specific functionality. This modular design improves system organization, maintainability, and scalability.

🔹 5.3.1 User Module

This module handles all user-related operations.

Functions:

- User registration
- User authentication (login/logout)
- Searching available buses
- Booking tickets
- Viewing booking history
- Cancelling bookings

🔹 5.3.2 Bus Management Module

This module is used by the admin to manage bus-related data.

Functions:

- Add new buses
- Update bus details
- Delete buses
- Manage bus routes and availability

🔹 5.3.3 Booking Module

This module handles ticket booking operations.

Functions:

- Enter passenger details
- Basic seat allocation
- Booking confirmation
- Storing booking details in database

🔹 5.3.4 Payment Module

This module manages payment-related activities.

Functions:

- Simulated payment processing
- Payment status tracking
- Linking payment with booking
- Handling cancelled payments

🔹 5.3.5 Admin Module

This module provides administrative control over the system.

Functions:

- Admin login
- Monitoring users
- Monitoring bookings
- Managing buses

🔹 5.3.6 Archived Booking Module

This module manages cancelled bookings separately.

Functions:

- Move cancelled bookings to archive
- Maintain payment linkage
- Support refund tracking

5.4 Process Logic

The process logic describes how different modules interact with each other and how data flows through the system during various operations.

🔷 5.4.1 User Registration & Login Process

- User accesses the registration page
- User enters details and submits the form
- Password is encrypted using BCrypt
- User data is stored in the Users table
- During login, credentials are validated
- On success, the user is redirected to the dashboard

🔷 5.4.2 Bus Search Process

- User enters source and destination
- System retrieves matching buses from the Bus table
- Available buses are displayed to the user

🔷 5.4.3 Ticket Booking Process

- User selects a bus and enters booking details
- Fare is calculated based on predefined bus price
- Booking details are stored in the Booking table
- User is redirected to the payment process

🔷 5.4.4 Payment Process

- User selects payment mode (simulated)
- Payment details are processed
- Payment status is updated as PAID or CANCELLED
- Payment record is stored in the Payment table

🔷 5.4.5 Booking Cancellation Process

- User requests cancellation
- Booking status is updated
- Booking is moved to ArchivedBooking table
- Refund process is initiated (informational)

5.5 PROJECT SCREENSHOTS:

ADMIN SIDE FLOW

5.5.1 Admin Home Dashboard

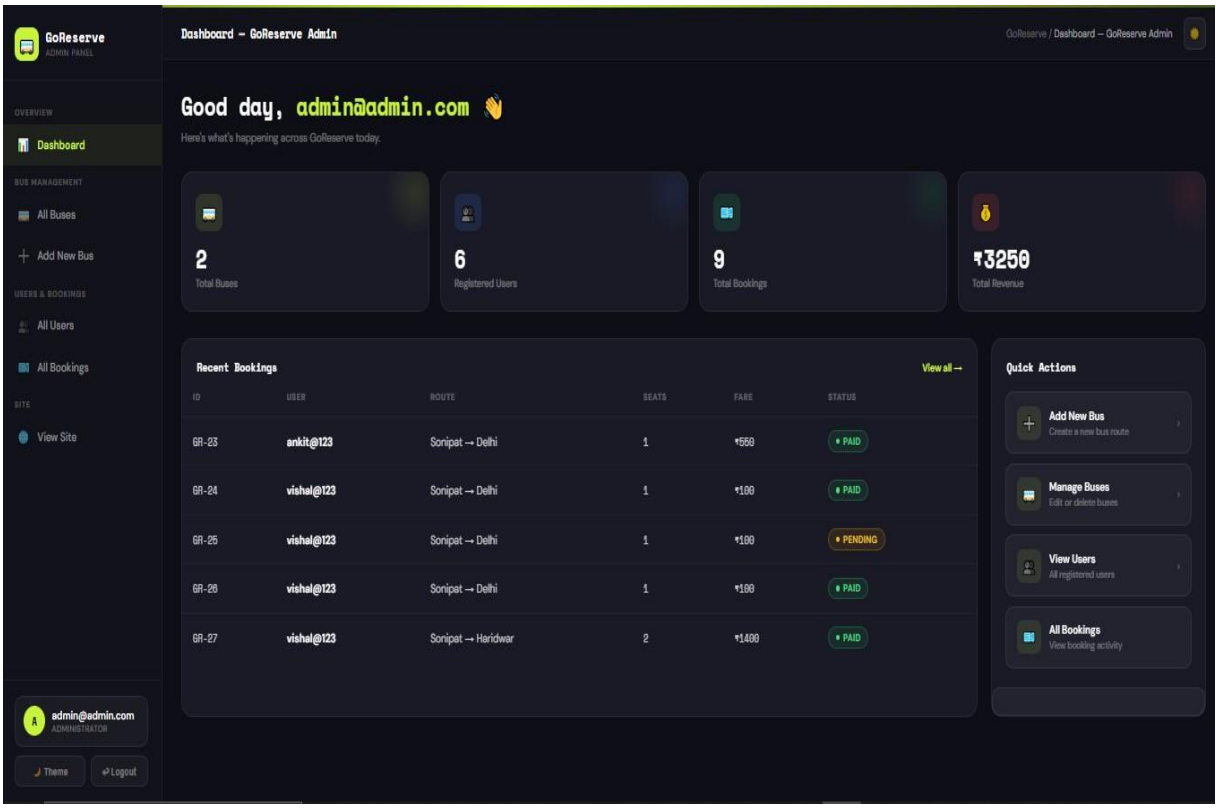


Figure 5.1

Explanation:

The Admin Dashboard serves as the central control panel of the GoReserve system. It provides administrators with an overview of important system activities and allows quick access to management features. In the example shown above, the dashboard displays statistics such as total buses, registered users, total bookings, and total revenue generated. The recent bookings section shows booking details including user name, route, seat count, fare amount, and booking status. Quick action buttons are also provided for common administrative tasks such as adding new buses, managing bus information, viewing users, and monitoring bookings. The sidebar navigation helps the admin easily switch between different modules of the system. This dashboard improves system management by organizing all important information and controls in a single interface.

5.5.2 Admin Add Bus Page

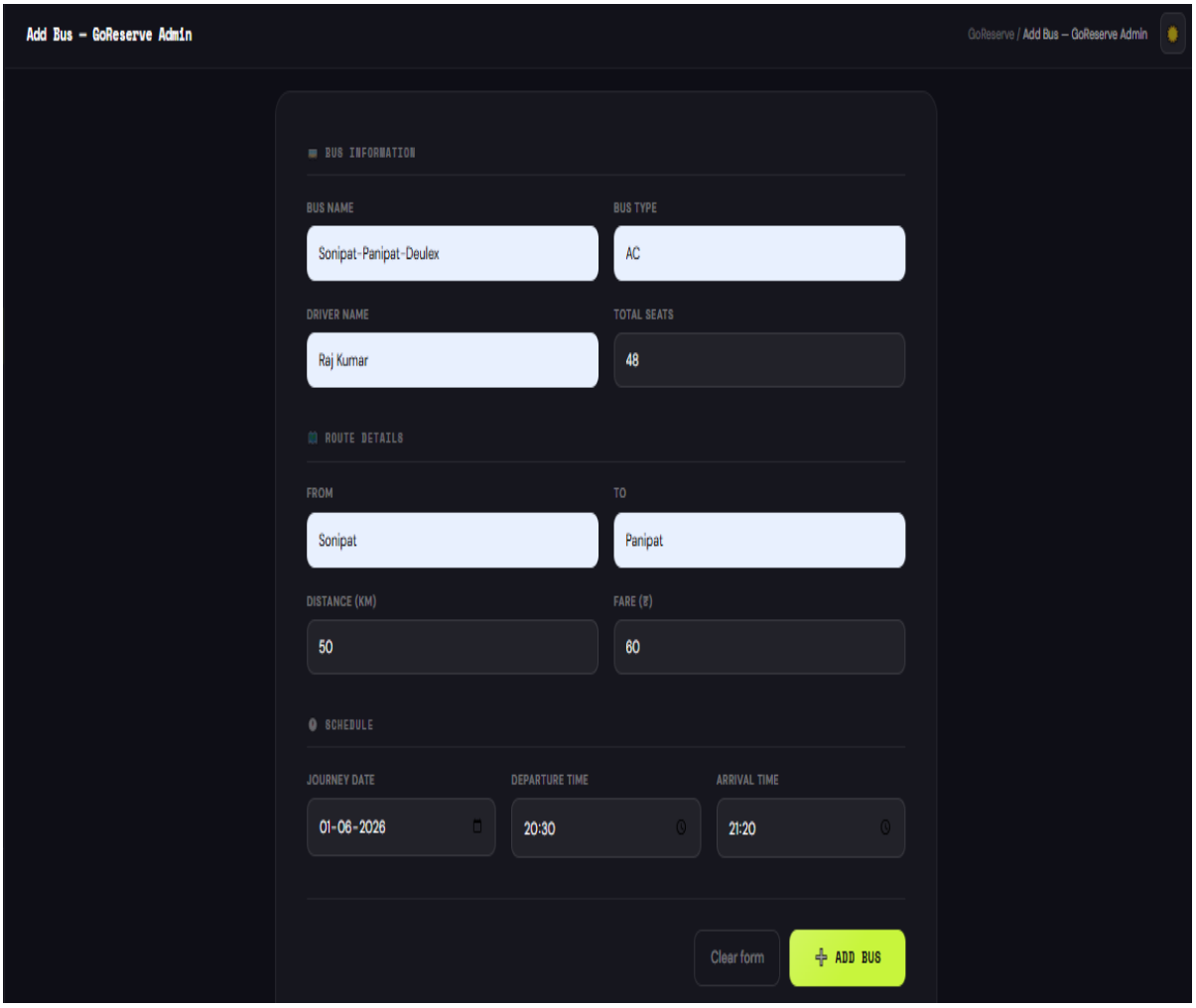


Figure 5.2

Explanation:

This screen is used by the administrator to add new buses into the GoReserve system. The form allows the admin to enter complete bus information, including bus name, bus type, driver name, total seats, route details, fare, and travel schedule. In the example shown above, a bus named "Sonipat–Panipat–Deulex" is added with AC bus type, 30 total seats, and a route from Sonipat to Panipat. The journey distance is set to 50 km with a base fare of ₹60(Pricing Formula = Base Price * Distance). The admin can also specify the journey date along with departure and arrival times. After entering all required details, the admin can click the "Add Bus" button to store the bus information in the database. This module helps administrators manage and update transportation data efficiently within the system.

5.5.3 Admin Bus List Page

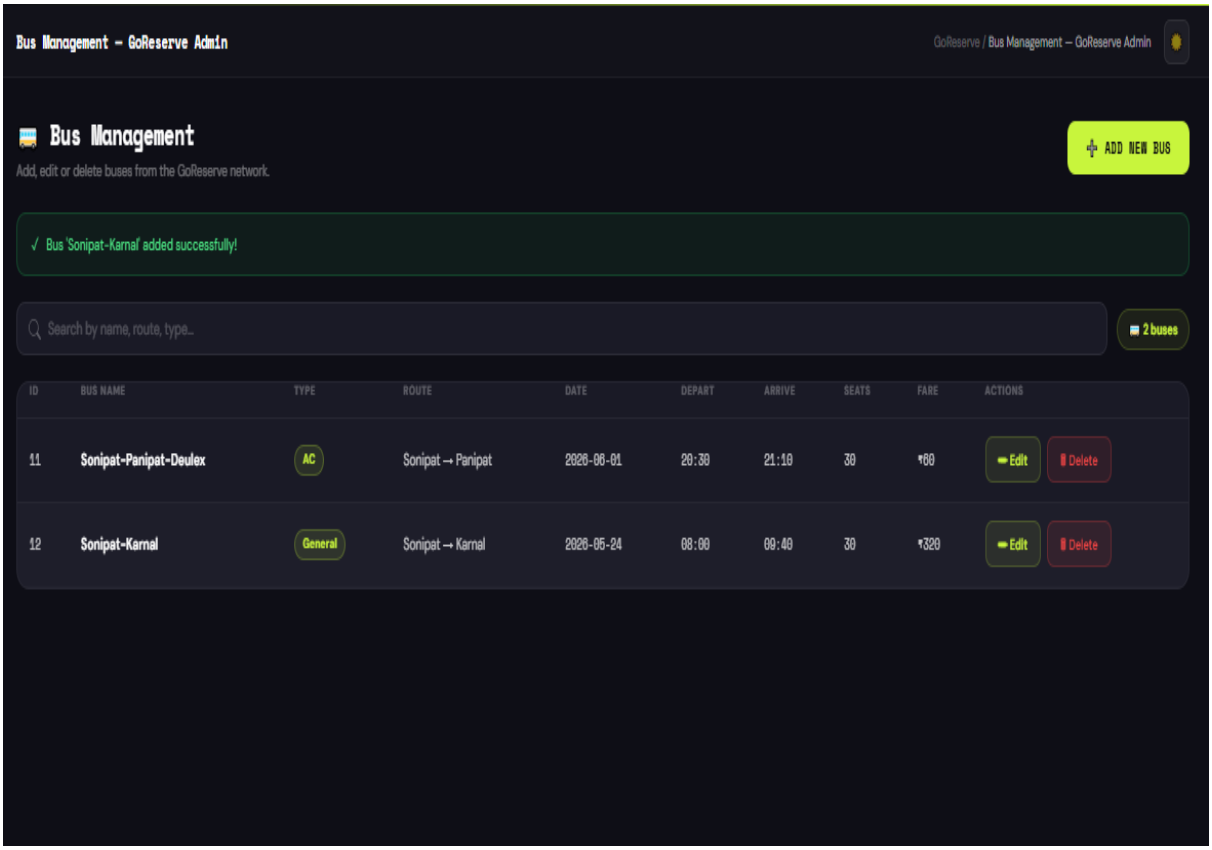


Figure 5.3

Explanation:

This screen allows the administrator to manage all buses available in the GoReserve system. The page displays important bus information such as bus name, bus type, route, journey date, departure and arrival time, total seats, and fare amount. In the example shown above, two buses are listed: "**Sonipat–Panipat–Deulex"** and **"Sonipat–Karnal**." The admin can search buses using the search bar and perform operations such as editing or deleting bus records through the action buttons provided. A success message is also displayed after adding a new bus, confirming that the operation was completed successfully. The "Add New Bus" button allows administrators to quickly insert additional bus details into the system. This module helps maintain accurate and updated transportation information efficiently.

5.5.4 Admin Booking Tracing

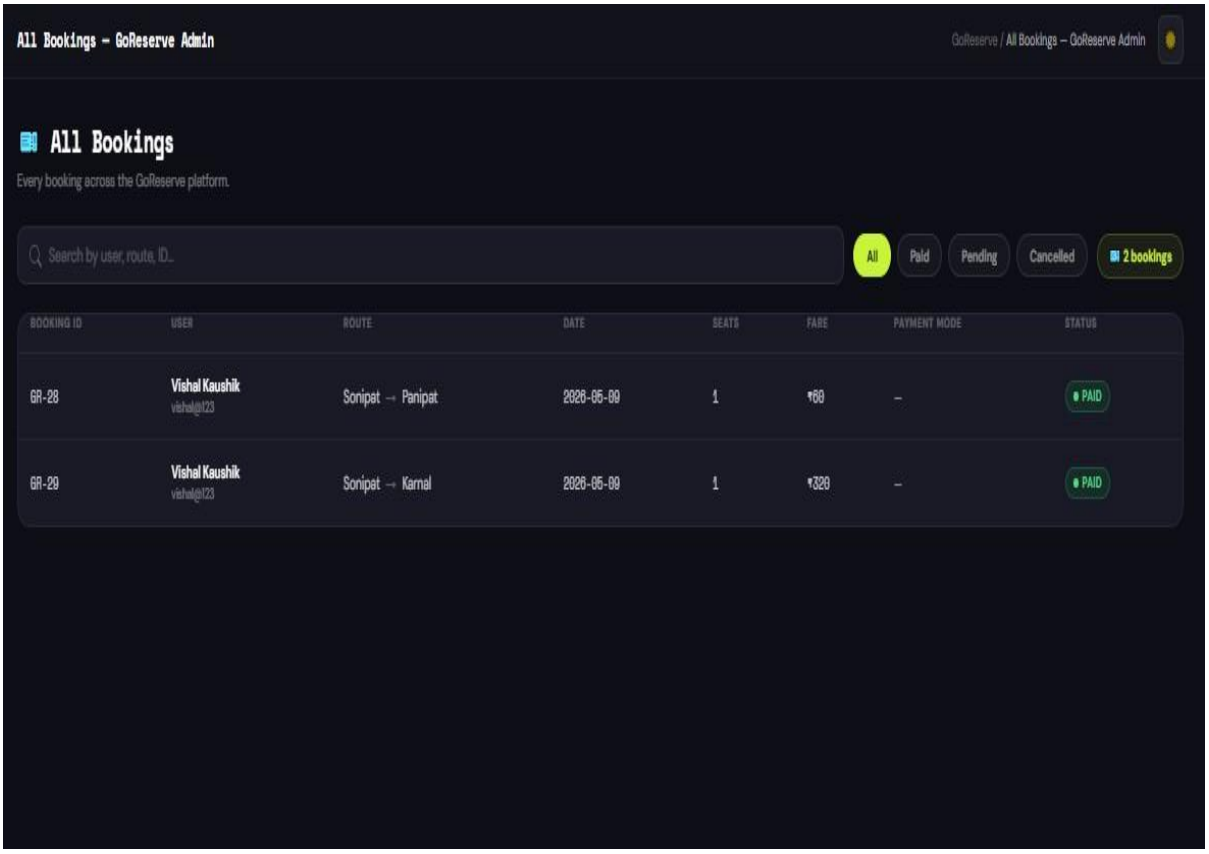


Figure 5.4

Explanation:

This screen displays all booking records available in the GoReserve system and is accessible through the admin panel. The page allows administrators to monitor booking activities performed by users across the platform. In the example shown above, booking details such as booking ID, user name, travel route, journey date, number of seats, fare amount, payment mode, and booking status are displayed in a tabular format. The admin can also filter bookings based on payment status such as Paid, Pending, or Cancelled using the filter buttons provided at the top of the page. A search bar is included to quickly locate bookings by **user name, route, or booking ID**. This module helps administrators efficiently track reservations and maintain proper booking records within the system.

5.5.5 Admin User Tracing Page

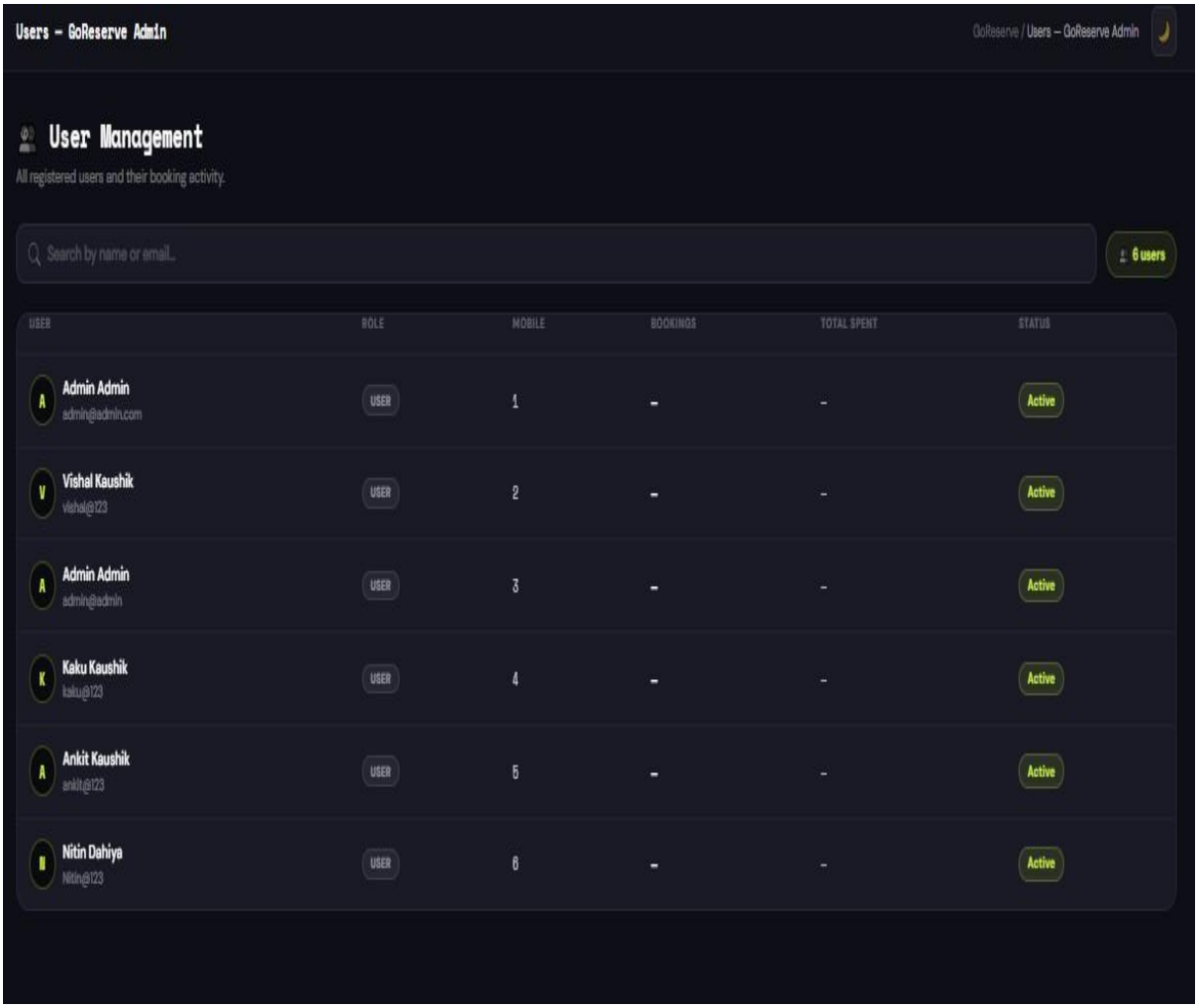


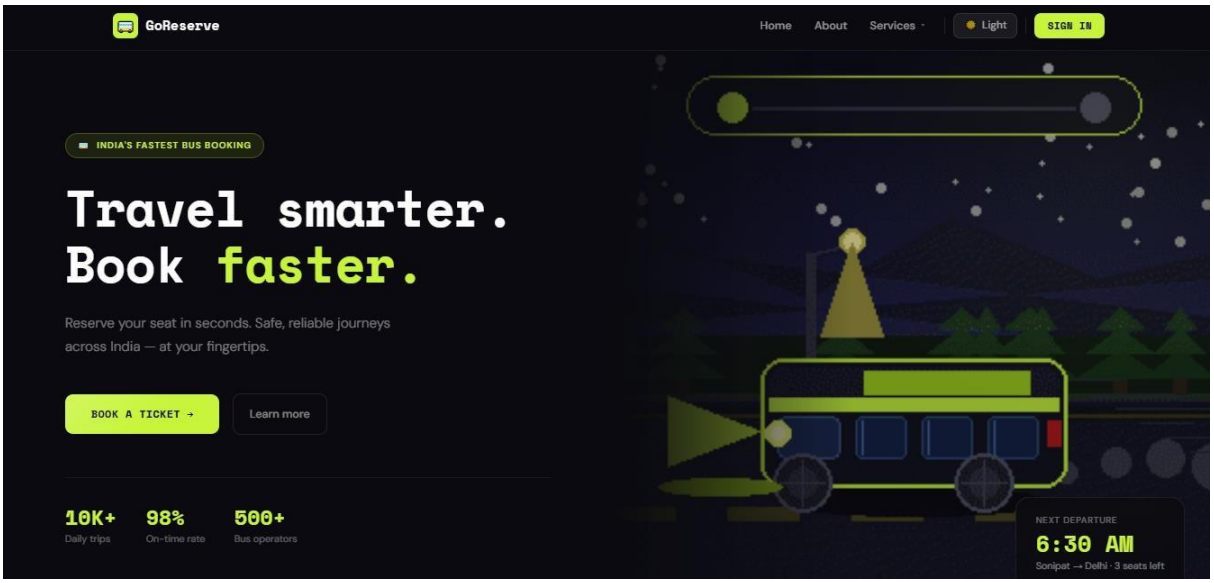
Figure 5.5

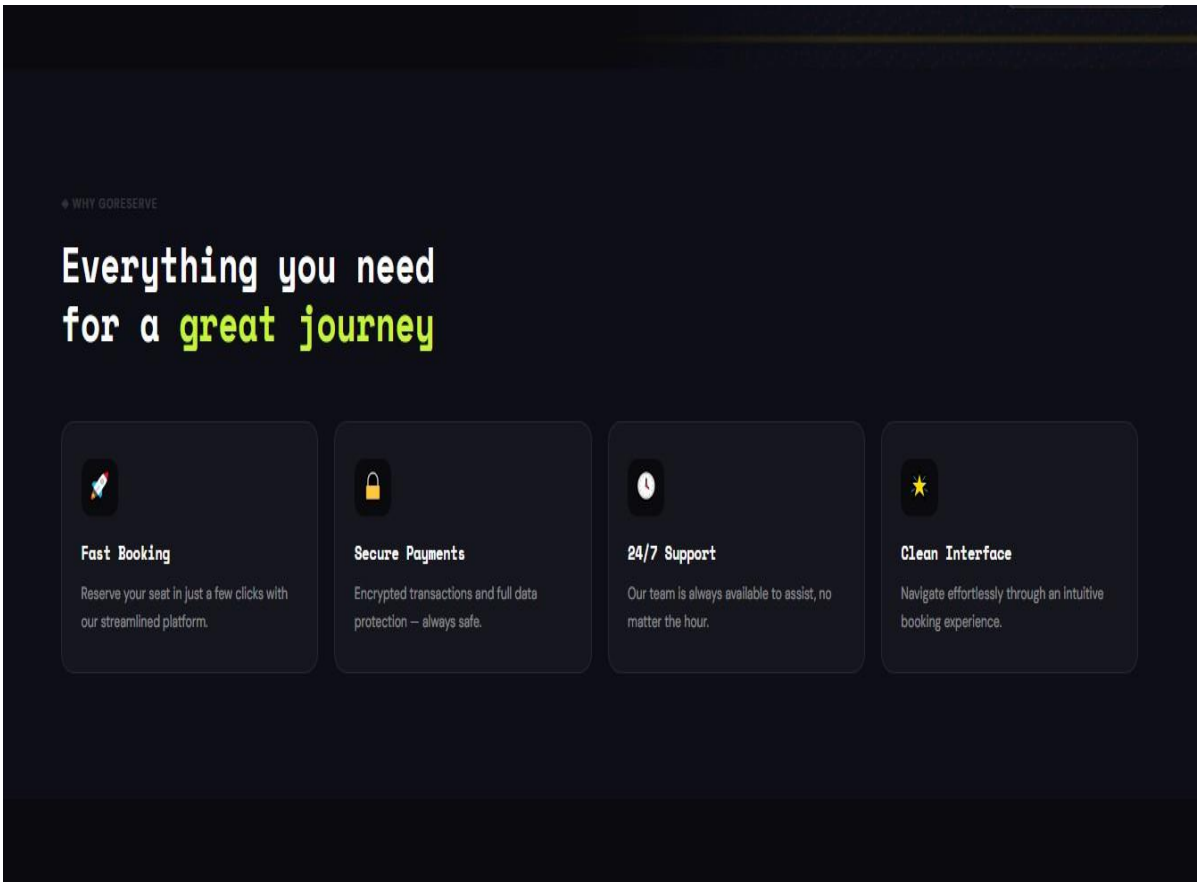
Explanation:

This screen is used by the administrator to manage and monitor registered users of the GoReserve system. The page displays user-related information such as user name, email ID, role, number of bookings, total amount spent, and account status. In the example shown above, multiple users are listed along with their assigned roles and activity details. A search bar is also provided to quickly find users by name or email address. The sidebar navigation allows the administrator to access other modules such as dashboard, bus management, and booking management. This module helps administrators keep track of user activity and maintain proper management of registered accounts within the system.

USER SIDE FLOW

5.5.6 Home Page





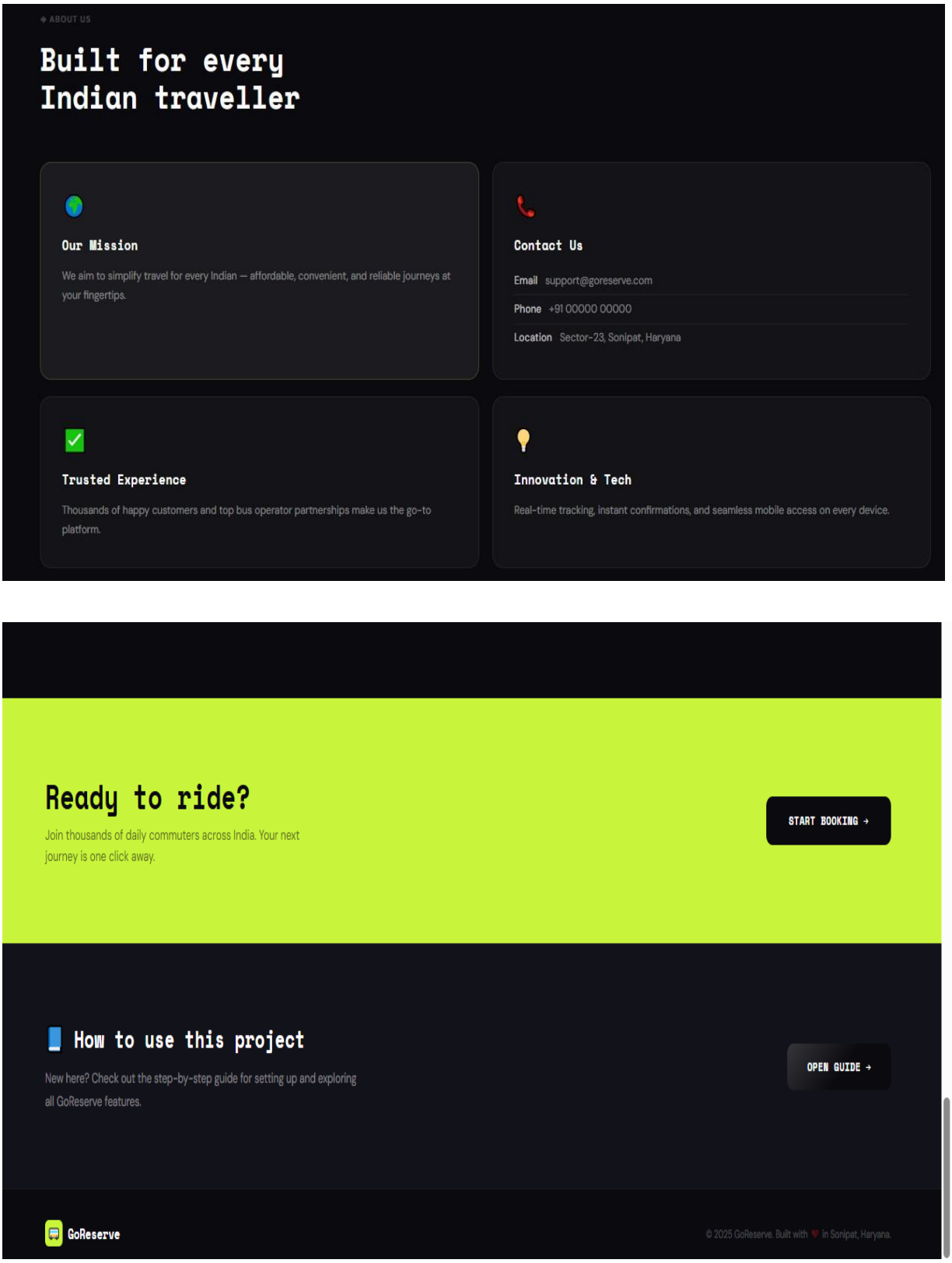

ABOUT US
Built for every Indian traveller
Our Mission
We aim to simplify travel for every Indian — affordable, convenient, and reliable journeys at your fingertips.
Contact Us
Email support@goreserve.com
Phone +91 00000 00000
Location Sector-23, Sonipat, Haryana
Trusted Experience
Thousands of happy customers and top bus operator partnerships make us the go-to platform.
Innovation & Tech
Real-time tracking, instant confirmations, and seamless mobile access on every device.
Ready to ride?
Join thousands of daily commuters across India. Your next journey is one click away.
START BOOKING →
How to use this project
New here? Check out the step-by-step guide for setting up and exploring all GoReserve features.
OPEN GUIDE →
GoReserve
© 2025 GoReserve. Built with ♥ in Sonipat, Haryana.

Figure 5.6

Explanation:

This screen represents the main landing page of the GoReserve Online Bus Reservation System. The home page is designed to provide users with a modern, attractive, and user-friendly interface for accessing different features of the system. The top navigation bar contains options such as **Home, About, Services**, and authentication controls, allowing users to navigate easily through the application. A **dark/light mode** toggle is also provided to improve user experience and accessibility.

The main banner section highlights the primary objective of the system with the message "Travel smarter. Book faster." along with a call-to-action button for ticket booking. The interface also displays travel-related information such as daily trips, online reviews, and supported bus operators to create a more realistic booking platform experience.

The second section of the page presents the core features of the system, including fast booking, secure payments, 24/7 support, and a clean user interface. These features help users understand the benefits and capabilities of the GoReserve platform.

Another section provides information about the project's mission, contact details, trusted service experience, and technology-driven approach. This helps in building user trust and giving additional information about the system.

At the bottom of the page, quick-access options such as "Start Booking" and "How to Use This Project" are provided, allowing users to begin using the system easily. Overall, the home page creates a professional first impression and improves user interaction with the application.

5.5.7 Registration Page

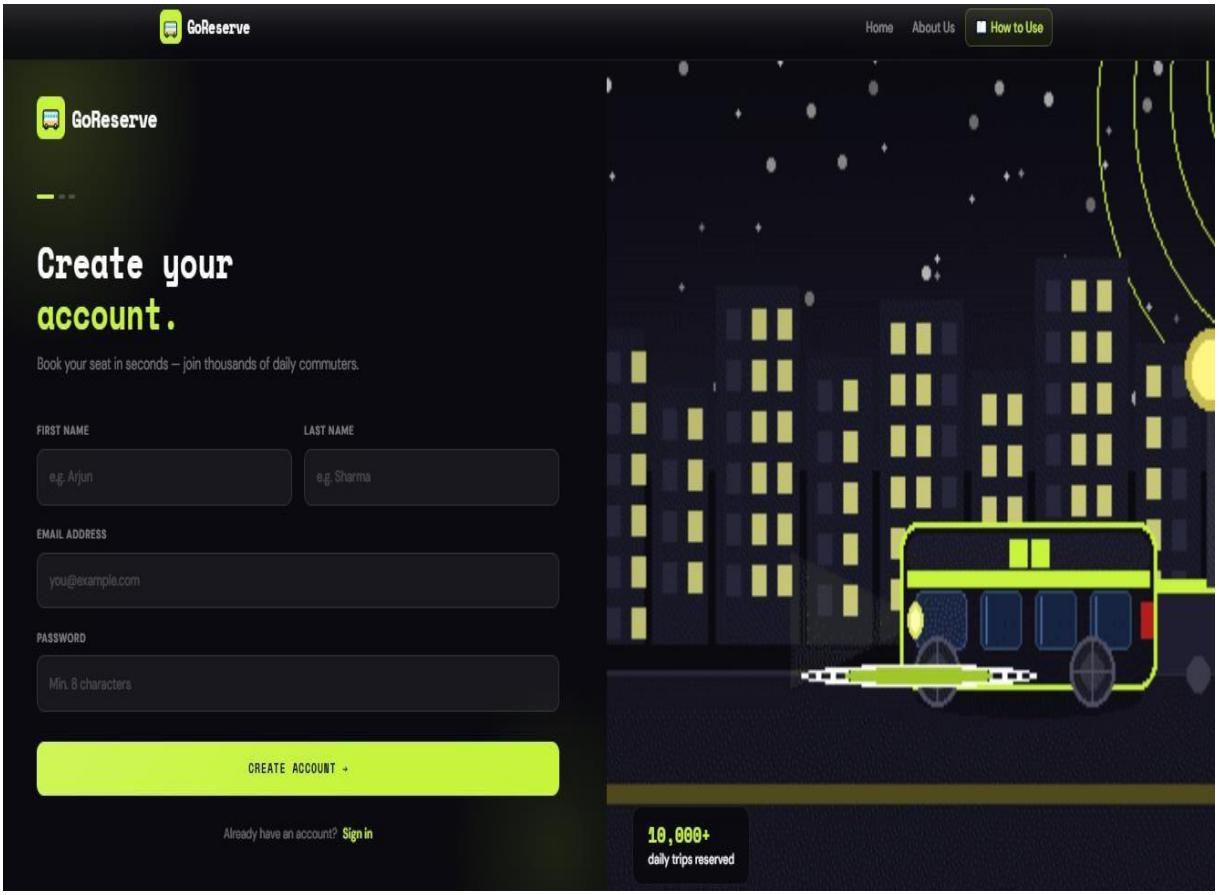


Figure 5.7

Explanation:

This screen allows new users to create an account in the GoReserve system. The registration form collects essential user details such as first name, last name, email address, and password. In the example shown above, the user enters personal information to register successfully on the platform. Password validation is also implemented to ensure secure account creation. After submitting the form, the user details are stored securely in the database, and authentication mechanisms are applied using Spring Security. The page is designed with a clean and responsive interface to provide a smooth registration experience for users. Additionally, users who already have an account can directly navigate to the sign-in page using the provided login link.

5.5.8 Login Page

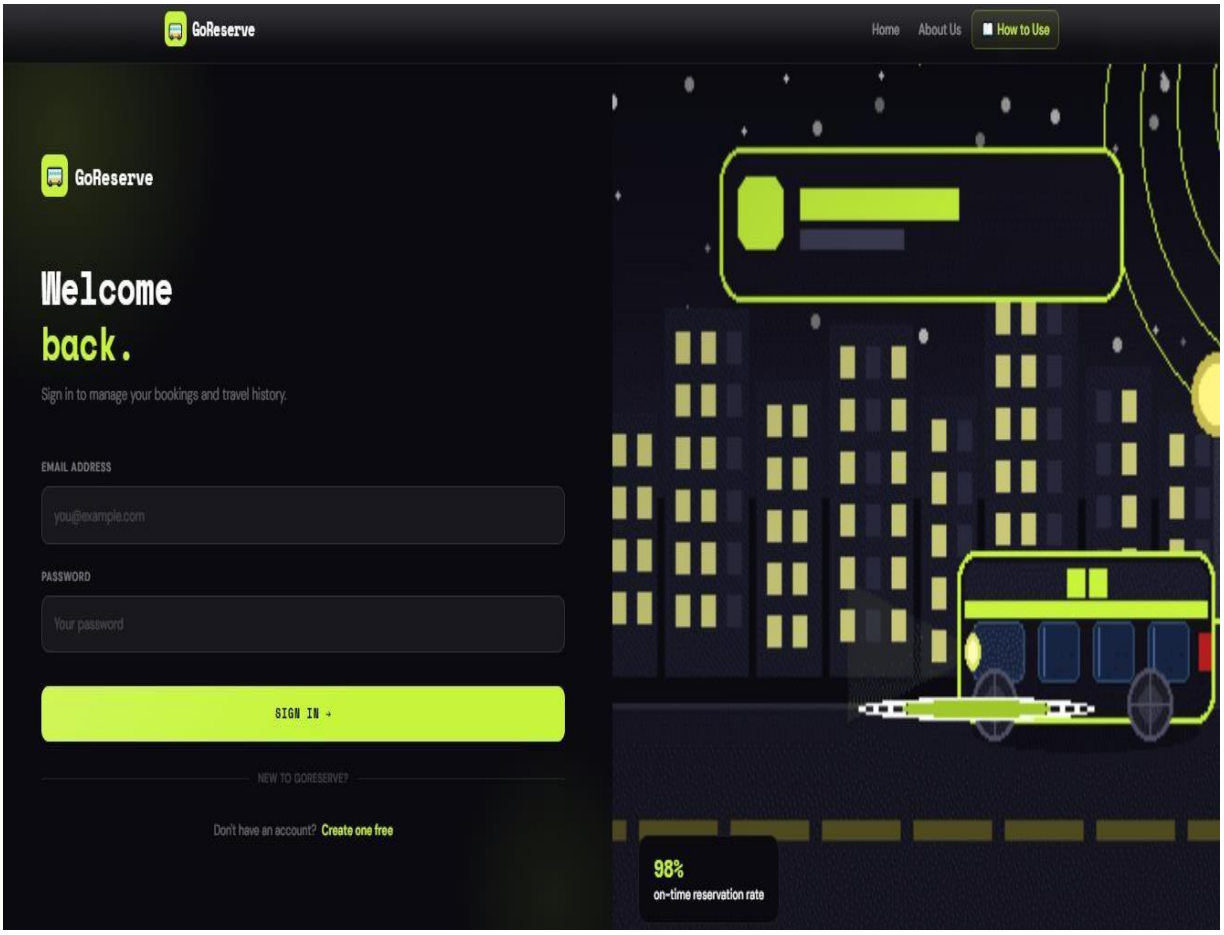


Figure 5.8

Explanation:

This screen represents the login page of the GoReserve system, where registered users can securely access their accounts. The form requires users to enter their email address and password for authentication. In the example shown above, the user provides valid login credentials to access booking-related features such as bus search, ticket booking, and booking history. The login process is secured using Spring Security, which verifies user credentials before granting access to the system. The page also includes a link for new users to create an account if they are not already registered. The clean and responsive interface improves usability and provides a smooth authentication experience for users.

5.5.9 Find Bus Page

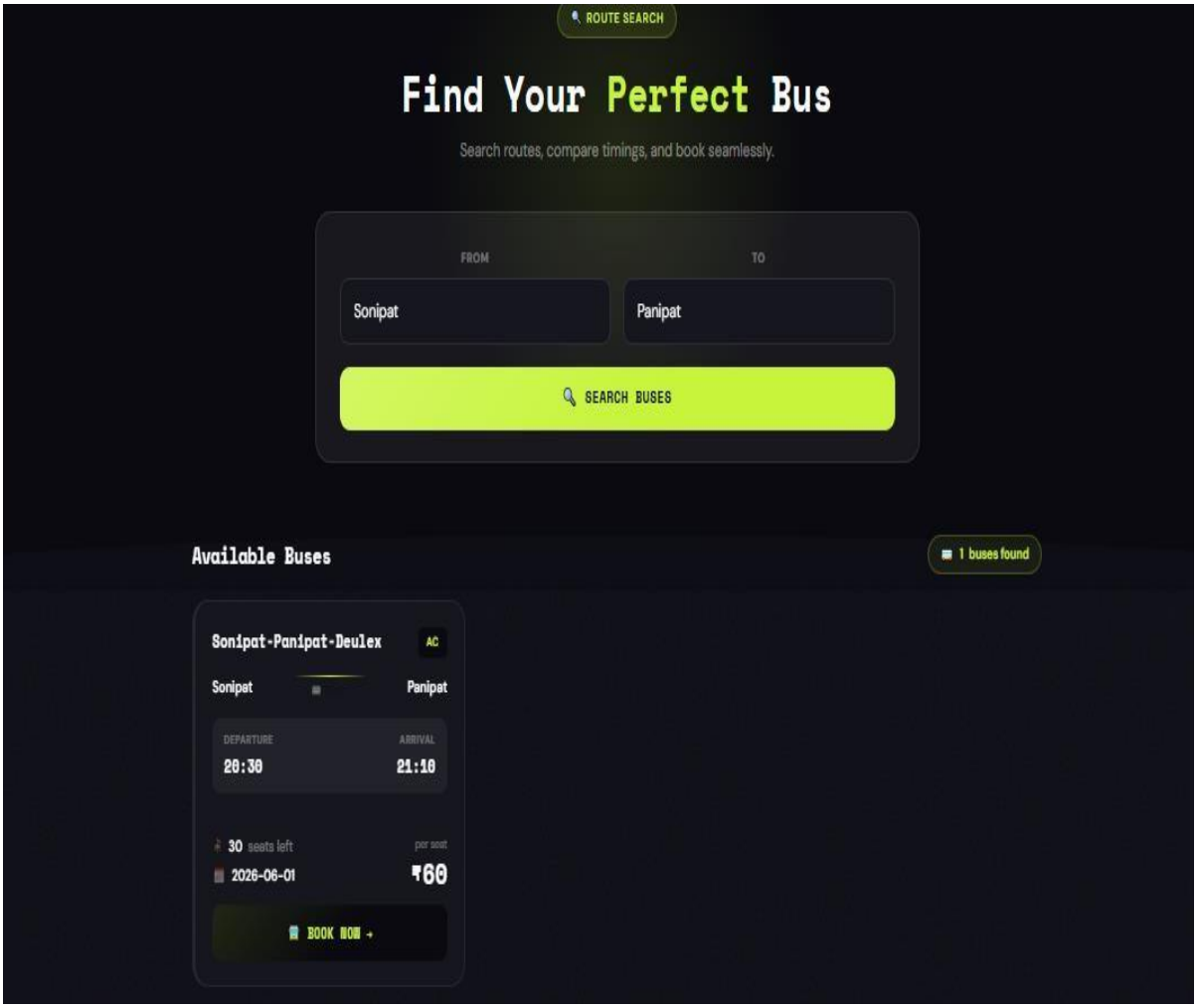


Figure 5.9

Explanation:

This screen allows users to search for available buses based on source and destination locations. In the example shown above, the user searches for buses traveling from Sonipat to Panipat. After submitting the search request, the system retrieves matching bus records from the database and displays the available results below the search form. The displayed bus information includes bus name, bus type, departure and arrival time, available seats, journey date, and ticket fare. Users can review the travel details and proceed to ticket booking by clicking the "Book Now" button. This module improves convenience by allowing users to quickly find suitable buses without manually checking schedules.

5.5.10 Bus List Page

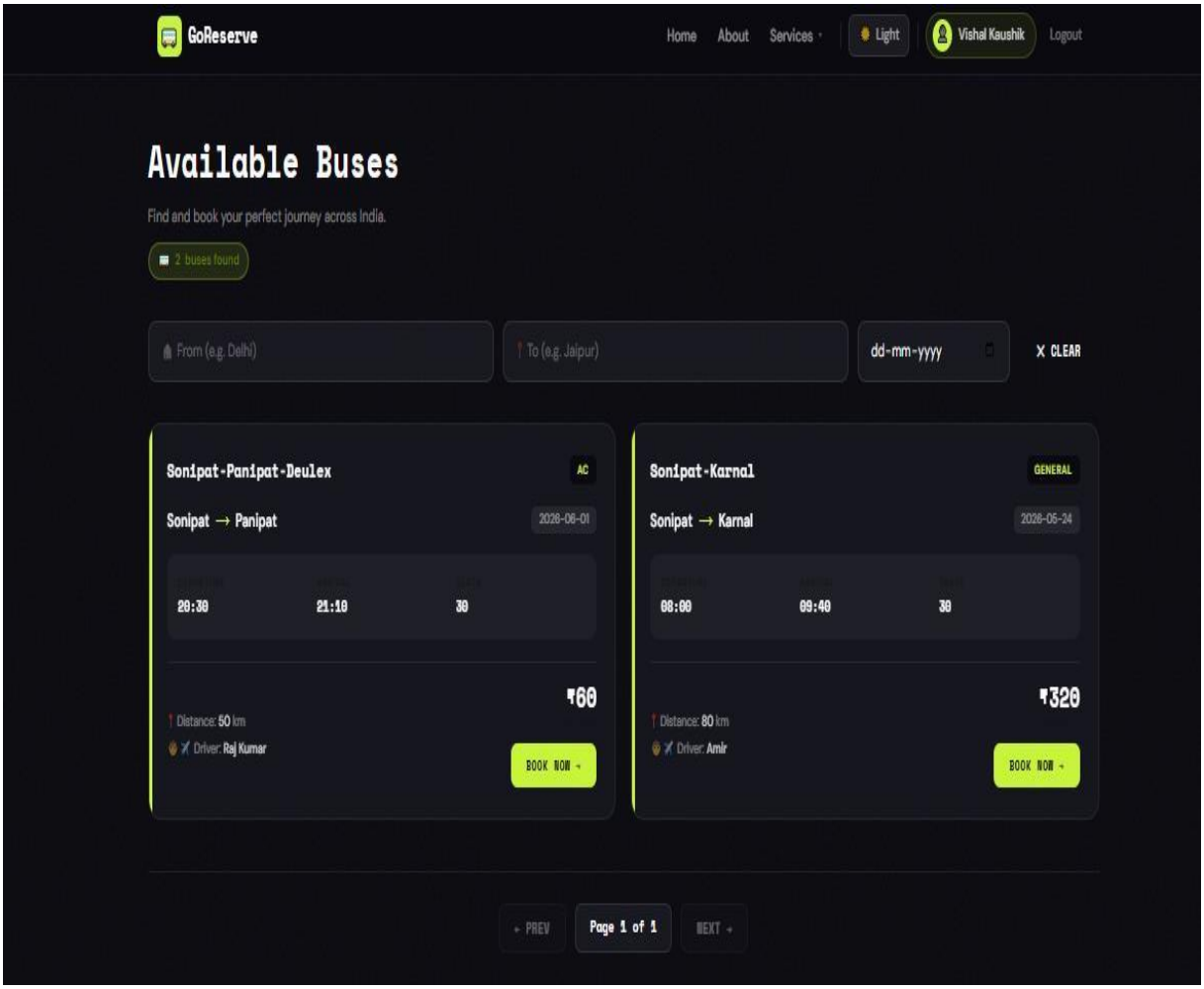


Figure 5.10

Explanation:

This screen displays all available buses present in the GoReserve system for users to explore and book tickets. The page provides detailed information about each bus, including bus name, route, bus type, departure time, arrival time, total seats, distance, driver name, and ticket fare. In the example shown above, two buses are available for booking**: "Sonipat–Panipat–Deulex" and "Sonipat–Karnal."** Users can also filter buses using source, destination, and date fields provided at the top of the page. The "Book Now" button allows users to proceed directly to the booking process for the selected bus. Pagination is also implemented at the bottom of the page to manage large numbers of bus records efficiently. This module helps users easily compare available buses and select suitable travel options according to their requirements.

5.5.11 Booking Page

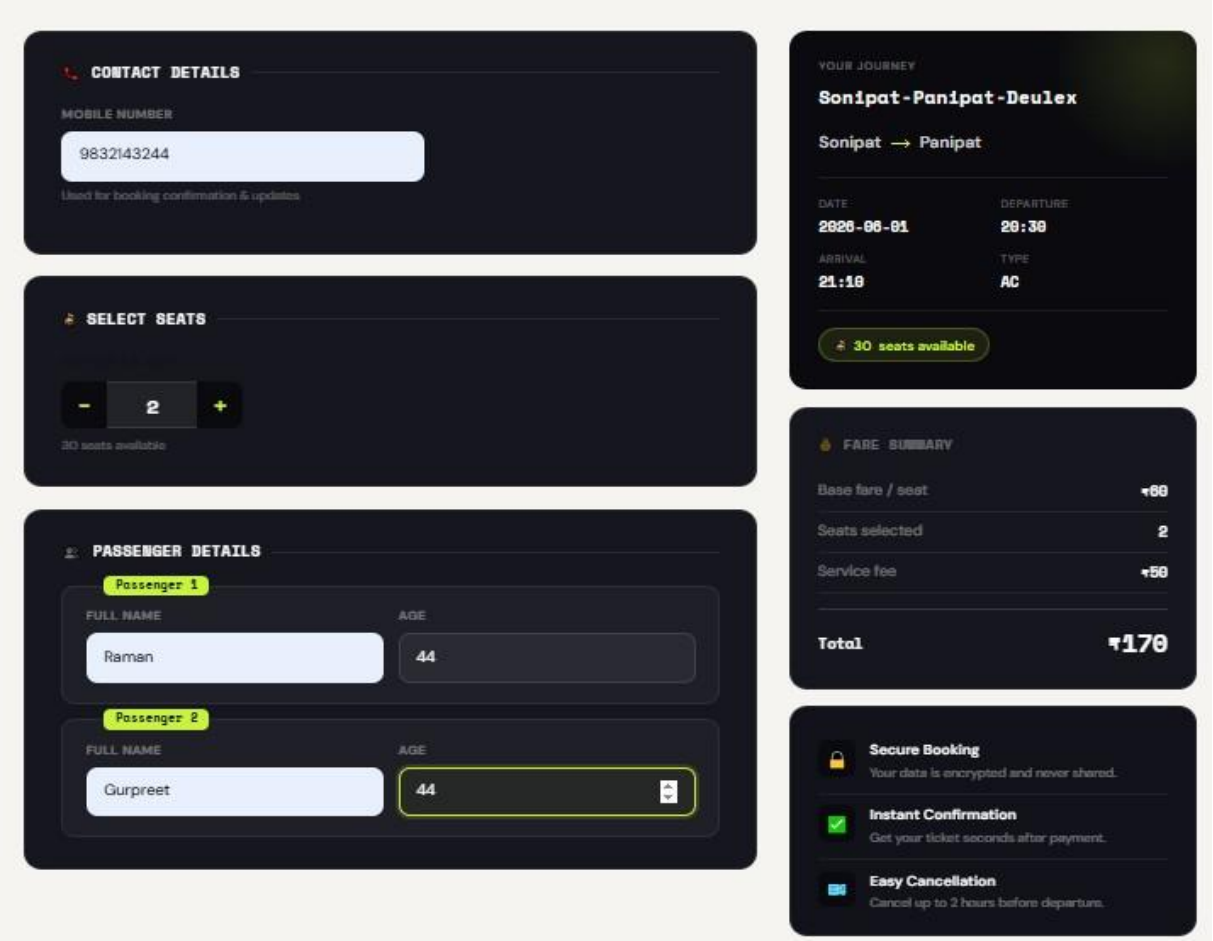


Figure 5.11

Explanation:

This screen represents the ticket booking page of the GoReserve system, where users enter travel and passenger details to confirm their reservation. The page displays journey information such as bus name, travel route, date, departure time, arrival time, bus type, and available seats. In the example shown above, the user is booking tickets for the "Sonipat–Panipat–Deulex" bus with two selected seats. Passenger details including name and age are entered for each traveler. The system also provides a fare summary section showing the base fare, number of selected seats, service fee, and total payable amount. Additional booking information such as secure booking, instant confirmation, and cancellation support is also displayed to improve user confidence. This module ensures a smooth and organized booking experience for users before proceeding to payment.

5.5.12 Payment Page

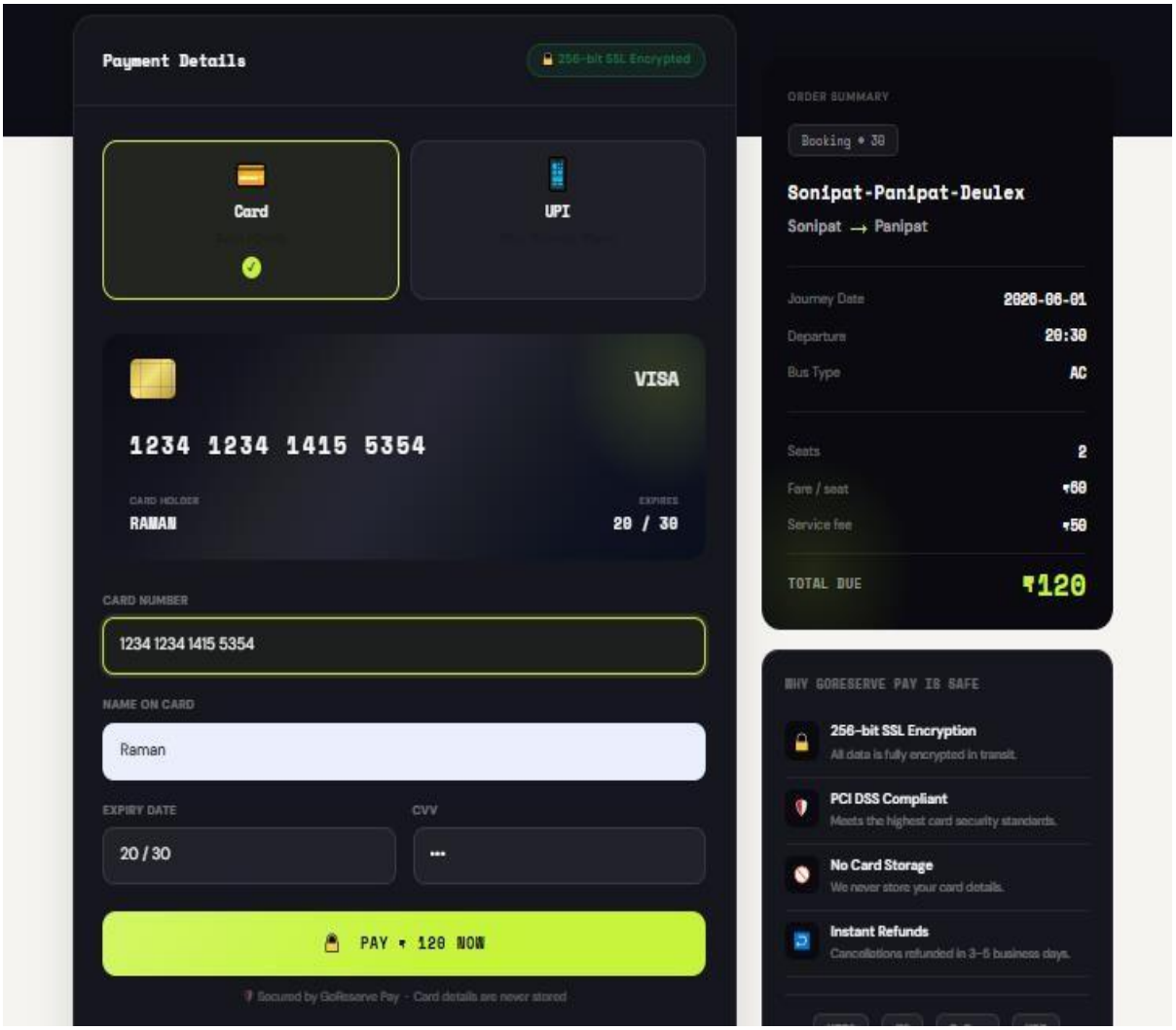


Figure 5.12

Explanation:

This screen represents the payment module of the GoReserve system, where users complete the payment process for ticket booking. The page allows users to choose between different payment methods such as Card and UPI. In the example shown above, the user selects the card payment option and enters card details including card number, card holder name, expiry date, and CVV. The order summary section on the right side displays important booking information such as bus name, travel route, journey date, departure time, number of seats, fare amount, service fee, and total payable amount. Security-related information such as SSL encryption and PCI DSS compliance is also displayed to assure users that their payment details are protected.

5.5.13 Payment Confirmation Page

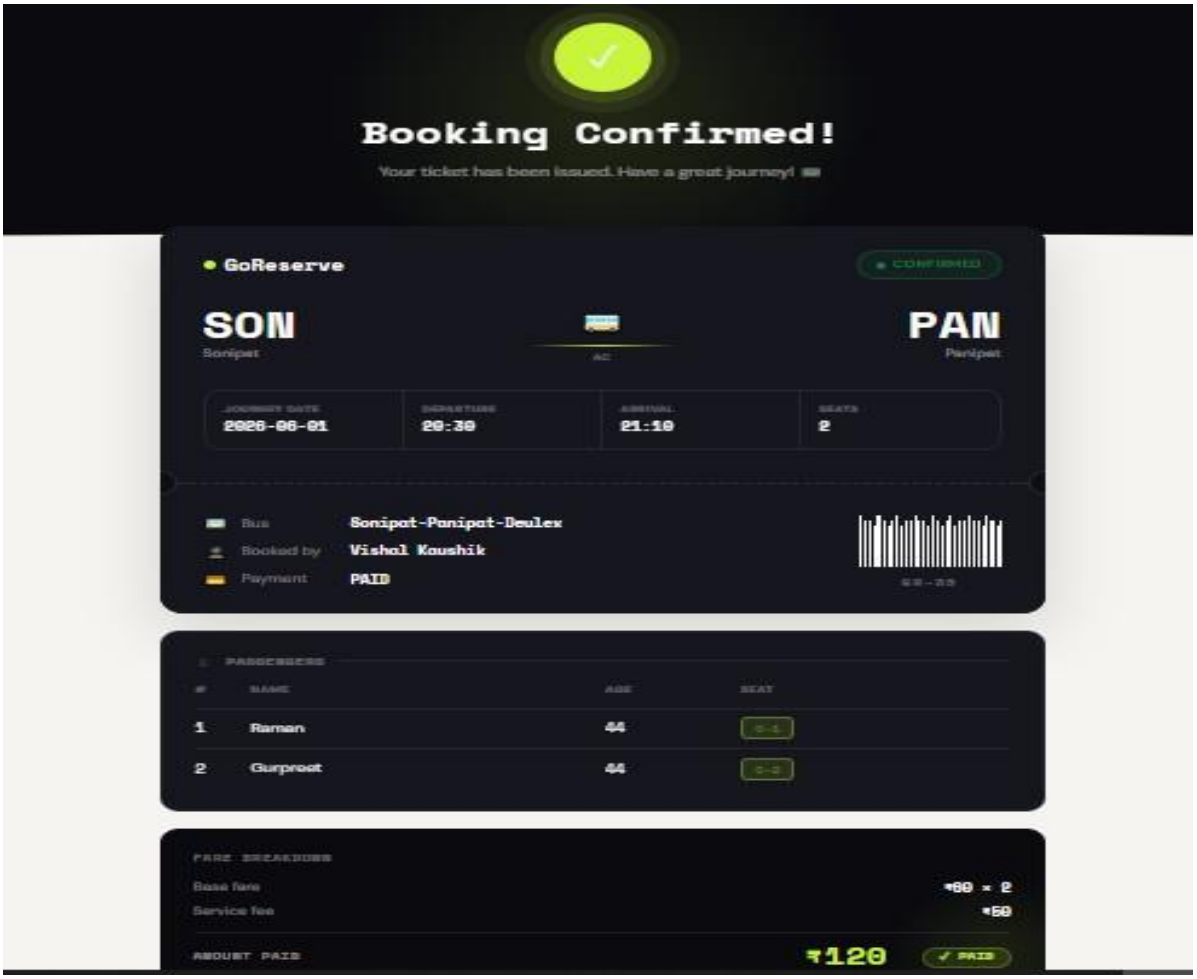


Figure 5.13

Explanation:

This screen is displayed after the successful completion of ticket booking and payment in the GoReserve system. The page provides users with a digital ticket containing complete journey and passenger information. In the example shown above, the booking confirmation is generated for the “Sonipat–Panipat–Deulex” bus route. The ticket displays important travel details such as source and destination, journey date, departure and arrival time, number of seats booked, and booking status. Passenger information including passenger names, age, and allocated seat numbers is also shown clearly. The fare breakdown section displays the base fare, service fee, and total amount paid for the booking. A payment status indicator confirms that the payment was completed successfully. This module helps users verify their reservation details and acts as a digital proof of booking within the system.

5.5.14 Booking History Page

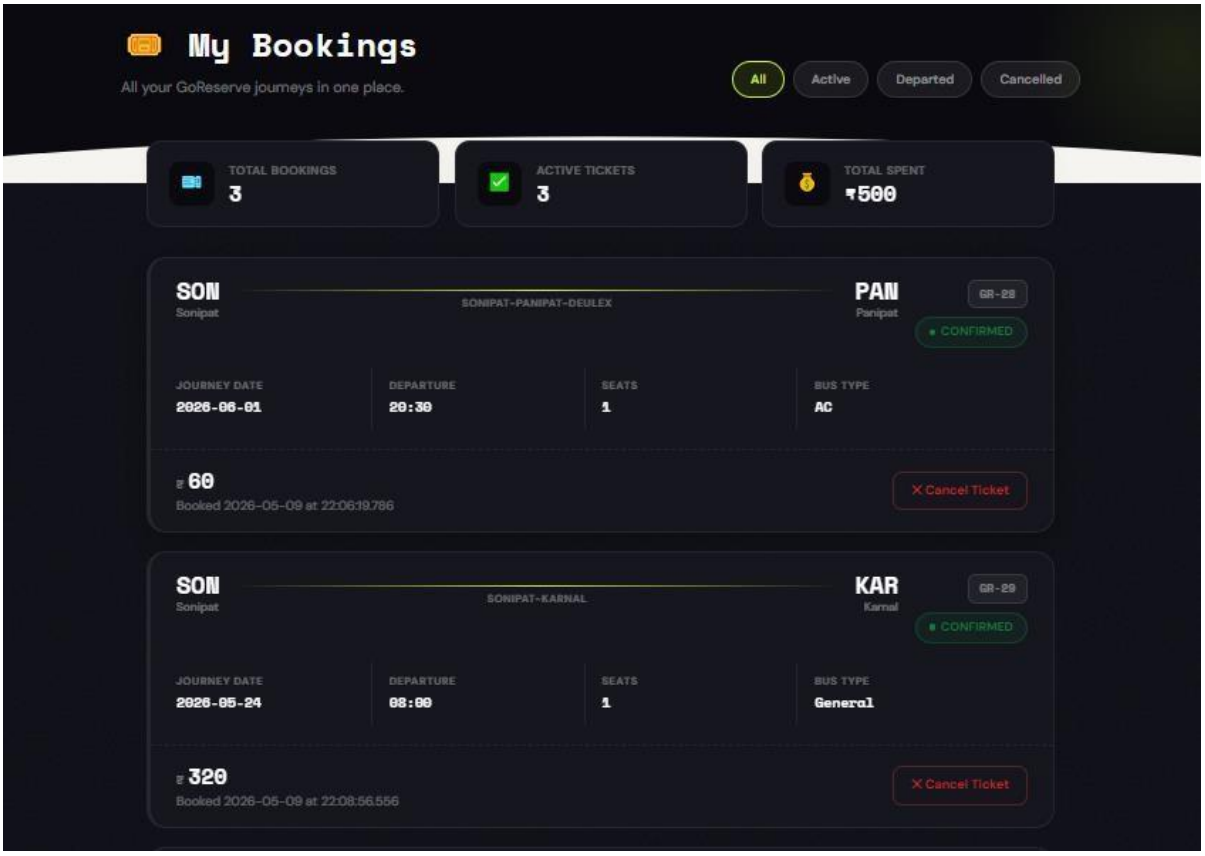


Figure 5.14

Explanation:

This screen represents the booking history module of the GoReserve system, where users can view and manage all their booked tickets in one place. The page displays booking statistics such as total bookings, active tickets, and total amount spent by the user. In the example shown above, the user has booked tickets for the “Sonipat–Panipat–Deulex” and “Sonipat–Karnal” routes. Each booking card contains important journey information including source, destination, journey date, departure time, number of seats, bus type, fare amount, and booking status. The system also provides filter options such as All, Active, Departed, and Cancelled to help users organize their bookings efficiently. A “Cancel Ticket” button is available for users to cancel their reservations when required. This module improves user convenience by maintaining a complete and organized history of all bookings within the system.

Chapter 6

TESTING

6.1 Introduction to Testing:

Testing is an important part of software development that helps ensure the system works correctly and meets the requirements of users. No matter how well a system is designed or developed, there can always be errors or unexpected behavior. Testing helps in identifying these issues and fixing them before the system is fully used. It plays a key role in improving the quality, reliability, and performance of the application.

In the GoReserve project, testing is carried out to verify that all the features of the system are working properly. This includes checking functionalities such as user registration, login, bus search, ticket booking, payment processing, and booking cancellation. Each feature is tested to make sure it produces the correct output for different types of inputs.

The main goal of testing in this project is to ensure that the system behaves as expected under different conditions. For example, valid inputs should give correct results, while invalid inputs should be handled properly with appropriate error messages. This helps in making the system more user-friendly and reliable.

In this project, testing is mainly done using **manual testing techniques**. This means that different scenarios are tested by manually entering inputs and observing the outputs. Manual testing is simple and effective for small to medium-sized projects like GoReserve. It allows the developer to understand how the system behaves from a user's perspective.

Testing is not done only at the end of the project but is performed throughout the development process. Individual modules such as login, booking, and admin operations are tested separately first. After that, these modules are combined and tested together to ensure smooth interaction between them.

Another important aspect of testing is checking how the system handles errors and unusual situations. For example, entering incorrect login details or leaving fields empty should not crash the system. Instead, the system should respond properly with clear messages.

6.1.2 Testing Strategies

The testing strategy for the GoReserve system is designed to ensure that all functionalities work correctly and the system performs as expected under different conditions. Since this is a web-based application, the main focus is on verifying user interactions, data handling, and overall system behaviour. The testing approach used in this project is mainly based on **manual testing**, which is suitable for projects of this scale.

In this project, testing is carried out in a structured manner by dividing it into different stages. Each module of the system is tested individually first, and then all modules are tested together. This approach helps in identifying errors at an early stage and ensures that the system works smoothly when all components are integrated.

The first step in the testing strategy is to identify the important features that need to be tested. These include user registration, login functionality, bus search, ticket booking, payment processing, and booking cancellation. These features are tested with both valid and invalid inputs to check how the system responds.

Another important part of the strategy is **input validation testing**. Different types of inputs are provided to the system, such as empty fields, incorrect data, and invalid login credentials. This helps in checking whether the system can handle incorrect inputs properly and display appropriate error messages.

The system is also tested for **end-to-end functionality**, where the complete flow is checked. For example, a user logs in, searches for a bus, books a ticket, makes a payment, and then views the booking history. This ensures that all modules are properly connected and working together.

Basic **security testing** is also considered in this project. The system checks whether unauthorized users can access restricted pages. Only authenticated users are allowed to perform booking operations, and admin functionalities are restricted to admin users only.

The results of testing are recorded in the form of test cases, where expected outputs are compared with actual outputs. If the system behaves as expected, the test case is marked as passed; otherwise, it is marked as failed.

6.1.3 Types of Testing

In the GoReserve project, different types of testing are performed to ensure that the system works correctly and provides a smooth experience to users. Since the project is developed as a web application, the testing mainly focuses on functionality, user interaction, and system behavior.

One of the main types of testing performed is **functional testing**. In this type of testing, all the features of the system are checked to verify whether they are working as expected. Functions such as user registration, login, bus search, ticket booking, payment process, and booking cancellation are tested carefully. Each function is executed with valid inputs to ensure correct results.

Another important type is **validation testing**. This testing checks how the system behaves when incorrect or unexpected inputs are provided. For example, entering wrong login details, leaving required fields empty, or entering invalid data. The system should handle such cases properly by displaying appropriate error messages instead of crashing.

Integration testing is also performed to verify that different modules of the system work together smoothly. In this testing, the flow between modules is checked. For example, after logging in, the user should be able to search for buses, book tickets, and make payments without any issues. This ensures that data flows correctly between different parts of the system.

The project also includes **system testing**, where the entire application is tested as a whole. This helps in checking the overall performance of the system. Complete workflows are tested from start to end to ensure that all features are properly connected and functioning together.

In addition, **basic security testing** is carried out to ensure that unauthorized users cannot access restricted areas of the system. Only registered users can log in and perform booking operations, while admin functionalities are protected and accessible only to admin users.

Overall, these types of testing help in ensuring that the GoReserve system is reliable, secure, and user-friendly. By performing different types of testing, the system is checked from multiple perspectives, reducing the chances of errors and improving overall quality.

6.1.4 Test Phases

The testing of the GoReserve system is carried out in a structured manner by dividing it into different phases. Each phase focuses on a specific aspect of testing and helps in identifying errors at different stages of development. This step-by-step approach ensures that the system is reliable, efficient, and free from major issues.

🔹 Phase 1: Test Planning

In this phase, the overall testing process is planned. The main objective is to identify what needs to be tested and how testing will be performed. The important features of the system, such as user registration, login, bus search, booking, payment, and cancellation, are identified. A manual testing approach is selected, and test cases are prepared for different scenarios.

🔹 Phase 2: Unit Testing

In this phase, individual modules of the system are tested separately. Each module is checked to ensure that it is working correctly on its own. For example, the login module is tested to verify whether users can log in with valid credentials, and the booking module is tested to ensure that booking details are stored correctly. This helps in identifying errors at an early stage.

🔹 Phase 3: Integration Testing

After testing individual modules, they are combined and tested together in this phase. The main purpose is to check whether different modules interact properly with each other. For example, after logging in, the user should be able to search for buses and proceed with booking without any issues. This phase ensures smooth data flow between modules.

🔹 Phase 4: System Testing

In this phase, the entire system is tested as a complete application. All functionalities are checked together to verify overall performance. End-to-end scenarios are tested, such as logging in, searching for a bus, booking a ticket, making payment, and viewing booking history. This helps in ensuring that the system works correctly as a whole.

⬙ Phase 5: Validation Testing

This phase focuses on checking how the system handles invalid or unexpected inputs. For example, entering incorrect login details, leaving fields empty, or providing invalid data. The system should respond with appropriate error messages and should not crash. This improves system reliability and user experience.

⬙ Phase 6: Result Evaluation

In the final phase, the results of all test cases are analyzed. The actual output is compared with the expected output to determine whether the system is working correctly. Most of the test cases are successfully passed, which indicates that the system is functioning properly and meets the required objectives.

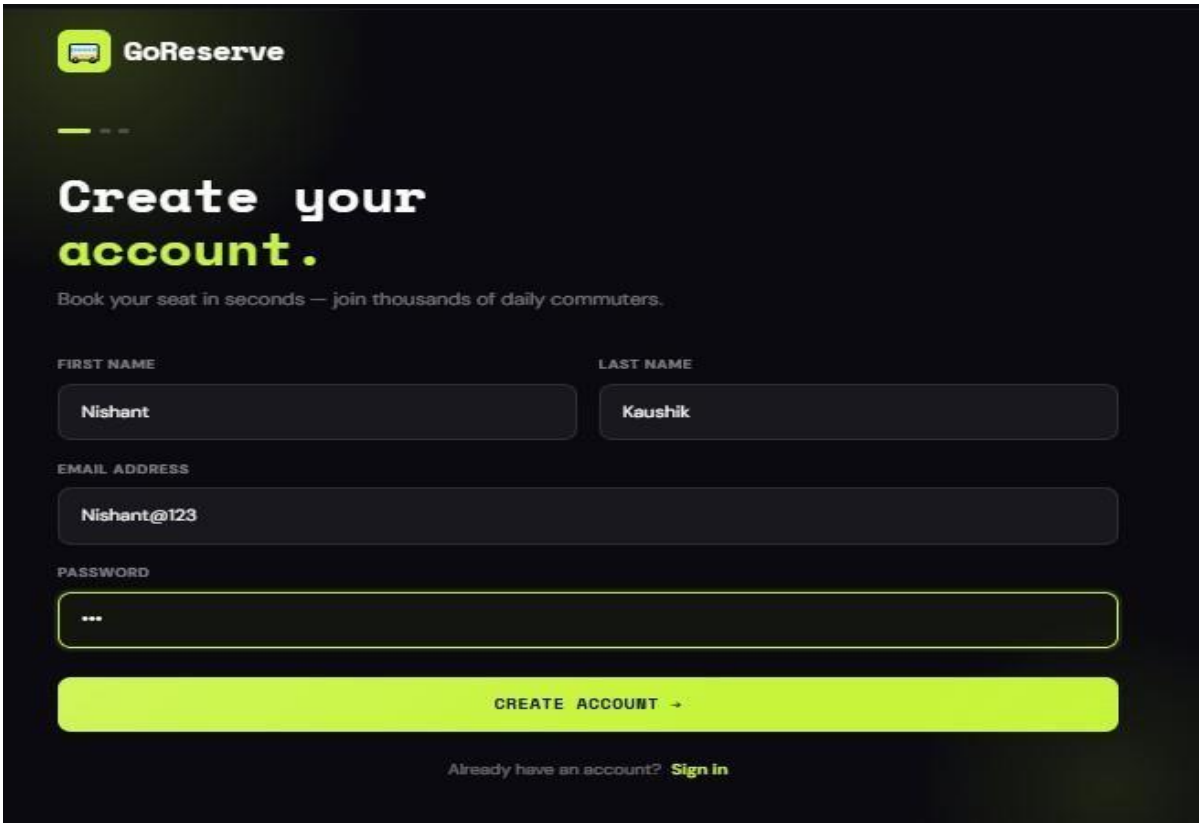
6.2 Test Cases and Results

Test Case 1: User Registration

Test Case ID	TC-01
Module	User Registration
Objective	Verify that a new user can register successfully
Input	Valid user details
Expected Result	User account should be created successfully
Actual Result	User registered successfully
Status	**Passed**

Table 6.1

Proof



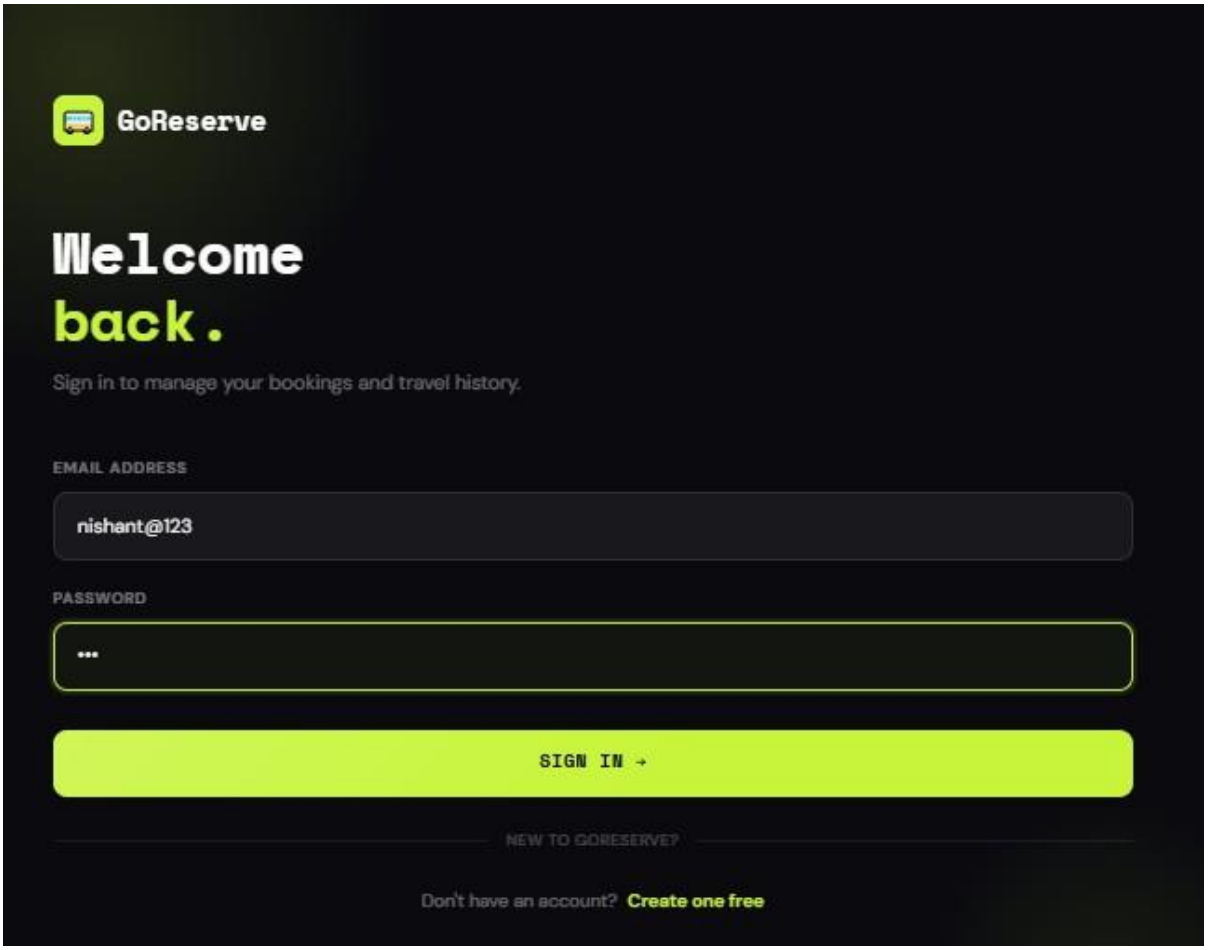
id	email	name	password
1	admin@admin.com	Admin Admin	$2a$10$8ufPequIEFkBCC/Pi2dBIeJB8bOpvcFWQWfhRr71J.bs9TMSDymtC
2	vishal@123	Vishal Kaushik	$2a$10$qTbEbnmuNcZikNYKynwMt.ZvkdRIMgZlWiWzzHdv5q/.swpVFtOVe
3	admin@admin	Admin Admin	$2a$10$roNZxnzkQ8uXncqbMctMfeJ8oMBHTLTPBhXoSK1mlDsrViP3Ewyz.
4	kaku@123	Kaku Kaushik	$2a$10$xjWyNtMIIEEJ3zEehprcgOc5kU0ttCjGdoFl9jBwZjU/5KSAcnyuW
5	ankit@123	Ankit Kaushik	$2a$10$zDy1HxFW8C0zOfy/Nggusuya.Jqj.3md9apm33xouYjXPh1iY.3vW
6	Nitin@123	Nitin Dahiya	$2a$10$9sQ1KdFrOe6dmFqdwe1Eh.F979ZyYc7udJ3odXsPtnwHOdcCWBLmi
7	Nishant@123	Nishant Kaushik	$2a$10$8dlZDKMPS3ggFniLV9a6gOwD8VI5AKyBNrDR3j.4M0rPz6ij4U.F2

Test Case 2: User Login

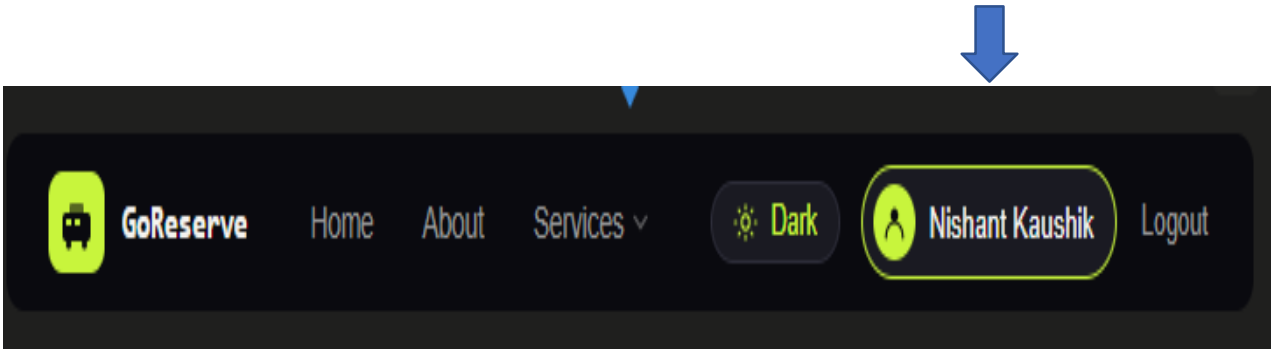
Test Case ID	TC-02
Module	User Login
Objective	Verify that registered users can log in
Input	Valid email and password
Expected Result	User should access dashboard/home page
Actual Result	Login successful
Status	**Passed**

Table 6.2

Proof



Name of User will be shown here

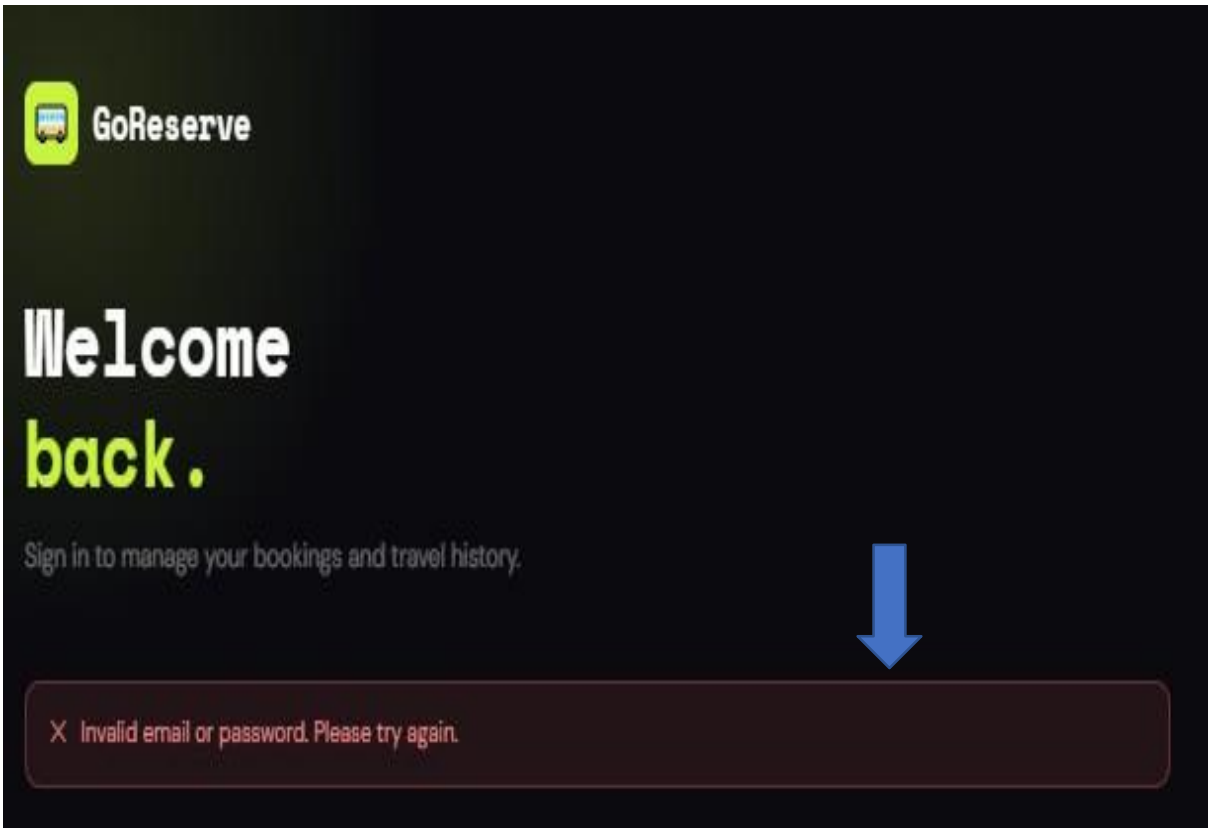


Test Case 3: Authentication

Test Case ID	TC-03
Module	Authentication
Objective	Verify system behaviour for invalid login
Input	Incorrect password
Expected Result	Error message should appear
Actual Result	Invalid credentials message displayed
Status	**Passed**

Table 6.3

Proof with Wrong Password

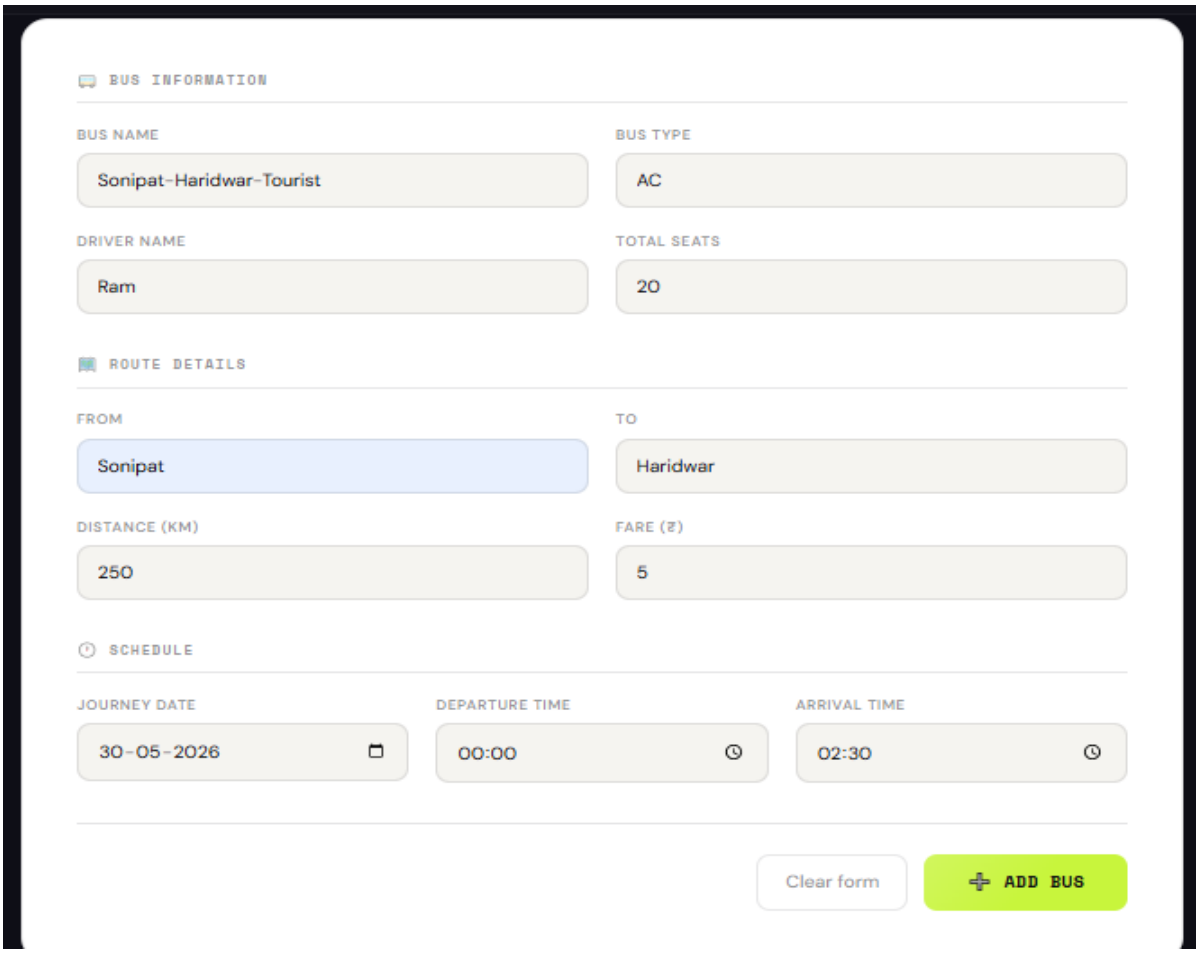


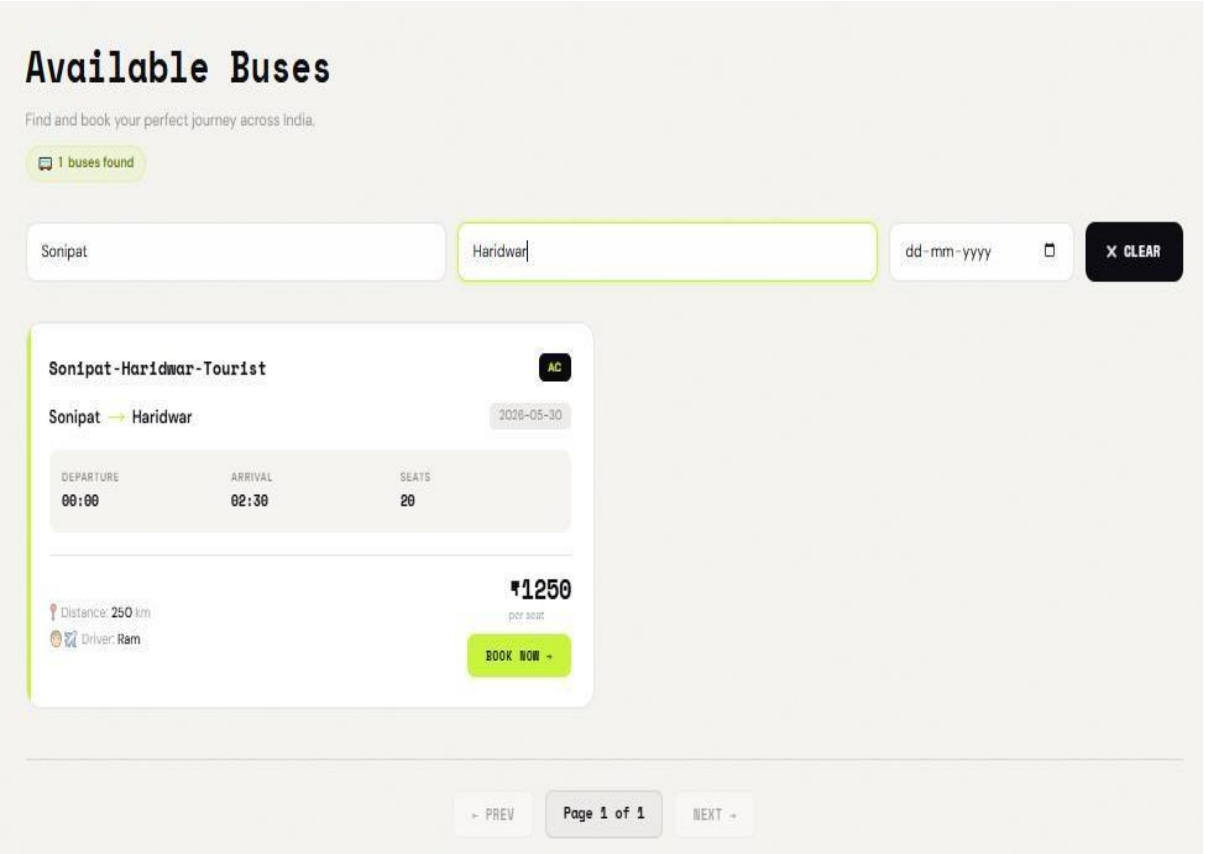
Test Case 4: Bus Search

Test Case ID	TC-04
Module	Bus Search
Objective	Verify that buses are displayed for valid route
Input	Source: Sonipat, Destination: Haridwar
Expected Result	Matching buses should be displayed
Actual Result	Bus list displayed successfully
Status	**Passed**

Table 6.4

Proof



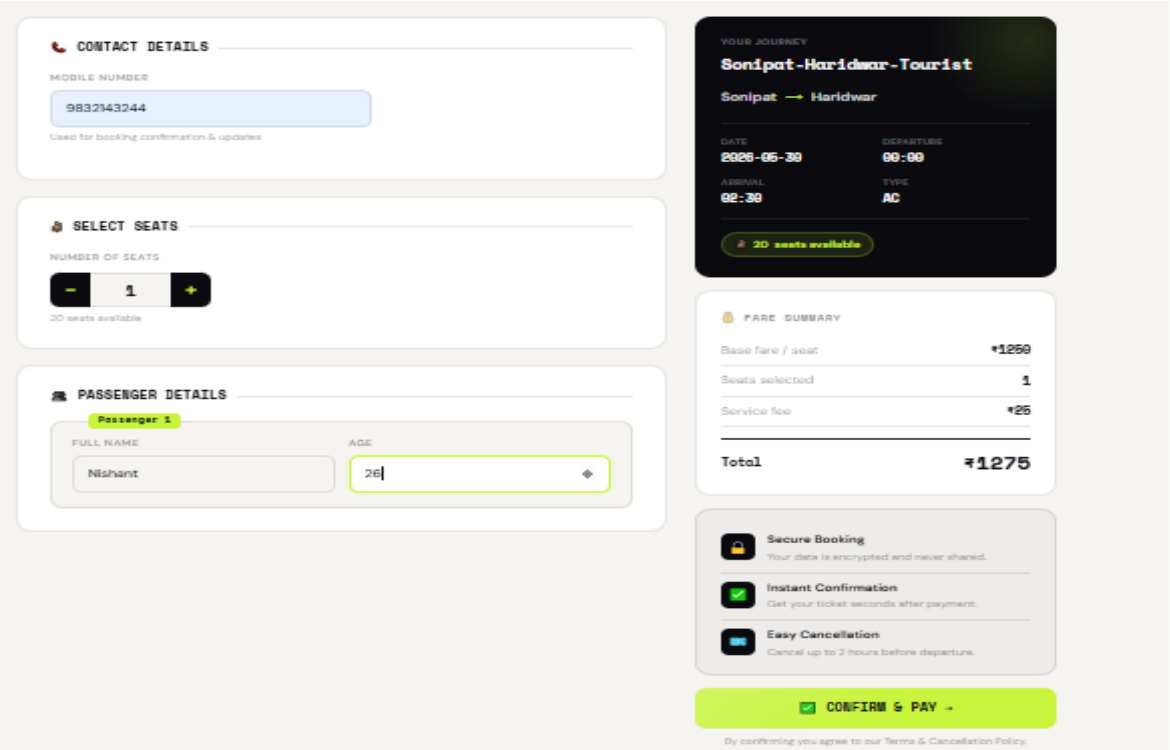


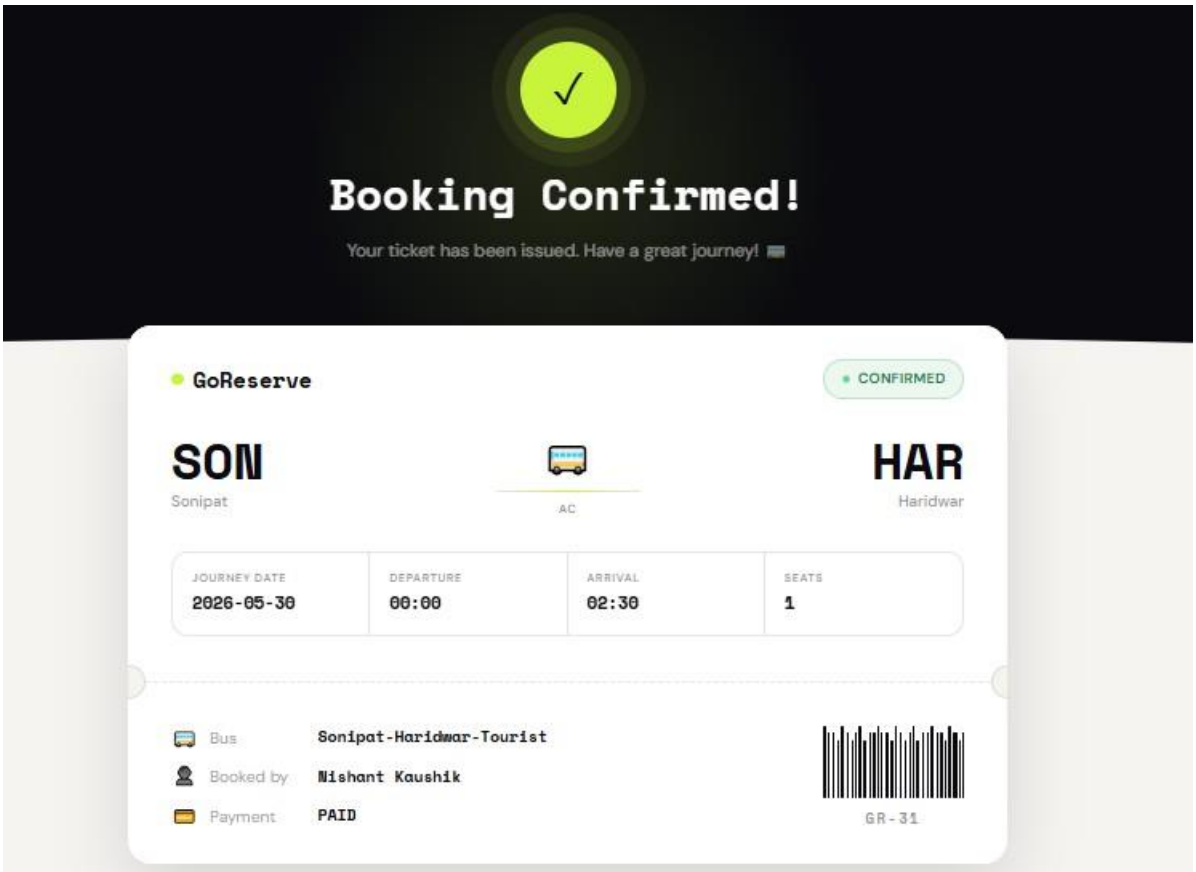
Test Case 5: Ticket Booking

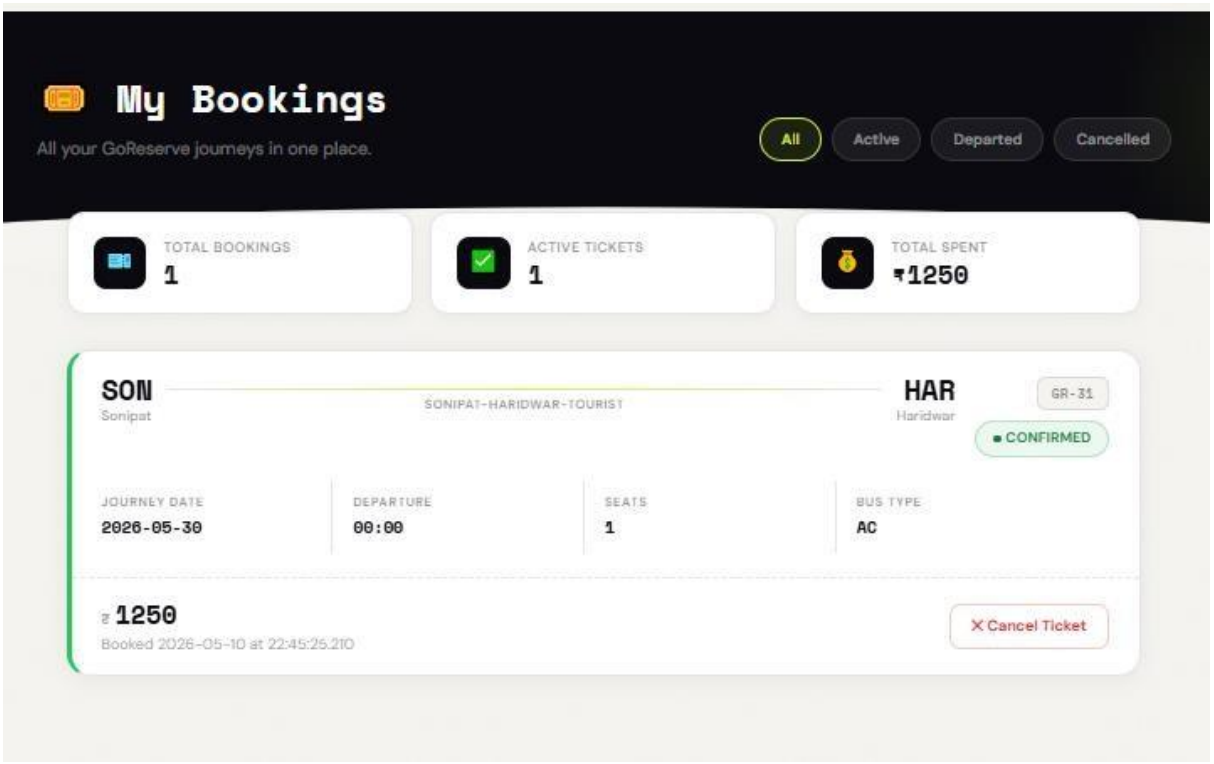
Test Case ID	TC-05
Module	Ticket Booking
Objective	Verify successful ticket booking
Input	Passenger details and seat selection
Expected Result	Booking confirmation should be generated
Actual Result	Ticket booked successfully
Status	**Passed**

Table 6.5

Proof


CONTACT DETAILS
MOBILE NUMBER
9832143244
Used for booking confirmation & updates
SELECT SEATS
NUMBER OF SEATS
1
20 seats available
PASSENGER DETAILS
Passenger 1
FULL NAME
Nishant
AGE
26
YOUR JOURNEY
Sonipat-Haridwar-Tourist
Sonipat → Haridwar
DATE
2026-05-30
DEPARTURE
00:00
ARRIVAL
02:30
TYPE
AC
20 seats available
FARE SUMMARY
Base fare / seat
₹1250
Seats selected
1
Service fee
₹25
Total
₹1275
Secure Booking
Your data is encrypted and never shared.
Instant Confirmation
Get your ticket seconds after payment.
Easy Cancellation
Cancel up to 2 hours before departure.
CONFIRM & PAY →
By confirming you agree to our Terms & Cancellation Policy.


Booking Confirmed!
Your ticket has been issued. Have a great journey!
GoReserve
CONFIRMED
SON
Sonipat
AC
HAR
Haridwar
JOURNEY DATE
2026-05-30
DEPARTURE
00:00
ARRIVAL
02:30
SEATS
1
Bus
Sonipat-Haridwar-Tourist
Booked by
Nishant Kaushik
Payment
PAID
GR-31

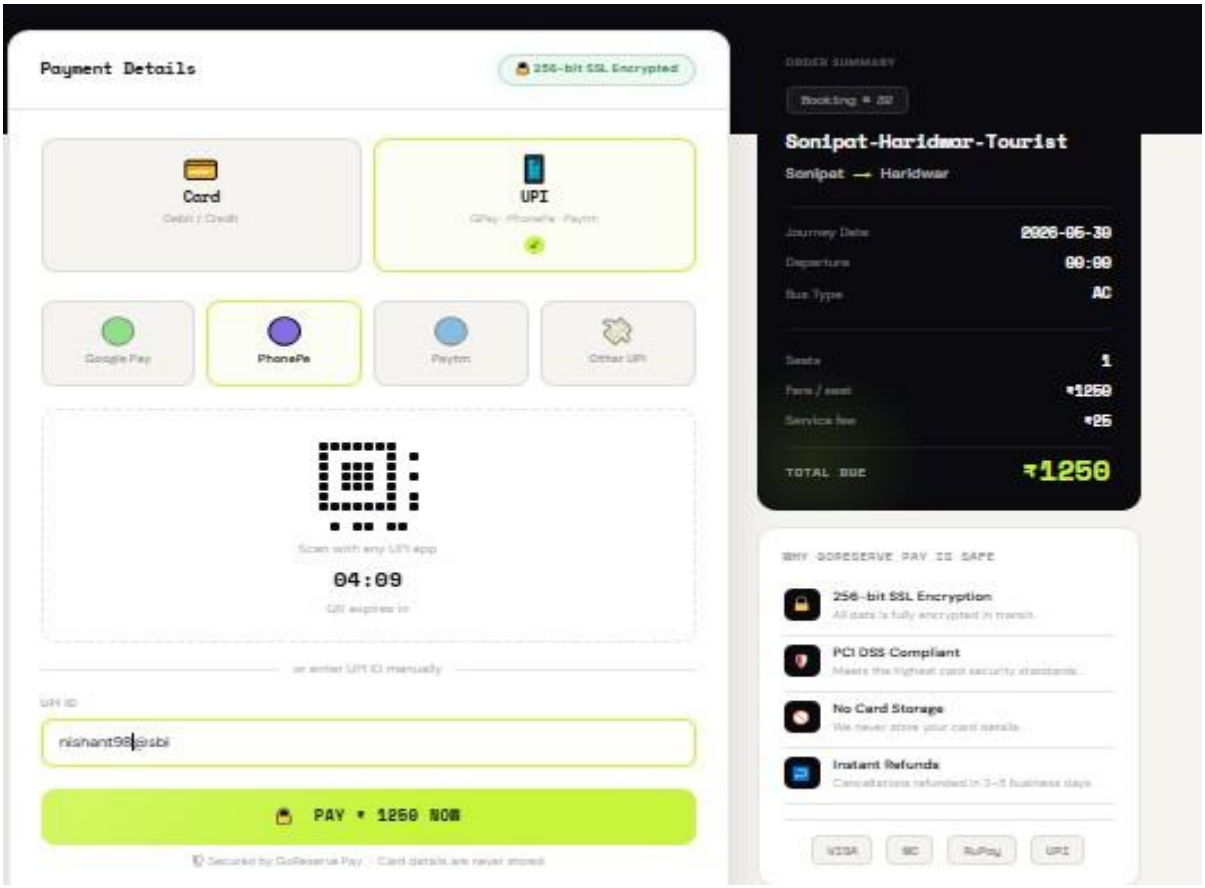


Test Case 6: Payment

Test Case ID	TC-06
Module	Payment
Objective	Verify payment process
Input	Card/UPI details
Expected Result	Payment status should update to PAID
Actual Result	Payment completed successfully
Status	**Passed**

Table 6.6

Proof



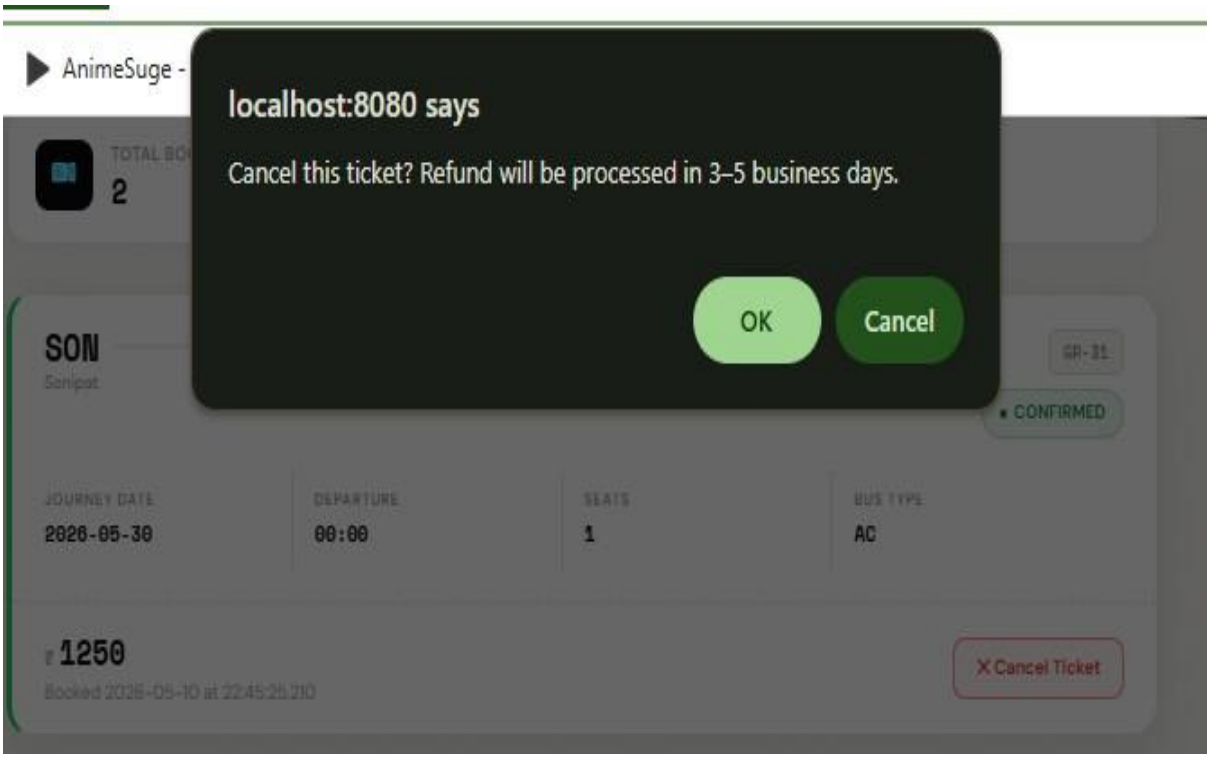

```
3 rows in set (0.0009 sec)
MySQL localhost:33060+ ssl goreserve SQL > select * from payment;
+----+--------------------+--------------+--------------+----------------+--------------+---------------------------+
| id | card_number        | payment_date | payment_mode | payment_status | payment_time | upi_id                    |
+----+--------------------+--------------+--------------+----------------+--------------+---------------------------+
|  2 | 7687568686         | NULL         | NULL         | CANCELLED      | NULL         |                           |
|  6 | 7687568686         | NULL         | NULL         | CANCELLED      | NULL         |                           |
|  9 |                    | NULL         | NULL         | CANCELLED      | NULL         | rahim@123                 |
| 15 |                    | NULL         | NULL         | CANCELLED      | NULL         | ram@hdfc                  |
| 17 |                    | NULL         | NULL         | CANCELLED      | NULL         | rahim@123                 |
| 18 | 7687568686         | NULL         | NULL         | CANCELLED      | NULL         |                           |
| 19 |                    | NULL         | NULL         | CANCELLED      | NULL         | dfgdffhdg                 |
| 20 | 12335667332256     | NULL         | NULL         | CANCELLED      | NULL         |                           |
| 21 |                    | NULL         | NULL         | CANCELLED      | NULL         | kapil@123                 |
| 22 | 980989909          | NULL         | NULL         | CANCELLED      | NULL         |                           |
| 23 | 980989909          | NULL         | NULL         | CANCELLED      | NULL         |                           |
| 30 | 980989909          | NULL         | NULL         | CANCELLED      | NULL         |                           |
| 31 |                    | NULL         | NULL         | CANCELLED      | NULL         | vishal@axl                |
| 35 |                    | NULL         | NULL         | CANCELLED      | NULL         | visha ushik490@gmail.com  |
| 37 |                    | NULL         | NULL         | CANCELLED      | NULL         | 1234@                     |
| 38 |                    | NULL         | NULL         | CANCELLED      | NULL         | 123@a                     |
| 42 | 1234 5678 9000 5555| NULL         | NULL         | PAID           | NULL         |                           |
| 45 | 2345 6214 1415 4363| NULL         | NULL         | PAID           | NULL         |                           |
| 46 | 1234 1234 1415 5354| NULL         | NULL         | PAID           | NULL         |                           |
| 47 | 4364 6554 2562 4624| NULL         | NULL         | PAID           | NULL         |                           |
| 48 |                    | NULL         | NULL         | PAID           | NULL         | nishant98@sbi             |
+----+--------------------+--------------+--------------+----------------+--------------+---------------------------+
21 rows in set (0.0011 sec)
```

Test Case 7: Booking Cancellation

Test Case ID	TC-07
Module	Booking Cancellation
Objective	Verify cancellation feature
Input	Cancel booking request
Expected Result	Booking should be cancelled
Actual Result	Booking cancelled successfully
Status	**Passed**

Table 6.7

Proof



Booking Database proof before and after cancel booking

```
21 rows in set (0.0011 sec)
MySQL  localhost:33060+ ssl  goreserve  SQL > select * from  booking;
+----+--------------+--------------+----------+----------+------+---------------+-----------+----------+
| id | booking_date | booking_time | bookseat | distance | fare | mobile_number | route_from | route_to |
+----+--------------+--------------+----------+----------+------+---------------+-----------+----------+
| 28 | 2026-05-09   | 22:06:19.786 |        1 |       50 |   60 | 9832143244    | Sonipat    | Panipat  |
| 29 | 2026-05-09   | 22:08:56.556 |        1 |       80 |  320 | 9832143244    | Sonipat    | Karnal   |
| 30 | 2026-05-09   | 22:53:38.031 |        2 |       50 |  120 | 9832143244    | Sonipat    | Panipat  |
| 31 | 2026-05-10   | 22:45:25.21  |        1 |      250 | 1250 | 9832143244    | Sonipat    | Haridwar |
| 32 | 2026-05-10   | 22:49:44.925 |        1 |      250 | 1250 | 9832143244    | Sonipat    | Haridwar |
+----+--------------+--------------+----------+----------+------+---------------+-----------+----------+
5 rows in set (0.0010 sec)
MySQL  localhost:33060+ ssl  goreserve  SQL > select * from  booking;
+----+--------------+--------------+----------+----------+------+---------------+-----------+----------+
| id | booking_date | booking_time | bookseat | distance | fare | mobile_number | route_from | route_to |
+----+--------------+--------------+----------+----------+------+---------------+-----------+----------+
| 28 | 2026-05-09   | 22:06:19.786 |        1 |       50 |   60 | 9832143244    | Sonipat    | Panipat  |
| 29 | 2026-05-09   | 22:08:56.556 |        1 |       80 |  320 | 9832143244    | Sonipat    | Karnal   |
| 30 | 2026-05-09   | 22:53:38.031 |        2 |       50 |  120 | 9832143244    | Sonipat    | Panipat  |
+----+--------------+--------------+----------+----------+------+---------------+-----------+----------+
```

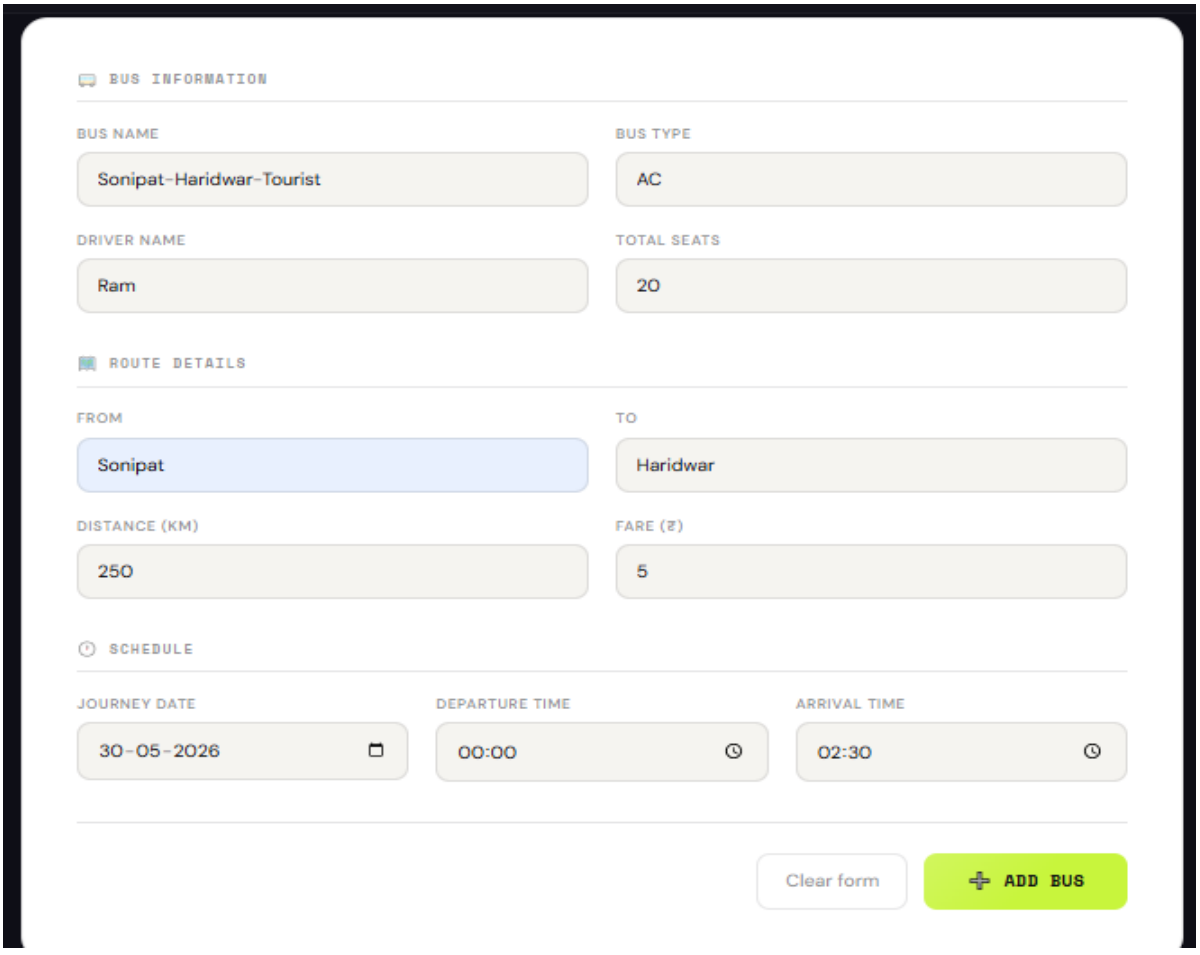
```
3 rows in set (0.0006 sec)
MySQL  localhost:33060+ ssl  goreserve  SQL > select * from archived_booking;
+----+--------------+--------------+----------+------+------------+----------+-----------+
| id | booking_date | booking_time | bookseat | fare | route_from | route_to | status    |
+----+--------------+--------------+----------+------+------------+----------+-----------+
|  1 | 2025-07-17   | 20:01:24.785 |        1 | 1000 | Sonipat    | Delhi    | CAN  LED  |
|  2 | 2025-07-17   | 20:12:01.356 |        1 | 1000 | Sonipat    | Delhi    | CAN  LED  |
|  3 | 2025-07-17   | 20:31:00.508 |        1 | 1000 | Sonipat    | Delhi    | CAN  LED  |
|  4 | 2025-07-20   | 18:20:13.262 |        2 | 2000 | Sonipat    | Delhi    | CAN  LED  |
|  5 | 2025-07-21   | 23:26:07.158 |        1 | 1000 | Sonipat    | Delhi    | CAN  LED  |
|  6 | 2025-07-22   | 19:09:52.807 |        1 | 1000 | Sonipat    | Delhi    | CAN  LED  |
|  7 | 2025-07-23   | 16:26:12.736 |        1 | 1000 | Sonipat    | Delhi    | CAN  LED  |
|  8 | 2025-07-30   | 20:45:14.201 |        2 |  160 | Panipat    | Sonipat  | CAN  LED  |
|  9 | 2025-07-30   | 22:00:10.165 |        1 |  200 | Panipat    | Sonipat  | CAN  LED  |
| 11 | 2025-09-09   | 22:33:35.829 |        1 |  550 | Sonipat    | Delhi    | CAN  LED  |
| 12 | 2025-12-27   | 17:06:58.594 |        1 |  550 | Sonipat    | Delhi    | CAN  LED  |
| 13 | 2026-02-07   | 11:08:26.864 |        1 | 8000 | Sonipat    | Mumbai   | CAN  LED  |
| 15 | 2026-02-09   | 12:19:08.559 |        1 |  550 | Sonipat    | Delhi    | CAN  LED  |
| 17 | 2026-03-09   | 15:52:59.705 |        1 |  550 | Sonipat    | Delhi    | CAN  LED  |
| 18 | 2026-03-09   | 15:58:23.206 |        1 |  550 | Sonipat    | Delhi    | CAN  LED  |
| 19 | 2026-03-09   | 17:12:26.43  |        2 | 1100 | Sonipat    | Delhi    | CA   LED  |
| 31 | 2026-05-10   | 22:45:25.21  |        1 | 1250 | Sonipat    | Haridwar | CANCELLED |
| 32 | 2026-05-10   | 22:49:44.925 |        1 | 1250 | Sonipat    | Haridwar | CANCELLED |
+----+--------------+--------------+----------+------+------------+----------+-----------+
```

Test Case 8: Admin Bus Management

Test Case ID	TC-08
Module	Admin Bus Management
Objective	Verify admin can add buses
Input	Bus details
Expected Result	Bus should be stored in database
Actual Result	Bus added successfully
Status	**Passed**

Table 6.8

Proof

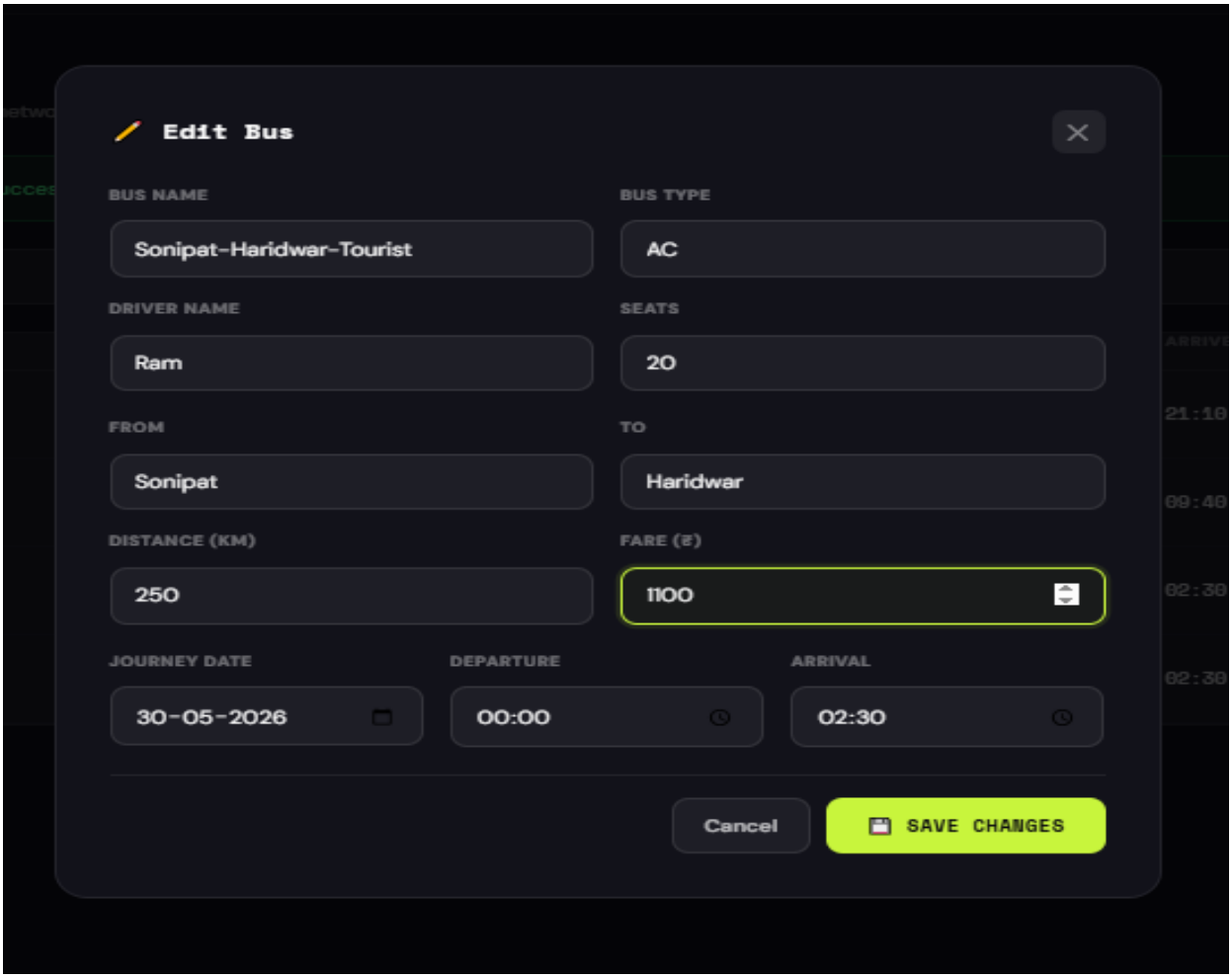


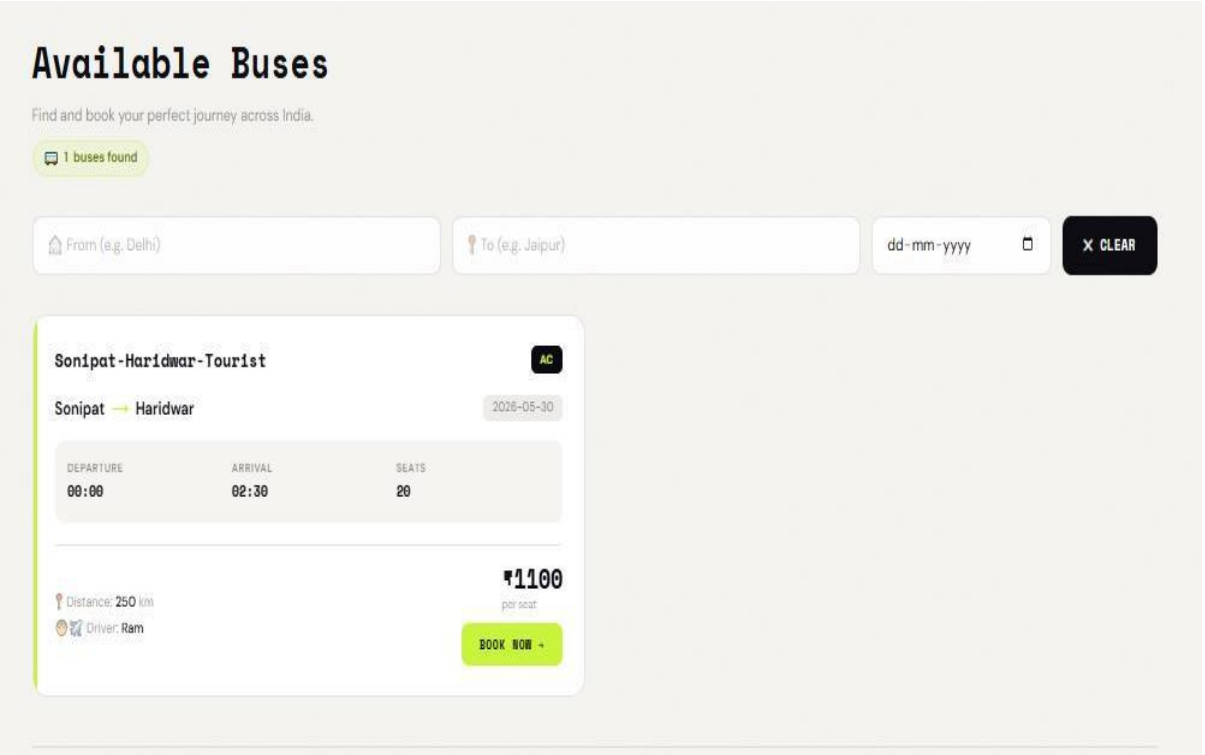
Test Case 9: Bus Management

Test Case ID	TC-09
Module	Bus Management
Objective	Verify admin can edit bus details
Input	Updated bus information
Expected Result	Updated information should be saved
Actual Result	Bus updated successfully
Status	**Passed**

Table 6.9

Proof



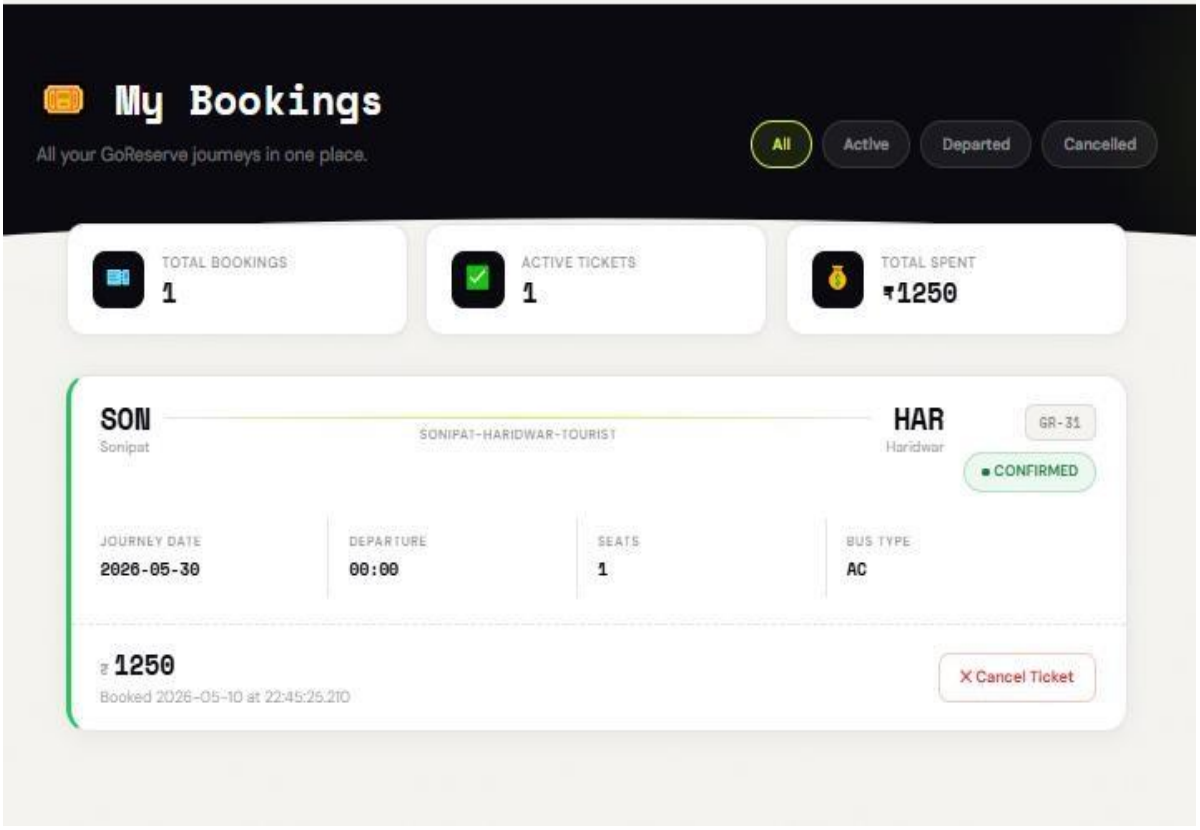


Test Case 10: Booking History

Test Case ID	TC-10
Module	Booking History
Objective	Verify user can view previous bookings
Input	Logged-in user
Expected Result	Booking records should display
Actual Result	Booking history displayed successfully
Status	**Passed**

Table 6.10

Proof


My Bookings
All your GoReserve journeys in one place.
All
Active
Departed
Cancelled
TOTAL BOOKINGS
1
ACTIVE TICKETS
1
TOTAL SPENT
₹1250
SON
Sonipat
SONIPAT-HARIDWAR-TOURIST
HAR
Haridwar
GR-31
CONFIRMED
JOURNEY DATE
2026-05-30
DEPARTURE
00:00
SEATS
1
BUS TYPE
AC
₹ 1250
Booked 2026-05-10 at 22:45:25.210
Cancel Ticket

6.3 Test Table

Test Case ID	Module	Test Scenario	Input	Expected Result	Actual Result	Status
TC-01	User Registration	Register a new user with valid details	Valid name, email, password	User account should be created successfully	User registered successfully	**Passed**
TC-02	User Login	Login with valid credentials	Correct email and password	User should access booking page	Login successful	**Passed**
TC-03	Authentication	Login with invalid credentials	Incorrect password	Error message should appear	Invalid credentials message displayed	**Passed**
TC-04	Bus Search	Search buses for valid route	Source: Sonipat, Dest: Panipat	Matching buses should be displayed	Bus list displayed successfully	**Passed**
TC-05	Ticket Booking	Book ticket with valid details	Passenger details & seat selection	Booking confirmation should be generated	Ticket booked successfully	**Passed**
TC-06	Payment Processing	Complete payment process	Card/UPI details	Payment status should update to PAID	Payment completed successfully	**Passed**
TC-07	Booking Cancellation	Cancel booked ticket	Cancellation request	Booking should be cancelled successfully	Booking cancelled successfully	**Passed**
TC-08	Admin Bus Management	Add a new bus	Valid bus details	Bus should be added to database	Bus added successfully	**Passed**
TC-09	Bus Management	Update existing bus details	Updated bus information	Updated details should be saved	Bus updated successfully	**Passed**
TC-10	Booking History	View previous bookings	Logged-in user account	Booking records should display	Booking history displayed successfully	**Passed**

All 10 test cases passed successfully.

Table 6.11

6.3.1 Result Analysis

The result analysis phase is used to evaluate the outcomes of all the test cases performed on the GoReserve system. In this phase, the actual outputs obtained during testing are compared with the expected outputs to determine whether the system is working correctly.

During testing, all major functionalities of the system such as user registration, login, bus search, ticket booking, payment process, and booking cancellation were executed successfully. The system responded correctly to valid inputs and produced the expected results in each case. This indicates that the core features of the application are functioning properly.

The system was also tested with invalid inputs, such as incorrect login credentials, empty fields, and invalid data entries. In these cases, the system handled the inputs effectively by displaying appropriate error messages without crashing. This shows that proper validation and error handling mechanisms are implemented in the system.

Integration testing results showed that different modules of the system are well connected and work smoothly together. For example, users were able to log in, search for buses, complete bookings, and view booking history without any issues. This confirms that data flow between modules is consistent and reliable.

The performance of the system was also satisfactory during testing. The application responded quickly to user actions and did not show any major delays or failures under normal conditions.

Chapter 7

(System Implementation)

This chapter explains the complete implementation process of the GoReserve project, including project setup, development environment configuration, database creation, dependency management, frontend and backend development, and application execution. The chapter also describes how different technologies such as Spring Boot, Thymeleaf, Spring Security, Hibernate/JPA, and MySQL were integrated to build the system successfully.

The implementation process started with creating the project using Spring Initializr and importing it into Eclipse Spring Tool Suite (STS). Required dependencies for web development, database connectivity, template rendering, and security were added during project setup. The backend of the application was developed using Java and Spring Boot, while the frontend interface was designed using HTML, CSS, Bootstrap, Tailwind CSS, JavaScript, and Thymeleaf templates.

The GoReserve project uses MySQL as the relational database management system, where all important information such as users, buses, bookings, archived bookings, and payments is stored. Database connectivity was configured using the application.properties file, and Hibernate/JPA was used for ORM-based database interaction.

This chapter further explains the structure of the project, package organization, security implementation using Spring Security, authentication flow, and the process of running the application on the embedded Apache Tomcat server. The objective of this chapter is to provide a clear understanding of how the GoReserve system was practically implemented from initial setup to final execution.

7.1 Steps to Setup Database

The GoReserve Online Bus Reservation System uses MySQL Workbench and MySQL for database management and storage. The database is connected with the Spring Boot application using Spring Data JPA and Hibernate for ORM-based operations. The following steps were performed to set up the database for the project.

Step 1: Install MySQL and MySQL Workbench

- Download and install MySQL Server.
- Install MySQL Workbench for database creation and management.
- Configure the root username and password during installation.

Step 2: Open MySQL Workbench

- Launch MySQL Workbench.
- Connect to the local MySQL server using the configured username and password.
- Open a new SQL editor tab.

Step 3: Create the Database

Execute the following SQL query to create the database used in the project.

```
CREATE DATABASE GoReserve;
```

After creating the database, select it using:

```
USE GoReserve;
```

Step 4: Configure Spring Boot Database Connection

The database connection was configured inside the **application.properties** file of the Spring Boot project.

```
spring.datasource.url=jdbc:mysql://localhost:3306/GoReserve
spring.datasource.username=root
spring.datasource.password=your_password

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
```

This configuration connects the Spring Boot application with the GoReserve MySQL database.

Step 5: Create Entity Classes

Entity classes were created in Spring Boot using JPA annotations. These entity classes automatically generate database tables using Hibernate.

Main entity classes used in the project:

- User
- Role
- Bus
- Booking
- ArchivedBooking
- Payment

Annotations used:

- @Entity
- @Table
- @Id
- @GeneratedValue
- @ManyToOne
- @OneToMany

Step 6: Run the Spring Boot Application

After configuring the database and entity classes:

- Run the project as:

  Run As → Spring Boot App

- Spring Boot automatically creates the required tables inside the GoReserve database using Hibernate JPA.

Step 7: Verify Tables in MySQL Workbench

After running the application:

1. Refresh the schema in MySQL Workbench.
2. Open the GoReserve database.
3. Verify that all project tables are created successfully.

Expected tables include:

- users
- roles
- bus
- booking
- archived_booking
- payment

Step 8: Insert and Test Data

Sample data was added through the application interface, including:

- User registration
- Bus details
- Booking records
- Payment information

The inserted data was verified inside MySQL Workbench using SQL queries such as:

```
SELECT * FROM users;
SELECT * FROM booking;
SELECT * FROM payment;
```

Database Setup Summary

The database setup process successfully connected the Spring Boot application with the MySQL database named **GoReserve**. Hibernate and Spring Data JPA were used to automatically manage database tables and simplify CRUD operations within the project.

7.2 Steps to Setup the Project and Execution

The GoReserve Online Bus Reservation System was developed using Spring Boot, Maven, Thymeleaf, Spring Security, Hibernate/JPA, and MySQL. The following steps describe the complete setup and execution process of the project.

Step 1: Open Spring Initializr

The project was created using Spring Initializr.

Configuration Used:

- Project Type: Maven Project
- Language: Java
- Spring Boot Version: 3.3.0
- Group ID: JFS6WDE
- Artifact ID: OnlineBusTicketBooking
- Project Name: OnlineBusTicketBooking
- Packaging: Jar
- Java Version: 17

After configuration, the project was generated and downloaded as a ZIP file.

Step 2: Add Required Maven Dependencies

The following dependencies were added to the pom.xml file for project development.

Spring Boot Dependencies Used:

Dependency	**Purpose**
spring-boot-starter-web	Web application development
spring-boot-starter-thymeleaf	Thymeleaf template engine
spring-boot-starter-data-jpa	Database interaction using JPA/Hibernate

Dependency	Purpose
spring-boot-starter-security	Authentication and authorization
spring-boot-starter-validation	Form validation
thymeleaf-extras-springsecurity6	Security integration with Thymeleaf
mysql-connector-j	MySQL database connectivity
spring-boot-devtools	Automatic restart during development
lombok	Reduces boilerplate code
spring-boot-starter-test	Testing support
spring-security-test	Security testing support
h2 database	Runtime testing database

Step 3: Import Project into Eclipse STS

The downloaded project was imported into Spring Tool Suite using the following steps:

1. Open Eclipse STS
2. Click:

File → Import → Existing Maven Project

3. Select the extracted project folder
4. Click Finish

After importing, Maven automatically downloaded all required dependencies from the Maven repository.

Step 4: Configure Database Connectivity

Database connection settings were added inside the application.properties file.

```
spring.datasource.url=jdbc:mysql://localhost:3306/GoReserve
spring.datasource.username=root
spring.datasource.password=your_password

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

This configuration connected the Spring Boot application with the MySQL database named **GoReserve**.

Step 5: Create Project Packages

The project was organized into multiple packages for better structure and maintainability.

Main Packages:

- controller
- service
- serviceimpl
- repository
- entity
- config
- dto

Step 6: Develop Backend Modules

Backend functionality was implemented using Spring Boot.

Backend Components Included:

- Controllers for request handling
- Services for business logic
- Repositories for database operations

- Entity classes for database mapping
- Spring Security configuration for authentication

Step 7: Develop Frontend Interface

Frontend pages were created using:

- HTML5
- CSS3
- Bootstrap
- Tailwind CSS
- Thymeleaf
- JavaScript

Responsive and dark-mode enabled interfaces were developed for better user experience.

Step 8: Configure Spring Security

Spring Security was implemented for secure authentication and authorization.

Features Implemented:

- User login authentication
- Role-based access control
- BCrypt password encryption
- Admin and user redirection
- Logout functionality

Step 9: Run the Application

After completing development and configuration:

1. Right-click the project
2. Select:

Run As → Spring Boot App

The embedded Apache Tomcat server started automatically.

Console output confirmed successful execution of the application.

Step 10: Access the Application

The GoReserve application was accessed using the following URL:

http://localhost:8080

Users could then:

- Register accounts
- Login securely
- Search buses
- Book tickets
- Make payments
- View booking history

Project Execution Summary

The GoReserve Online Bus Reservation System was successfully implemented using Spring Boot and Maven-based project architecture. The integration of Spring Security, Hibernate/JPA, Thymeleaf, and MySQL enabled secure, scalable, and efficient execution of the application.

CHAPTER 8

Results and Discussion

8.1 Introduction to Results and Discussion:

This chapter presents the results obtained from the development and testing of the GoReserve system and provides a detailed discussion of its performance and functionality. After designing and implementing the system, it is important to evaluate how well it works in real scenarios and whether it meets the objectives defined at the beginning of the project.

The main purpose of this chapter is to analyze the outputs generated by the system and understand how effectively it performs different operations. This includes examining how the system responds to user inputs, how accurately it processes data, and how efficiently it handles different tasks such as booking, searching, and payment processing.

In addition to analyzing outputs, this chapter also focuses on evaluating the overall performance of the system. Factors such as response time, system stability, and ease of use are considered to determine whether the application provides a smooth and reliable user experience. The behavior of the system under different conditions, including valid and invalid inputs, is also discussed.

Another important aspect covered in this chapter is the discussion of results in relation to the project objectives. The system is evaluated to see how well it solves the problems identified in earlier chapters, such as reducing manual effort, improving booking efficiency, and providing a user-friendly interface.

Overall, this chapter helps in understanding the strengths of the GoReserve system and highlights how it improves the traditional bus reservation process. It also provides insights into the practical performance of the system and sets the foundation for the final conclusion of the project.

8.2 System Execution Overview:

The GoReserve system is designed as a web-based application that allows users and administrators to interact with the system through a browser. When the application is executed, it runs on a local server using Spring Boot with an embedded Tomcat server. Users can access the system through a web browser, where they are presented with the home page.

The execution of the system begins when a user opens the application and navigates to the home page. From there, the user can either register as a new user or log in using existing credentials. Once the user logs in successfully, they are redirected to the main dashboard, where they can access different features of the system.

The next step in the flow is the bus search process. The user enters the source and destination, and the system retrieves matching bus details from the database. The results are displayed to the user in a clear and organized format. After selecting a suitable bus, the user proceeds to the booking page.

During the booking process, the user enters passenger details. The system validates the input and calculates the fare based on predefined pricing. Once the details are confirmed, the booking is stored in the database, and the user is redirected to the payment process.

In the payment stage, the system simulates the payment process and updates the payment status accordingly. After successful payment, the system generates a booking confirmation, which can be viewed later in the booking history section.

Users also have the option to cancel bookings. When a cancellation request is made, the system updates the booking status and moves the record to the archived booking table.

From the admin side, the execution flow includes logging into the admin panel, managing buses, and monitoring user and booking data. Admins can add new buses, update details, and ensure that the system data remains accurate.

8.3 Output Analysis

The output analysis of the GoReserve system focuses on examining the results generated by the application in response to different user actions. It helps in understanding whether the system is producing correct, consistent, and user-friendly outputs.

When users interact with the system, such as during registration or login, the outputs are displayed immediately based on the input provided. For valid inputs, the system successfully registers users and allows them to log in without any issues. In case of invalid inputs, such as incorrect credentials or missing fields, the system displays appropriate error messages. This ensures that users are guided properly and prevents incorrect data from being processed.

During the bus search process, when users enter valid source and destination details, the system retrieves relevant data from the database and displays a list of available buses. The output is presented in a clear and organized format, making it easy for users to select a suitable option. If no buses are available for a given route, the system displays a message indicating that no results were found.

In the booking process, the system generates outputs such as booking confirmation details, including passenger information, travel details, and total amount. After the booking is completed, users can view their booking history, where all records are displayed in a structured manner.

The payment process also produces clear outputs. When a payment is successful, the system displays a confirmation message along with the payment status. In cases where the payment is cancelled or fails, the system updates the status accordingly and informs the user.

Additionally, when a booking is cancelled, the system updates the booking status and provides a message indicating that the cancellation has been processed. This ensures transparency and improves user trust.

8.4 Functional Results

The functional results of the GoReserve system describe how effectively each feature of the application performs when tested in real scenarios. The system was developed with multiple functionalities such as user management, bus search, booking, payment, and admin operations.

Each of these features was tested to ensure that they are working correctly and providing the expected results.

The **user registration and login functionality** worked successfully during testing. New users were able to register by entering valid details, and the system stored the data correctly in the database. During login, users were authenticated properly, and only valid credentials allowed access to the system. Invalid login attempts were handled with appropriate error messages.

The **bus search feature** also performed as expected. When users entered valid source and destination details, the system displayed a list of available buses. The results were accurate and retrieved efficiently from the database. In cases where no buses were available, the system displayed a clear message indicating that no results were found.

The **booking functionality** is one of the core features of the system, and it worked smoothly during testing. Users were able to select a bus, enter passenger details, and confirm their booking. The system stored booking information correctly and generated confirmation messages. The booking history feature also worked properly, allowing users to view their previous bookings.

The **payment functionality** was implemented as a simulated process, and it performed correctly. Users were able to complete the payment process, and the system updated the payment status accordingly. Both successful and cancelled payment scenarios were handled properly.

The **booking cancellation feature** also worked as expected. Users were able to cancel their bookings, and the system updated the booking status. The cancelled bookings were moved to the archived booking section, and appropriate messages were displayed to the user.

From the admin side, the **bus management functionality** was tested successfully. Admin users were able to add new buses, update existing details, and manage system data effectively. The admin dashboard provided access to important system information.

8.5 User Interface Evaluation

The user interface of the GoReserve system plays an important role in providing a smooth and convenient experience to users. A well-designed interface makes it easier for users to interact

with the system and perform tasks without confusion. In this project, the interface is designed with simplicity and clarity in mind.

The layout of the system is clean and well-organized, allowing users to easily navigate between different pages. Important features such as login, registration, bus search, and booking are clearly visible and accessible. The use of proper spacing, headings, and buttons helps users understand the system quickly, even if they are using it for the first time.

The navigation structure of the application is simple and user-friendly. Users can move from one page to another without difficulty. For example, after logging in, users can easily access the bus search page, view results, and proceed to booking without any complex steps. This smooth navigation improves the overall usability of the system.

The system also provides clear feedback to users during different operations. For example, when a user logs in successfully, books a ticket, or cancels a booking, the system displays confirmation messages. Similarly, error messages are shown when invalid inputs are entered. These messages help users understand what is happening and guide them in using the system correctly.

Responsiveness is another important aspect of the interface. The system is designed using responsive web technologies, which allows it to adjust properly on different screen sizes. This ensures that the application can be used on desktops as well as other devices without major issues.

8.6 Performance Evaluation

The performance evaluation of the GoReserve system focuses on how efficiently the application responds to user actions and handles different operations. A well-performing system should be fast, stable, and able to process requests without delays or errors. During testing, the system was evaluated based on response time, system behavior, and overall user experience.

One of the key aspects of performance is **response time**. The system responded quickly to user actions such as login, searching for buses, and booking tickets. The pages loaded within a reasonable time, and there were no noticeable delays during normal usage. This indicates that

the application is optimized for basic operations and can handle typical user requests effectively.

Another important factor is **system stability**. During testing, the system did not crash or show unexpected behavior while performing different operations. Users were able to navigate through different pages, complete bookings, and perform cancellations without any issues. This shows that the system is stable and reliable under normal conditions.

The performance of the **database operations** was also satisfactory. Data such as user details, bus information, and booking records were stored and retrieved efficiently. The system was able to fetch search results and booking history quickly, which improved the overall user experience.

Since the system was tested in a controlled environment, it performed well for a moderate number of users. However, performance under heavy load conditions, such as multiple users accessing the system simultaneously, was not tested due to project limitations.

8.7 Database Performance

The database plays a crucial role in the GoReserve system, as it is responsible for storing and managing all the data related to users, buses, bookings, and payments. The performance of the database was evaluated based on how efficiently it handles data storage, retrieval, and updates during system operations.

During testing, the database performed efficiently in handling different types of queries. When users searched for buses, the system was able to quickly retrieve matching records from the database and display the results without noticeable delay. This shows that the database is well-structured and supports fast data access.

The insertion of data, such as user registration details and booking records, was also handled smoothly. When a user completed a booking, the system successfully stored all relevant information in the database without any errors. Similarly, updates such as booking cancellation and payment status changes were processed correctly and reflected immediately in the system.

The database design, which uses relational tables with proper relationships, helped in maintaining data consistency and integrity. Foreign key relationships ensured that data was linked correctly between different tables, such as users, bookings, and payments.

Another important aspect is data retrieval. Features like booking history and admin monitoring rely on retrieving stored data. During testing, these operations were performed efficiently, and the system was able to display records quickly.

Since the system was tested with a moderate amount of data, the database performed well under normal conditions. However, performance under very large datasets or heavy user load was not evaluated due to project limitations.

8.8 Security Evaluation

Security is an important aspect of any web-based application, especially when it involves user data and booking information. In the GoReserve system, basic security measures are implemented to ensure that user data is protected and only authorized users can access specific features of the system.

One of the main security features in the system is **user authentication**. Users are required to log in using valid credentials before accessing the system. The login process verifies the user's email and password, and access is granted only if the credentials are correct. This prevents unauthorized users from entering the system.

The system also uses **role-based access control**, where different levels of access are provided to users and administrators. Normal users can perform operations such as searching for buses, booking tickets, and viewing their booking history. On the other hand, admin functionalities such as managing buses and monitoring users are restricted only to admin accounts. This ensures that sensitive operations are not accessible to unauthorized users.

Another important security feature is **password protection**. User passwords are encrypted using BCrypt before being stored in the database. This means that even if the database is accessed, the actual passwords are not visible, which improves data security.

The system also includes **basic validation checks** to prevent incorrect or harmful inputs. Input fields are validated to ensure that only proper data is accepted, which helps in reducing potential security risks.

Although the system provides essential security features, advanced security measures such as encryption of all data transmissions, penetration testing, and protection against complex attacks were not implemented due to project scope limitations.

8.9 Result Summary

The results obtained from the development and testing of the GoReserve system show that the application is functioning successfully and meets the objectives defined at the beginning of the project. All major features of the system, including user registration, login, bus search, ticket booking, payment processing, and booking cancellation, were implemented and tested effectively.

The system produced accurate outputs for valid inputs and handled invalid inputs properly by displaying appropriate error messages. This indicates that the validation and error handling mechanisms are working correctly. The integration between different modules was also smooth, allowing users to perform complete operations without any issues.

The user interface was found to be simple and easy to use, which improves the overall user experience. Users were able to navigate through different sections of the system without confusion. The system also showed good performance in terms of response time and stability during testing.

From a database perspective, data storage and retrieval were handled efficiently, ensuring that all records are maintained correctly. Security features such as authentication and role-based access control were also implemented successfully.

8.10 Discussion

The development and evaluation of the GoReserve system show that it successfully addresses many of the problems associated with traditional reservation methods. Earlier systems were time-consuming, required manual effort, and lacked accessibility. In contrast, the proposed

system provides a simple and efficient online platform that allows users to perform booking operations easily from anywhere.

One of the key strengths of the system is its user-friendly design. The interface is simple and easy to understand, which makes it accessible even for users with basic technical knowledge. The step-by-step process of searching for buses, booking tickets, and managing bookings reduces confusion and improves the overall experience.

Another important aspect is the automation of the booking process. Tasks that were previously done manually are now handled by the system, which reduces human errors and improves accuracy. The system also ensures proper data management by storing all information in a structured database.

The implementation of features such as booking history and cancellation improves transparency and gives users more control over their bookings. Similarly, the admin module helps in managing system data efficiently, which is important for maintaining system performance.

However, the system also has some limitations. It does not include advanced features such as real-time seat selection, live payment gateway integration, or performance testing under heavy user load. These limitations are mainly due to the scope and time constraints of the project.

Despite these limitations, the system achieves its main goal of providing a functional and user-friendly reservation platform. It demonstrates how web technologies can be used to improve traditional systems and make services more accessible and efficient.

Chapter 9

Conclusion and Future Scope

9.1 Conclusion:

The GoReserve project was developed with the aim of creating a simple, efficient, and user-friendly bus reservation system that overcomes the limitations of traditional booking methods. Throughout the development process, the focus was on designing a system that reduces manual effort, improves accessibility, and provides a smooth booking experience for users.

The system successfully implements all the major functionalities required for an online reservation platform. Users can easily register, log in, search for available buses, book tickets, and manage their bookings. The inclusion of features such as booking history and cancellation improves user convenience and transparency. From the administrative side, the system provides tools to manage bus details and monitor system activities effectively.

The use of modern technologies such as Spring Boot, MySQL, and Thymeleaf helped in building a structured and reliable system. The implementation of the MVC architecture ensured proper separation of concerns, making the system organized and easier to maintain. Security features such as authentication and password encryption further enhanced the reliability of the application.

During testing, all major functionalities were verified, and the system performed as expected. The outputs generated were accurate, and the system handled both valid and invalid inputs effectively. The overall performance of the system was satisfactory, with fast response times and stable behavior under normal conditions.

Although the system has some limitations, it successfully achieves the main objectives of the project. It demonstrates how a web-based solution can improve traditional reservation systems by making them more efficient, accessible, and user-friendly.

In conclusion, the GoReserve system is a functional and practical implementation of an online bus reservation platform. It provides a strong foundation for future enhancements and can be further improved to meet real-world requirements on a larger scale.

9.2 Limitations of the Project:

Although the GoReserve system successfully implements the core functionalities of an online bus reservation platform, there are certain limitations due to the scope and time constraints of the project.

One of the main limitations is that the system uses a **simulated payment process** instead of integrating a real payment gateway. While this is sufficient for demonstration purposes, it does not fully represent real-world transaction handling. In a practical system, secure online payment integration would be necessary.

Another limitation is the absence of a **real-time seat selection feature**. The system allows booking based on available data, but users cannot visually select specific seats. This feature is commonly found in modern reservation systems and would improve user experience.

The system is also tested under **limited conditions**, mainly using a single machine and a moderate amount of data. It has not been tested for high traffic or multiple users accessing the system simultaneously, which may affect performance in real-world scenarios.

In addition, the user interface, although simple and functional, does not include advanced design elements or enhanced interactivity. Features such as animations, real-time notifications, or mobile optimization could further improve usability.

Lastly, the system does not include advanced features such as route optimization, dynamic pricing, or integration with external APIs. These features could make the system more powerful and closer to real-world applications.

Despite these limitations, the project successfully demonstrates the core concepts of a bus reservation system and provides a solid base for future improvements.

9.3 Future Enhancements

Although the GoReserve system successfully implements the basic functionalities of a bus reservation platform, there are several areas where the system can be further improved and expanded in the future. These enhancements can make the system more advanced, efficient, and suitable for real-world applications.

One of the major improvements that can be made is the integration of a **real-time payment gateway**. Currently, the system uses a simulated payment process, but in the future, secure payment methods such as UPI, debit/credit cards, and digital wallets can be integrated. This will make the system more practical and ready for actual deployment.

Another important enhancement is the implementation of a **seat selection feature**. This would allow users to view the seating layout of the bus and select their preferred seats. It would improve user experience and bring the system closer to modern booking platforms.

The system can also be extended by developing a **mobile application**. With the increasing use of smartphones, having a mobile app would make the system more accessible and convenient for users. It would allow users to book tickets anytime and anywhere with ease.

In addition, features such as **real-time notifications** can be added. Users can receive updates about booking confirmations, cancellations, and travel reminders through email or SMS. This would improve communication and user engagement.

Performance improvements can also be considered in the future. The system can be optimized to handle a large number of users simultaneously by implementing load balancing and better database management techniques.

Another possible enhancement is the addition of **multi-language support**, which would make the system accessible to users from different regions. This would increase usability and reach a wider audience.

Advanced features such as **route optimization, dynamic pricing, and integration with external APIs** can also be included to make the system more intelligent and efficient.

References / Bibliography

References

1. Spring Boot Official Documentation, “Spring Boot Reference Guide.” [Online]. **Available: https://spring.io/projects/spring-boot**
2. MySQL Official Documentation, “MySQL 8.0 Reference Manual.” [Online]. **Available: https://dev.mysql.com/doc/**
3. Thymeleaf Official Documentation, “Thymeleaf Documentation.” [Online]. **Available: https://www.thymeleaf.org/documentation.html**
4. Spring Security Official Documentation, “Spring Security Reference.” [Online]. **Available: https://spring.io/projects/spring-security**
5. Hibernate Documentation, “Hibernate ORM Guide.” [Online]. Available: **https://hibernate.org/orm/documentation/**
6. Bootstrap Documentation, “Bootstrap Documentation.” [Online]. Available: **https://getbootstrap.com/docs/**
7. Java SE Oracle, “Java Platform Standard Edition Documentation.” [Online]. **Available: https://docs.oracle.com/javase/**
8. TutorialsPoint, “Spring Boot Tutorial.” [Online]. Available: **https://www.tutorialspoint.com/spring_boot**
9. GeeksforGeeks, “Web Development and Database Concepts.” [Online]. Available: **https://www.geeksforgeeks.org**
10. Craig Walls, *Spring in Action*, 6th ed., Manning Publications, 2022.
11. Herbert Schildt, *Java: The Complete Reference*, 11th ed., McGraw-Hill, 2018.

Appendices

Sample Codes:

Appendix A - Controller Layer:

```java
package JFS6WDE.OnlineBusTicketBooking.Controller;
import java.util.List;

@Controller
public class RouteController {

    @Autowired
    private BusServiceImpl busService;

    @GetMapping("/showRoute")
    public String showSearchBusPage(Model model) {
        model.addAttribute("pageTitle", "Search Buses - GoReserve");
        model.addAttribute("body", "showroute");
        model.addAttribute("fullWidth", true);
        return "layout";
    }

    @GetMapping("/searchBus")
    public String searchBus(@RequestParam("routeFrom") String routeFrom,
                            @RequestParam("routeTo") String routeTo,
                            Model model) {
        List<Bus> buses = busService.findByRouteFromAndRouteTo(routeFrom, routeTo);
        model.addAttribute("buses", buses);
        model.addAttribute("pageTitle", "Search Results - GoReserve");
        model.addAttribute("body", "showroute");
        model.addAttribute("fullWidth", true);
        return "layout";
    }
}
```

Appendix B - Entity Layer:

```java
package JFS6WDE.OnlineBusTicketBooking.Entities;

import jakarta.persistence.*;

@Setter
@Getter
@NoArgsConstructor
@AllArgsConstructor
@Entity
@Table(name="roles")
public class Role
{
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable=false, unique=true)
    private String name;

    @ManyToMany(mappedBy="roles")
    private List<User> users;

    // @ManyToMany(mappedBy="roles")
    // private List<Admin> admins;
}
```

Appendix C - Repository Layer:

```java
package JFS6WDE.OnlineBusTicketBooking.Repository;

import java.util.List;

public interface BookingRepository extends JpaRepository<Booking, Long> {

    List<Booking> findByUserAndBus(User user, Bus bus);

    List<Booking> findByUser(User user);

    List<Booking> findByBus(Bus bus);

    @Query("SELECT SUM(b.bookseat) FROM Booking b WHERE b.bus = :bus")
    Integer countBookedSeatsByBus(@Param("bus") Bus bus);

    // ⊟ Show only paid bookings for current user (used for history)
    @Query("SELECT b FROM Booking b WHERE b.user.email = :email AND b.payment.paymentStatus = 'PAID'")
    List<Booking> findPaidBookingsByUserEmail(@Param("email") String email);

    // ⊟ Optional: show all bookings (paid + cancelled)
    @Query("SELECT b FROM Booking b WHERE b.user.email = :email ORDER BY b.bookingDate DESC, b.bookingTime DESC")
    List<Booking> findAllBookingsByUserEmail(@Param("email") String email);

    List<Booking> getPaidBookingsByUserEmail(String email);

}
```

Appendix D - Service Layer:

```java
package JFS6WDE.OnlineBusTicketBooking.Services;

import java.util.List;

public interface BookingService {

    List<Booking> getAllBookings(); // renamed for plural consistency

        void deleteBookingById(long id);

        Booking getBookingById(long id);

        // Save a new booking and return its ID
        Long saveBooking(BookingDto bookingDto, String userEmail, long busId);

        void updateBookingWithPayment(Booking booking);

        List<Booking> getPaidBookingsByUserEmail(String email);

        void cancelBooking(Long bookingId);

        void saveUpdatedBooking(Booking booking);
}
```

Appendix E - HTML Template:

```
<!DOCTYPE html>
<html lang="en"
    xmlns:th="http://www.thymeleaf.org"
>
<head>
    <meta charset="UTF-8">
    <title>Registration and Login System</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/css/bootstrap.min.css"
          rel="stylesheet"
          integrity="sha384-EVSTQN3/azprG1Anm3QDgpJLIm9Nao0Yz1ztcQTwFspd3yD65VohhpuuCOmLASjC"
          crossorigin="anonymous">
</head>
<body>
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <div class="container-fluid">
        <a class="navbar-brand" th:href="@{/index}">Registration and Login System</a>
        <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navba
            <span class="navbar-toggler-icon"></span>
        </button>
        <div class="collapse navbar-collapse" id="navbarSupportedContent">
            <ul class="navbar-nav me-auto mb-2 mb-lg-0">
                <li class="nav-item">
                    <a class="nav-link active" aria-current="page" th:href="@{/logout}">Logout</a>
                </li>
            </ul>
        </div>
    </div>
</nav>
<div class="container">
    <div class="row col-md-10">
        <h2>List of Registered Users</h2>
    </div>
    <table class="table table-bordered table-hover">
        <thead class="table-dark">
            <tr>
                <th>First Name</th>
                <th>Last Name</th>
                <th>Email</th>
            </tr>
```

Appendix: Additional Documentation:

CLASS DIAGRAM: